



Politechnika Warszawska

Wydział Matematyki
i Nauk Informatycznych

Praca dyplomowa inżynierska

na kierunku Informatyka i Systemy Informatyczne

Interaktywna platforma e-learningowa do nauki algorytmów i struktur danych

Wojciech Łapan

Numer albumu 327374

Urszula Obirek

Numer albumu 327386

Andrzej Kupiec

Numer albumu 327370

promotor

dr inż. Paweł Kotowski

WARSZAWA 2026

Streszczenie

Interaktywna platforma e-learningowa do nauki algorytmów i struktur danych

Praca przedstawia projekt interaktywnej aplikacji e-learningowej, wspomagającej naukę algorytmów i struktur danych. System umożliwia rozwiązywanie zadań programistycznych o różnym poziomie trudności, automatyczne sprawdzanie poprawności rozwiązań przy użyciu testów, analizę wydajności kodu oraz dostęp do materiałów teoretycznych i wizualizacji działania algorytmów. Aplikacja ma na celu łączyć wiedzę teoretyczną z umiejętnościami praktycznymi, poprzez udostępnianie dedykowanych kursów przechowujących w sobie lektury i zadania powiązane z wybranym tematem.

System jest aplikacją webową działającą w przeglądarce internetowej, co pozwala na łatwy dostęp z dowolnego urządzenia bez konieczności instalacji dodatkowego oprogramowania. Wykonywanie kodu odbywa się na specjalnym kontenerze z odpowiednimi zabezpieczeniami i ograniczeniami, aby uchronić dane użytkowników i zabezpieczyć aplikację przed groźnymi kodami.

Platforma jest zrobiona w sposób przejrzysty dla użytkownika, umożliwiając łatwe korzystanie i szybkie opanowanie funkcjonalności zarówno dla administratorów, jak również dla zwykłych użytkowników.

Słowa kluczowe: Aplikacja e-learningowa, nauka programowania, wykonywanie kodu, analiza wydajności, algorytmy i struktury danych, interaktywność

Abstract

Interaktywna platforma e-learningowa do nauki algorytmów i struktur danych

The thesis presents a project for an interactive e-learning application that supports the learning of algorithms and data structures. The system enables users to solve programming tasks of varying difficulty, automatically check the correctness of solutions using tests, analyse code performance, and access theoretical materials and visualisations of algorithm operation. The application aims to combine theoretical knowledge with practical skills by providing dedicated courses containing readings and tasks related to a selected topic.

The system is a web application that runs in a web browser, allowing easy access from any device without the need to install additional software. The code is executed on a special container with appropriate security measures and restrictions to protect user data and secure applications against malicious code.

The platform is designed to be user-friendly, allowing for easy use and quick mastery of its functionality for both administrators and regular users.

Keywords: E-learning application, programming education, code execution, performance analysis, algorithms and data structures, interactivity

Spis treści

1. Wstęp	11
1.1. Geneza i znaczenie problemu	11
1.2. Motywacja	12
1.3. Cel i zakres prac	13
1.4. Podział pracy	14
2. Wymagania funkcjonalne i нефункционалне	15
2.1. Wymagania funkcjonalne – przypadki użycia	15
2.2. Wymagania нефункционалне	18
3. Analiza istniejących rozwiązań	19
3.1. LeetCode	19
3.2. HackerRank	21
3.3. VisuAlgo	21
3.4. Strona Uniwersytetu w San Francisco	22
3.5. Podsumowanie	24
4. Architektura systemu	25
4.1. Struktura komponentów	25
4.1.1. Serwer API	26
4.1.2. Aplikacja internetowa	30
4.1.3. Baza danych	34
4.1.4. Serwis wykonujący kod (Code executor)	37
4.1.5. Serwis analizujący kod (Code analyzer)	37
4.1.6. Serwis obsługujący wizualizacje (Visualizer Service)	39
4.2. Infrastruktura i wdrożenie	39
4.2.1. Ogólna koncepcja wdrożenia	39
4.2.2. Konteneryzacja	39
4.3. Instrukcja wdrożenia	41
4.3.1. Wymagania wstępne	41

4.3.2.	Środowisko deweloperskie	42
4.3.3.	Środowisko produkcyjne	45
4.3.4.	Zewnętrzne zależności i utrzymanie	47
4.4.	Komunikacja między komponentami	48
4.4.1.	Proces komunikacji	48
4.4.2.	Uwierzytelnianie i autoryzacja	49
4.5.	Moduły systemu	49
4.5.1.	Moduł zarządzania użytkownikami	50
4.5.2.	Moduł kursów i materiałów edukacyjnych	50
4.5.3.	Moduł zadań programistycznych i interaktywnych	50
4.5.4.	Moduł uruchamiania i analizy kodu	51
4.5.5.	Moduł wizualizacji	51
5.	Zdalne wykonywanie kodu	53
5.1.	Możliwe sposoby implementacji	53
5.1.1.	Konteneryzacja w Dockerze	53
5.1.2.	Gotowy obraz kontenera	54
5.1.3.	Pula gotowych kontenerów	54
5.1.4.	Kontenery z serwisem wykonującym	54
5.1.5.	Roslyn – kompilator jako usługa	55
5.2.	Mechanizm działania	56
6.	Raport z testów aplikacji	63
6.1.	Testy jednostkowe	63
6.2.	Testy integracyjne	64
6.3.	Testy interfejsu użytkownika (UI)	66
6.4.	Testy end-to-end (E2E)	66
6.5.	Testy wydajnościowe	67
6.6.	Podsumowanie wyników testów	69
7.	Instrukcja obsługi platformy	70
7.1.	Proces rejestracji i logowania	70
7.1.1.	Rejestracja	70
7.1.2.	Logowanie	71
7.2.	Nawigacja	72
7.2.1.	Dashboard	73
7.2.2.	Lista kursów	73

7.2.3.	Widok kursu	74
7.2.4.	Lista wykładów	75
7.2.5.	Widok wykładu	75
7.2.6.	Lista zadań	75
7.2.7.	Widok zadania	77
7.2.8.	Widok wizualizacji	79
7.2.9.	Profil użytkownika	80
7.3.	Panel administratora	81
7.3.1.	Dodanie nowego zadania i edytowanie istniejącego	82
7.3.2.	Dodanie nowego wykładu i edytowanie istniejącego	84
7.3.3.	Monitorowanie komentarzy	85
7.3.4.	Monitorowanie zgłoszeń rozwiązań	85
8.	Podsumowanie	87
8.1.	Ewolucja koncepcji systemu	87
8.2.	Osiągnięcia projektu	88
8.3.	Kierunki dalszego rozwoju	89
8.4.	Wnioski	90

1. Wstęp

1.1. Geneza i znaczenie problemu

Nauka programowania uznawana jest za proces trudny i wymagający, jednak w ostatnich latach obserwuje się znaczący wzrost popularności oraz dostępności narzędzi i materiałów edukacyjnych. Powstają języki programowania, które nie wymagają od użytkowników złożonej wiedzy na temat zarządzania pamięcią, a także środowiska programistyczne, których uruchomienie nie wymaga długiego przygotowania ani specjalistycznej wiedzy. Także materiały naukowe przechodzą transformację i ich zakres ciągle poszerzany jest o takie, które nie są dedykowane specjalistom i osobom z zaawansowaną znajomością podstaw matematycznych, a pełnym początkującym. Podstawowe programy o nieznacznym stopniu złożoności mogą być obecnie pisane również przez osoby bez formalnego wykształcenia w tej dziedzinie. Pojawiają się różne opinie na temat tego, czy programowanie jako całość staje się coraz prostsze – istnieje więcej języków programowania, więcej frameworków, stopień skomplikowania tworzonych systemów jest zazwyczaj wyższy – ale niewątpliwie sam próg wejścia dla kogoś zupełnie początkującego jest dużo niższy.

A mimo to, nawet jeśli język programowania będzie dla użytkownika najbardziej intuicyjny, wciąż nie zbliży się do języka naturalnego i wciąż, bez odpowiedniego przygotowania, początkujący programista nie będzie w stanie rozwiązać trudniejszych problemów. O ile proste koncepcyjnie programy można tworzyć, kierując się jedynie intuicją i zdrowym rozsądkiem, to w końcu nieuniknione jest zetknięcie się z barierą. Zdobycie wiedzy teoretycznej z zakresu programowania wciąż pozostaje kluczowym elementem, którego nie da się ominąć.

Zmianie ulega za to sposób, w jaki jest ona przekazywana ze względu na upowszechnienie narzędzi cyfrowych i platform internetowych.

Głównymi źródłami naukowymi wciąż pozostają książki, prace naukowe i podręczniki, szczególnie w środowiskach akademickich. Mimo to, początkujący programiści coraz częściej sięgają po inne rozwiązania. Od wielu lat materiały edukacyjne publikowane są również w przestrzeni internetowej, ale zakres prezentowanej tam wiedzy i same metody prezentacji wciąż się zmieniają.

Czymś, co również zyskuje na popularności, są platformy e-learningowe. Badania przeprowadzone wśród studentów uczelni wyższych wyraźnie wskazują na to, że platformy e-learningowe

są postrzegane jako przydatne narzędzia wspierające proces uczenia się, a ich popularność wynika m.in. z wygody użytkowania, interaktywności oraz możliwości samodzielnego zarządzania tempem nauki [11]. Dostęp na dowolnym urządzeniu, często również mobilnym, przyciąga wiele osób, które nie mają możliwości lub środków, by kupić podręczniki i udać się na kursy. Nie potrzeba też nikogo, kto bezpośrednio przekazywałby wiedzę konkretnemu uczniowi. Pod wieloma względami jest to więc rozwiązanie dużo bardziej atrakcyjne od tych tradycyjnych.

Przyjrzenie się tego typu platformy w kontekście programowania może prowadzić do zauważenia problemu – wiele istniejących narzędzi oferuje albo jedynie wiedzę teoretyczną, albo jedynie środowisko do rozwiązywania problemów programistycznych oraz ich bazę. Pojawia się więc podział między wiedzą teoretyczną i praktyczną.

1.2. Motywacja

Do wyboru niniejszej pracy przyczyniło się kilka powodów, a najważniejsze z nich zostaną omówione poniżej.

Po pierwsze rosnąca popularność nauki programowania sprawia, że grupa użytkowników, którzy chcieliby skorzystać z dedykowanych narzędzi, ciągle rośnie. Ze względu na to potencjalna grupa odbiorcza tworzonej platformy, w najbliższych latach prawdopodobnie będzie rosnąć. Z całej tej grupy, opisana w niniejszej pracy aplikacja webowa dedykowana jest przede wszystkim studentom kierunków informatycznych. Ze względu na to jej zakres tematyczny został zawężony do *algorytmów i struktur danych*. Jest to dziedzina, z którą zapoznanie się – zarówno w teorii, jak i w praktyce – konieczne jest w toku studiów informatycznych, stanowiąc podstawy do dalszej nauki.

Drugim powodem do wyboru tematu pracy był fakt, że istniejące platformy e-learningowe rzadko oferują kompletną integrację materiałów teoretycznych, zadań programistycznych oraz wizualizacji algorytmów w jednym, spójnym środowisku. Dodatkową motywacją była chęć stworzenia platformy, która wyróżni się spośród tych istniejących już na rynku, nie tylko swoim poziomem złożoności. Za cel obrano stworzenie narzędzia, które odpowiadałoby na potrzeby współczesnych użytkowników: nowoczesnego, intuicyjnego i interaktywnego.

1.3. Cel i zakres prac

Celem niniejszej pracy jest zaprojektowanie i implementacja AlgoRhythm – interaktywnej platformy e-learningowej do nauki algorytmów i struktur danych. Jako technologie do stworzenia tej platformy wybrano: ASP.NET Core (serwer API), React z Typescript (aplikacja internetowa) oraz Microsoft SQL Server 2022 (baza danych). Całość postanowiono wdrożyć za pomocą środowiska Microsoft Azure, stosując konteneryzację – Docker. Na podstawie badań, obserwacji i analizy, stworzone zostały początkowe założenia projektu, które zdefiniowały dalszy przebieg prac. Jako że czas pracy był z góry ograniczony, kluczowym było określenie priorytetów, zdecydowanie, które funkcjonalności należy uznać za najbardziej istotne i jakimi zasadami należy się kierować tworząc aplikację webową. Podstawowe założenia dotyczące platformy zostały zdefiniowane następująco:

1. Stworzone narzędzie jest kompletne i umożliwia użytkownikowi zarówno naukę teorii, jak i sprawdzenie umiejętności w praktyce (za pomocą zadań programistycznych oraz wizualizacji). Nauka jest dla użytkownika intuicyjna.
2. Użytkownik ma możliwość robienia postępów i ich zapisu: materiały edukacyjne będzie można oznaczyć jako przeczytane, a informacje o rozwiązaniu zadania również zapiszą się w bazie danych.
3. Wykłady oraz zadania połączone są w kursy, które zbierają podobne tematycznie materiały (np. dotyczące algorytmów znajdowania najkrótszej ścieżki) i umożliwiają naukę, podczas której podąża się wyznaczoną ścieżką.
4. Kod platformy został zaprojektowany w sposób, który umożliwia jego dalszy rozwój i utrzymanie. W tym celu zastosowano modułarną strukturę projektu, utrzymywano spójne konwencje nazewnictwa oraz ograniczono zależności między komponentami.
5. Platforma dzieli się na dwa moduły ze względu na rodzaje użytkowników: moduł użytkowników i administratorów. Moduł użytkownika umożliwia mu wspomniane wcześniej rozwiązywanie zadań, przeglądanie materiałów teoretycznych oraz przechodzenie kursów. Moduł administratora z kolei odpowiada za dalszy rozwój treści umieszczanych na platformie i jej monitorowanie – administrator ma bowiem możliwość dodawania, edytowania oraz usuwania treści, a także przeglądania ostatnich aktywności na stronie.

Powyższe założenia stanowiły podstawy rozpoczęcia prac i były kluczowe przy dalszym jej tworzeniu.

1.4. Podział pracy

Andrzej Kupiec

- Implementacja systemu wizualizacji wykonania algorytmów.
- Implementacja serwisu wykonującego kod.
- Stworzenie serwisu analizującego kod.
- Konfiguracja Azure oraz deployment aplikacji na środowisko produkcyjne.
- Stworzenie testów jednostkowych i testów UI.
- Rozdziały pracy: analiza istniejących rozwiązań, zdalne wykonywanie kodu, instrukcja wdrożenia.

Wojciech Łapan

- Implementacja architektury serwera API.
- Implementacja systemu uwierzytelniania i autoryzacji oraz wysyłania e-maili.
- Stworzenie modeli danych i DTO (data transfer object).
- Stworzenie testów integracyjnych.
- Rozdziały pracy: architektura systemu, raport z testów, podsumowanie.

Urszula Obirek

- Implementacja interfejsu użytkownika i admina aplikacji webowej.
- Integracja interfejsu aplikacji z serwerem API.
- Implementacja funkcji submission dla zadań.
- Przygotowanie wstępnych danych dla bazy danych.
- Rozdziały pracy: wstęp, wymagania funkcjonalne i нефункционалне, instrukcja obsługi platformy.

2. Wymagania funkcjonalne i нефункционалне

Proces projektowania systemu rozpoczęto od dokładnej analizy wymagań, które stanowią fundament dla wszystkich kolejnych decyzji architektonicznych i implementacyjnych. Wymagania zostały podzielone na dwie główne kategorie: funkcjonalne, opisujące konkretne zachowania i możliwości systemu z perspektywy użytkowników, oraz нефункционалне, określające parametry jakościowe takie jak wydajność czy bezpieczeństwo.

2.1. Wymagania funkcjonalne – przypadki użycia

Projektując platformę, zaplanowano wprowadzenie dwóch aktorów: użytkownika aplikacji oraz administratora. Następnie, kierując się ich potrzebami, przygotowano przypadki użycia.

Użytkownik w tym przypadku jest osobą korzystającą z platformy w celu nauki i jej głównym odbiorcą. Wszyscy użytkownicy w tym rozumieniu posiadają te same uprawnienia: mogą wyświetlać zadania, wykłady oraz kursy, rozwiązywać zadania oraz korzystać z innych funkcjonalności, które mają na celu uatrakcyjnienie korzystania z platformy. Dokładnie funkcjonalności użytkowników opisane są w tabeli 2.3. Załączona jest również tabela 2.2, w której wyszczególniono wymagania powiązane z procesem autoryzacji.

Z kolei administrator jest osobą zarządzającą treściami, które pojawiają się na platformie i monitorującą aktywności. Przypadki użycia dotyczące administratora rozpisano szczegółowo w tabeli 2.1.

Tabela 2.1: Przypadki użycia – aktor: administrator

Aktor	Nazwa	Opis	Reakcja systemu
Administrator	Zarządzanie treściami	Administrator dodaje, edytuje i usuwa treści (wykłady, kursy, zadania) zamieszczone na platformie.	System zarządza procesem zmiany treści i udostępnia administratorowi proste formularze.
Administrator	Zarządzanie uprawnieniami	Administrator ma możliwość odebrania lub przyznania wybranym użytkownikom roli administratora.	System umożliwia zmianę uprawnień.
Administrator	Monitorowanie aktywności	Administrator wyświetla zamieszczone na platformie komentarze oraz wyniki zgłoszonych rozwiązań przez użytkowników.	System wyświetla listy komentarzy oraz zgłoszeń i zapewnia możliwość ich filtrowania.

Tabela 2.2: Przypadki użycia – aktor: użytkownik, autoryzacja

Aktor	Nazwa	Opis	Reakcja systemu
Użytkownik	Rejestracja	W celu utworzenia nowego konta, użytkownik podaje wymagane dane rejestracyjne	System tworzy konto użytkownika i wysyła kod weryfikacyjny na adres e-mail. Po podaniu kodu, użytkownik zostaje zalogowany.
Użytkownik	Logowanie	Użytkownik podaje swój adres e-mail oraz hasło w celu zalogowania się do systemu.	System weryfikuje wpisane dane i przyznaje dostęp do aplikacji.
Użytkownik	Logowanie za pomocą konta Google	Użytkownik loguje się, korzystając z konta Google, bez przechodzenia przez proces rejestracji w aplikacji.	System poprawnie łączy się z API Google'a i przyznaje użytkownikowi dostęp do systemu.
Użytkownik	Wylogowanie	Użytkownik postanawia wylogować się z sesji.	System zakańcza sesję i przekierowuje użytkownika na stronę logowania.

2.1. WYMAGANIA FUNKCJONALNE – PRZYPADKI UŻYCIA

Tabela 2.3: Przypadki użycia – aktor: użytkownik, treści

Aktor	Nazwa	Opis	Reakcja systemu
Użytkownik	Przeglądanie opublikowanych materiałów	Użytkownik w łatwy sposób przegląda dostępne wykłady, zadania oraz kursy.	System wyświetla listy zadań, wykładów oraz kursów w przejrzysty sposób.
Użytkownik	Filtrowanie treści	Użytkownik filtruje wykłady po tagach, a zadania po tagach, trudności i kursach, do których należą.	System wyświetla przefiltrowane treści.
Użytkownik	Przechodzenie kursów	Użytkownik może wyświetlić informacje o swoich postępach w kursach.	System automatycznie nalicza postępy użytkownika na podstawie rozwiązanych zadań i przeczytanych wykładów.
Użytkownik	Materiały edukacyjne	Użytkownik zapoznaje się z treściami wykładowymi.	System wyświetla treści, składające się z tekstu, zdjęć i krótkich nagrań.
Użytkownik	Zadania programistyczne	Użytkownik rozwiązuje zadania programistyczne i zapoznaje się z wynikami swojego rozwiązania.	System wykonuje kod użytkownika, sprawdza, czy zwraca poprawne wyniki i wyświetla informację użytkownikowi.
Użytkownik	Podpowiedzi	Użytkownik może poprosić o wskazówkę do aktualnego zadania	System wyświetla listę dostępnych podpowiedzi.
Użytkownik	Otrzymywanie osiągnięć	Użytkownik zdobywa osiągnięcia po osiągnięciu celów (np. gdy przeczytał 10 wykładów).	System przyznaje osiągnięcia i wyświetla powiadomienia o nich.
Użytkownik	Wizualizacje	Użytkownik ma możliwość napisania algorytmu korzystającego z grafu i uruchomienia jego wizualizacji.	System w przejrzysty sposób przedstawia kolejne kroki algorytmu.
Użytkownik	Komentarze	Użytkownik pisze, edytuje oraz usuwa komentarze pod zadaniami.	System zarządza komentarzami i wyświetla je użytkownikom w specjalnej zakładce.

2.2. Wymagania niefunkcjonalne

Wyznaczono również wymagania niefunkcjonalne, które musi spełnić finalna wersja platformy. Są to następująco:

1. Bezpieczeństwo

Platforma zapewnia bezpieczeństwo przechowywania danych użytkownika, szczególnie jeśli chodzi o hasła, które są szyfrowane. Poza tym komunikacja z serwerem odbywa się za pomocą bezpiecznego protokołu (HTTPS).

2. Skalowalność

System zaprojektowany jest w taki sposób, żeby w przyszłości mógł być w łatwy sposób rozwijany i rozszerzany. Takie zmiany nie powinny wymagać modyfikowania istniejących funkcjonalności. Liczba kontenerów wykonujących kod jest łatwo konfigurowalna, by zapewnić możliwość obsługi większej liczby użytkowników.

3. Niezawodność

System cechuje stabilność, wykonuje poprawnie podstawowe operacje, jak na przykład logowanie, przeglądanie treści oraz wyświetlanie wyników. Jeśli wystąpi błąd, użytkownik otrzymuje jasny komunikat o nim. Kontenery automatycznie restartują się po wystąpieniu błędu, aby zapewnić jak największą dostępność.

4. Wydajność

Aplikacja powinna działać płynnie, nawet jeśli jednocześnie korzysta z niej kilku użytkowników. Strony ładują się sprawnie, w krótkim czasie, nieprzekraczającym kilku sekund.

5. Użyteczność

Aplikacja jest przejrzysta, a korzystanie z niej intuicyjne – nie wymaga dodatkowych poradników. Użytkownik z łatwością porusza się po platformie i odnajduje interesujące go treści.

3. Analiza istniejących rozwiązań

Istnieje wiele aplikacji i stron internetowych, które wspierają naukę zagadnień związanych z algorytmami i strukturami danych. Każda z nich skupia się na nieco innych funkcjonalnościach i obejmuje inny zakres. Niektóre skupiają się na materiałach teoretycznych, inne oferują – w różnym stopniu rozbudowane – zadania programistyczne i tym samym wykonywanie kodu napisanego przez użytkownika. Część z nich obejmuje również wizualizacje wybranych algorytmów, często grafowych, czasem też i innych.

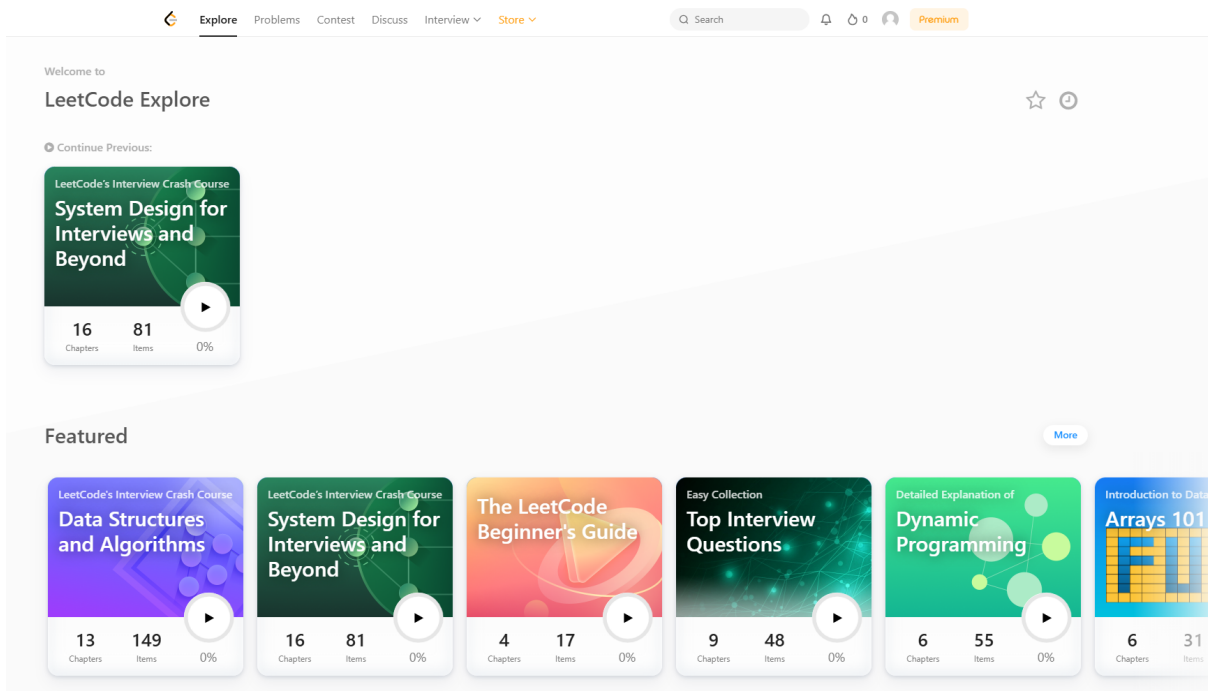
Pomimo dużej liczby dostępnych rozwiązań, wiele z nich skupia się na pojedynczych funkcjonalnościach. Tym samym brakuje rozwiązań kompleksowych, które przeprowadzałyby użytkownika przez cały proces nauki: zaczynając od wiedzy teoretycznej, przez zadania praktyczne, aż po wizualizacje. Drugim istotnym ograniczeniem wielu istniejących platform jest fakt, że często dostęp do kluczowych funkcjonalności lub bardziej rozbudowanych materiałów oferowany jest jedynie w płatnej wersji.

3.1. LeetCode

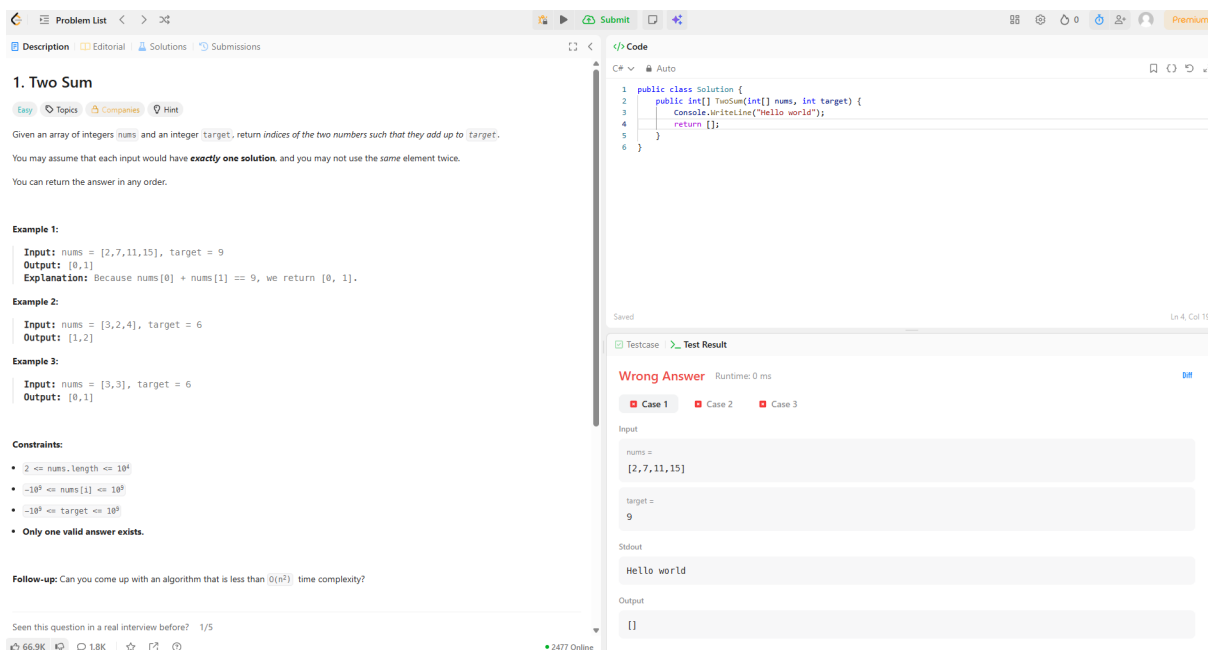
Do najbardziej popularnych stron do nauki programowania należy platforma LeetCode. Udośćpnia ona jeden z największych zbiorów zadań dostępnych w internecie. Umożliwia naukę różnorodnych zagadnień przede wszystkim przez rozwiązywanie zadań programistycznych w praktycznie dowolnym języku, od PHP, przez C#, aż po Go.

Nie ma na niej jednak możliwości zaznajomienia się z teorią w bezpłatnej wersji, a kursy (widoczne na rys. 3.1) potrafią kosztować nawet kilkadziesiąt dolarów. Wiele funkcji jest dodatkowo płatnych, w tym część zadań. Edytor kodu (rys. 3.2) w darmowej wersji nie podkreśla na bieżąco błędów, nie ma też możliwości podpowiedzi pisanych słów. Tematyka jest bardzo różnorodna, jednak część zagadnień, zwłaszcza teoretycznych, nie zostało uwzględnionych.

3. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ



Rysunek 3.1: Widok głównej strony LeetCode



Rysunek 3.2: Widok zadania programistycznego strony LeetCode

Tym, co wyróżnia platformę LeetCode oprócz dużej bazy zadań, są również rankingi użytkowników, które umożliwiają rywalizację. Na platformie znajduje się również rozbudowana przestrzeń do różnorodnych dyskusji. Jeśli za to chodzi o opcje wizualizacji, są one jednak bardzo ubogie. Jak widać na rys. 3.3, grafy przedstawione są w bardzo uproszczony sposób.

3.2. HACKERRANK



The screenshot shows the LeetCode web interface. On the left, a C++ code editor contains a program for constructing a binary tree from a string input. The code defines a `TreeNode` struct and functions like `stringToTreeNode` and `prettyPrintTree`. The `main` function reads input lines and prints the tree structure. On the right, the 'Output' panel shows the result: 'Finished in 0 ms' and a visual representation of a binary tree with root 1, left child 2, and right child 3.

```
1- /**
2-  * Definition for a binary tree node.
3-  * struct TreeNode {
4-  *     int val;
5-  *     TreeNode *left;
6-  *     TreeNode *right;
7-  *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
8-  * };
9-  */
10-
11- string treeNodeToString(TreeNode* root) {**}
12-
13- void trimLeftTrailingSpaces(string &input) {**}
14-
15- void trimRightTrailingSpaces(string &input) {**}
16-
17- TreeNode* stringToTreeNode(string input) {**}
18-
19- void prettyPrintTree(TreeNode* node, string prefix = "", bool isLeft = true) {**}
20-
21- int main() {
22-     string line;
23-     while (getline(cin, line)) {
24-         TreeNode* root = stringToTreeNode(line);
25-         prettyPrintTree(root);
26-     }
27-     return 0;
28- }
```

Rysunek 3.3: Widok wizualizacji na stronie LeetCode

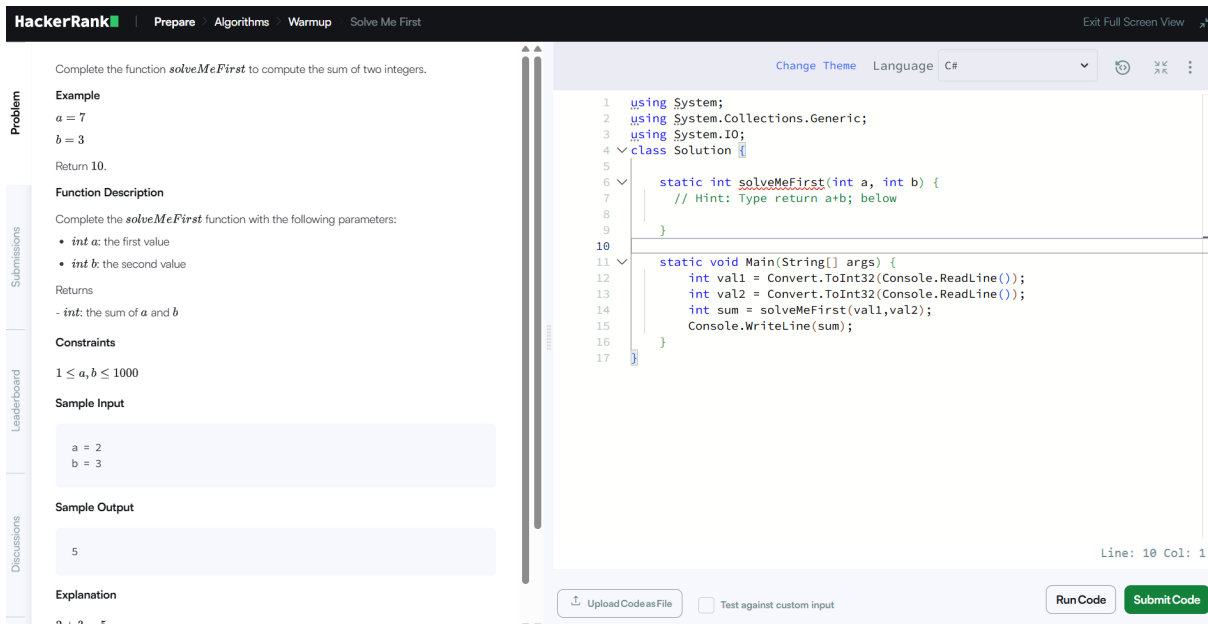
3.2. HackerRank

Drugą znaną platformą do nauki programowania jest strona HackerRank. Skupia się ona przede wszystkim na nauce do rozmów rekrutacyjnych. W porównaniu z poprzednią omawianą platformą, oferuje ona mniej funkcjonalności i nie udostępnia żadnej możliwości wizualizacji. Także i tutaj wiele użyteczności jest dodatkowo płatnych. Co warto zaznaczyć, platformę wyróżnia fakt, że edytor kodu działa w czasie rzeczywistym, podkreślając błędy i podpowiadając następne słowa (Rys. 3.4). Materiały teoretyczne w bezpłatnej wersji, podobnie jak na poprzedniej platformie, są mocno ograniczone.

3.3. VisuAlgo

VisuAlgo to narzędzie, które trafia do innej kategorii, niż poprzednie – nie służy bowiem do rozwiązywania zadań programistycznych, zamiast tego skupia się na aspekcie wizualizacji. Zostało zaprojektowane przez profesora Narodowego Uniwersytetu w Singapurze i jest jedną z najbardziej popularnych stron o tej tematyce. Zawiera ona duży zbiór zdefiniowanych wcześniej interaktywnych animacji (Rys. 3.5 oraz 3.6), tłumaczących kolejne kroki na podstawie komentarzy i pseudokodu. Przed każdą z nich znajduje się też krótkie wprowadzenie.

3. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ



Rysunek 3.4: Widok zadania programistycznego na stronie HackerRank

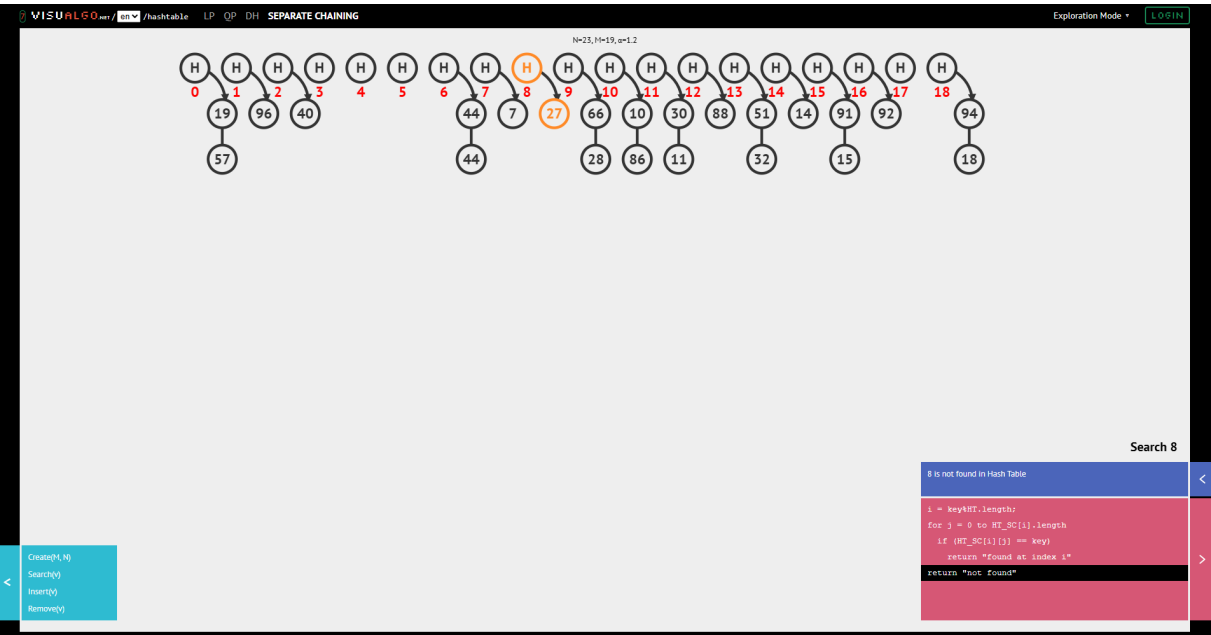


Rysunek 3.5: Widok części z animacji dostępnych na stronie VisuAlgo

3.4. Strona Uniwersytetu w San Francisco

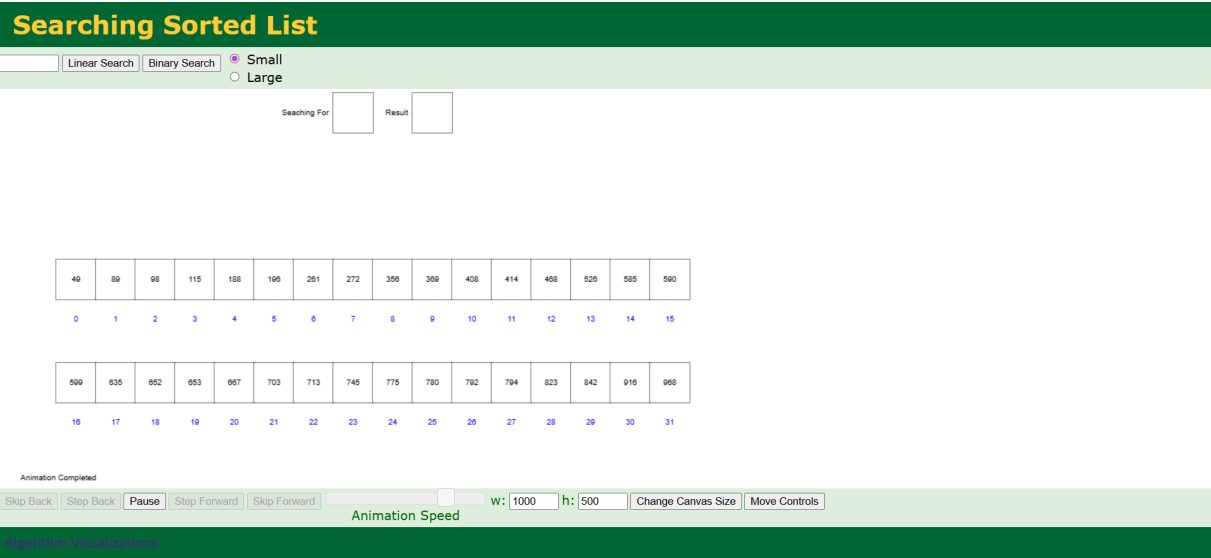
Wiele animacji jest też dostępnych na stronie *Data Structure Visualization* Uniwersytetu w San Francisco. Znaleźć tam można szeroki zbiór dostępnych interaktywnych animacji, prawie całkowicie kontrolowanych przez użytkownika. Nie można jednak na niej znaleźć materiałów

3.4. STRONA UNIwersYTETU W SAN FRANCISCO



Rysunek 3.6: Widok animacji na stronie VisuAlgo

teoretycznych, a sam interfejs, choć rozbudowany, nie jest tak przyjazny użytkownikom jak poprzednie platformy. Przykładowy widok animacji zaprezentowany jest na rysunku 3.7.



Rysunek 3.7: Widok animacji na stronie Uniwersytetu w San Francisco

3.5. Podsumowanie

W internecie istnieje znacząca liczba aplikacji do nauki programowania, w rozdziale tym omówiono tylko część najpopularniejszych z nich. Mało jest jednak narzędzi kompleksowych, które w bezpłatnej ofercie zawierałyby jednocześnie przyjazny użytkownikowi interfejs, interaktywność, a także możliwość pisania kodu. Rozwiązanie powstałe w ramach tej pracy stanowi podwaliny do platformy łączącej ze sobą jednocześnie możliwość nauki teorii i praktyki, czyniąc informatykę, a zwłaszcza dziedzinę algorytmów i struktur danych, bardziej przystępną i zrozumiałą.

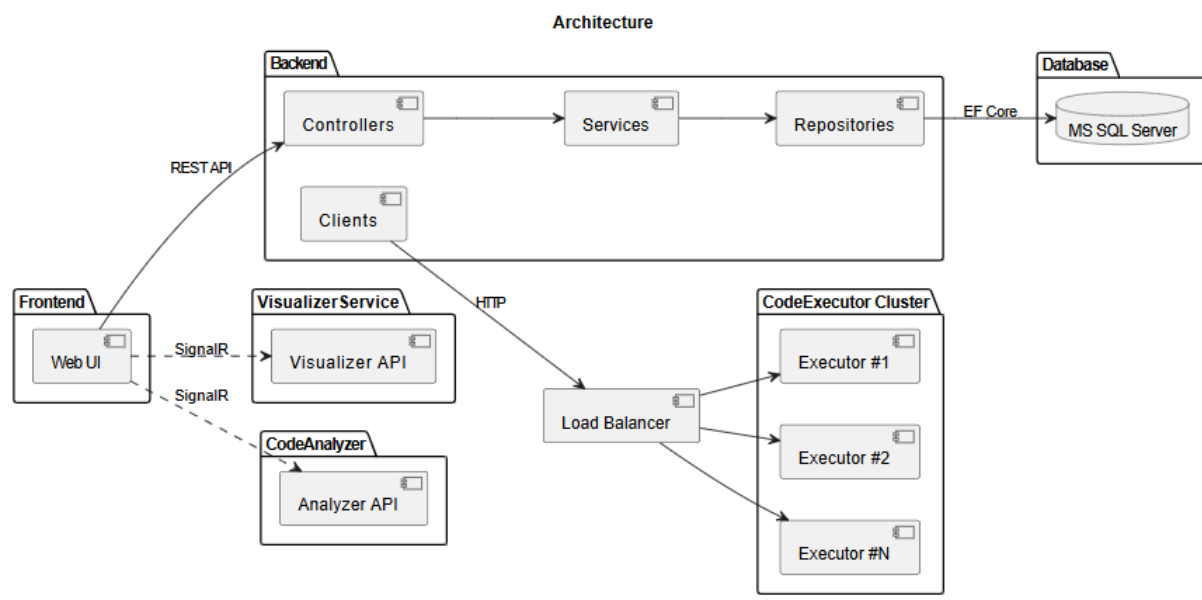
Funkcjonalność	LeetCode	HackerRank	VisuaAlgo	Uniwersytet w San Francisco
Materiały teoretyczne	ograniczone	ograniczone	krótki wstęp	✗
Zadania programistyczne	✓	✓	✗	✗
Wizualizacje algorytmów	ograniczone	✗	✓	✓

Tabela 3.1: Porównanie rozwiązań

4. Architektura systemu

4.1. Struktura komponentów

System został zaprojektowany w oparciu o architekturę wielowarstwową, która wyraźnie rozdziela odpowiedzialności poszczególnych elementów aplikacji. W skład architektury wchodzi warstwa prezentacji odpowiedzialna za interfejs użytkownika, warstwa logiki biznesowej realizowana przez serwer API, warstwa danych reprezentowana przez bazę danych oraz wyspecjalizowane komponenty umożliwiające wykonywanie i analizę kodu oraz wizualizacje algorytmów. Całość została zaprojektowana z myślą o wdrożeniu w środowisku chmurowym Microsoft Azure z wykorzystaniem konteneryzacji Docker. Schemat architektury systemu został zaprezentowany na Rys. 4.1.



Rysunek 4.1: Schemat architektury

Architektura aplikacji była planowana pod kątem skalowalności i bezpieczeństwa, częściowo na podstawie istniejących rozwiązań, opisanych w Haohui Liang et al [10] i Yutaka Watanobe, Md. Mostafizer Rahman, Taku Matsumoto, Uday Kiran Rage i Penugonda Ravikumar [2].

4.1.1. Serwer API

Serwer został zaimplementowany w technologii ASP.NET Core w wersji 9.0, która zapewnia wydajność, bezpieczeństwo oraz pełne wsparcie dla tworzenia interfejsów REST API [3].

Stanowi centralny element całego systemu. To on pobiera dane, które później wyświetlane są użytkownikowi na interfejsie aplikacji. W ten sposób serwer pełni rolę pośrednika między warstwą kliencką, a bazą danych oraz komponentem serwisu wykonującego kod. W celu zapewnienia bezpiecznego dostępu do zasobów systemu, stosowane są w nim mechanizmy uwierzytelniania i autoryzacji użytkowników z wykorzystaniem tokenów JWT. Z endpointów, które udostępnia, dostępne bez dodatkowych zabezpieczeń są jedynie te, które umożliwiają logowanie oraz rejestrację. Jeśli autoryzacja przebiegnie poprawnie, użytkownik uzyskuje dostęp do zapytań umożliwiających pobieranie danych (w tym materiałów teoretycznych, kursów, wykładów oraz osiągnięć), a także rozwiązywania zadań programistycznych. Po stronie serwera obliczane są również postępy w nauce, poprzez funkcje monitorowania progresu w danym kursie, które później zostają wyświetlane użytkownikom.

Serwer API odpowiada także za komunikację z oddzielnym komponentem, znajdującym się w dedykowanym kontenerze, który jest odpowiedzialny za poprawne wykonywanie kodu. Jest to istotne, bo dzięki temu może zarządzać całym procesem sprawdzania kodu oraz rozwiązań zadań i przekazywać warstwie interfejsu użytkownika jedynie kluczowe dla niego informacje. Kolejnym komponentem, z którym komunikuje się serwer, jest baza danych. Dostęp do niej realizowany jest za pomocą Entity Framework Core.

Serwer API komunikuje się również z dwiema zewnętrznymi usługami. Pierwszą z nich jest Azure Blob Storage. Jest to chmurowa usługa Microsoft Azure, która służy do przechowywania danych w postaci plików [5]. W tym przypadku została wykorzystana do przechowywania tam plików multimedialnych, takich jak zdjęcia oraz filmy. Drugą zewnętrzną usługą jest SendGrid, czyli narzędzie do wysyłania wiadomości e-mail z aplikacji. W tym przypadku użyte jest podczas procesu rejestracji, w którym każde stworzone konto musi zostać zweryfikowane. W tym celu użytkownik musi wpisać kod, który zostanie na adres e-mail.

Od strony implementacyjnej, serwer został podzielony na trzy warstwy. Pierwszą z nich są kontrolery, które odpowiadają za autoryzację zapytań i zwracanie właściwych odpowiedzi. Same w sobie nie zawierają jednak logiki biznesowej, dane przesyłane są dalej do warstwy serwisu, tak jak na listingu 1. Widoczny na nim kontroler umożliwia dodawanie nowej zawartości do istniejącego wykładu. Taka funkcjonalność dostępna jest jedynie dla użytkowników z rolą administratora, dlatego odpowiedzialnością kontrolera jest autoryzacja zapytania. Następnie odpytuje odpowiedni serwis i zwraca odpowiedź, a w przypadku niepowodzenia, odpowiednią informację

4.1. STRUKTURA KOMPONENTÓW

```
1  [HttpPost("{lectureId:guid}/contents")]
2  [Authorize(Roles = Roles.Admin)]
3  public async Task<ActionResult<LectureContentDto>> AddContent(
4      Guid lectureId,
5      [FromBody] LectureContentInputDto dto,
6      CancellationToken ct
7  )
8  {
9      try
10     {
11         var content = await _lectureService.AddContentAsync(lectureId, dto, ct);
12         return CreatedAtAction(
13             nameof(GetContentById),
14             new { lectureId, contentId = content.Id },
15             content
16         );
17     }
18     catch (ArgumentException ex)
19     {
20         return BadRequest(ex.Message);
21     }
22     catch (KeyNotFoundException)
23     {
24         return NotFound("Lecture not found");
25     }
26 }
```

Listing 1: Kod kontrolera obsługującego dodanie zawartości do wykładu

z kodem błędu.

Następną warstwą po kontrolerach jest warstwa serwisów. To w nich wykonywana jest właściwa logika. Moduł ten odpowiedzialny jest za trzymanie poprawności zasad zdefiniowanych w programie, tzw. domeny, a także orkiestracją danych pobieranych z pozostałych warstw. Na listingu 2 zaprezentowany jest fragment serwisu, który odpowiada za materiały dydaktyczne (*LectureService*). Widoczna funkcja wywoływana była uprzednio przez znajdujący się na listingu 1 kontroler. Odpowiada za walidację otrzymanych danych. W tym przypadku sprawdza, czy jeśli wybrano jedną z trzech opcji: tekst, zdjęcie lub film to w dalszej części zapytania uzupełniono wymagane dla niej pola. Serwis odpowiada również za komunikację z pozostałymi warstwami – w tym przypadku z repozytorium.

4.1. STRUKTURA KOMPONENTÓW

```
1  public async Task<LectureContentDto> AddContentAsync(  
2      Guid lectureId,  
3      LectureContentInputDto dto,  
4      CancellationToken ct  
5  )  
6  {  
7      LectureContent content;  
8  
9      if (dto.Type == "Text")  
10     {  
11         if (string.IsNullOrEmpty(dto.HtmlContent))  
12             throw new ArgumentException("HtmlContent is required for Text type content");  
13  
14         content = new LectureText  
15         {  
16             HtmlContent = dto.HtmlContent,  
17             Type = ContentType.Text  
18         };  
19     }  
20     else if (dto.Type == "Photo") { ... }  
21     else if (dto.Type == "Video") { ... }  
22     else  
23     {  
24         throw new ArgumentException($"Invalid content type: {dto.Type}");  
25     }  
26  
27     var createdContent = await _repo.AddContentToLectureAsync(lectureId, content, ct);  
28     return MapContentToDto(createdContent);  
29 }
```

Listing 2: Kod serwisu obsługującego dodawanie zawartości do wykładu

Ostatnią warstwą jest repozytorium, będące abstrakcją nad bazą danych. Komunikuje się z nią bezpośrednio i udostępnia odpowiednie metody serwisom. Przykład repozytorium został przedstawiony na listingu 3, gdzie zaobserwować można, jak po otrzymaniu danych przez serwis 2 wykonuje operacje niezbędne do zapisania rekordu w bazie.

```

1  public async Task<LectureContent> AddContentToLectureAsync(
2      Guid lectureId,
3      LectureContent content,
4      CancellationToken ct
5  )
6  {
7      var lecture = await _db.Lectures
8          .Include(l => l.Contents)
9          .FirstOrDefaultAsync(l => l.Id == lectureId, ct);
10
11     if (lecture == null)
12         throw new KeyNotFoundException("Lecture not found");
13
14     content.LectureId = lectureId;
15
16     await _db.LectureContents.AddAsync(content, ct);
17     await _db.SaveChangesAsync(ct);
18
19     return content;
20 }

```

Listing 3: Kod repozytorium obsługującego dodawanie zawartości do wykładu

4.1.2. Aplikacja internetowa

Do stworzenia graficznego interfejsu użytkownika użyto technologii React w wersji 19.1.1 z wykorzystaniem TypeScript w wersji 5.9.3. React to framework do budowy interaktywnych interfejsów użytkownika za pomocą komponentów [4]. Aplikacja powstała w formie *Single Page Application (SPA)*, w której nawigacja pomiędzy poszczególnymi widokami odbywa się bez konieczności przeładowywania całej strony. Całość została podzielona na podstawie odpowiedzialności na warstwę prezentacji, warstwę logiki aplikacyjnej oraz warstwę komunikacji z serwerem API.

4.1. STRUKTURA KOMPONENTÓW

Warstwa prezentacji została zrealizowana za pomocą widoków umieszczonych w katalogu *pages*. Niektóre fragmenty widoków, szczególnie te, które powtarzały się pomiędzy nimi, wydzielono jako osobne komponenty i umieszczono w katalogu *components*. Ze względu na to udało się uprościć kod poszczególnych widoków. Każdy z nich odpowiada jednej trasie aplikacji i reprezentuje logicznie odrębną część interfejsu użytkownika. Fundament aplikacji stanowią pary stron odpowiadające kolejno prezentacji materiałów dydaktycznych, zadań programistycznych oraz kursów – każdy z tych trzech modułów składa się z jednej strony (np. *Lectures*), która prezentuje użytkownikowi listę dostępnych materiałów oraz drugiej (np. *LectureDetails*), która odpowiada za wyświetlenie konkretnego materiału oraz interakcję z nim. Poza tym stworzone widoki obejmują profil użytkownika (*Profile* oraz *EditProfile*), widoki potrzebne w procesie rejestracji (*Register*, *Login*, *VerifyEmail*, *ResetPassword*), główną stronę aplikacji (*Dashboard*) oraz stronę odpowiadającą za wizualizacje algorytmów (*GraphVisualizer*). Drugą częścią warstwy prezentacji jest interfejs przeznaczony dla użytkowników z uprawnieniami administratora. Tacy użytkownicy mają dostępną jedną stronę (*Admin*), która obejmuje jednak mechanizm przełączania się między zakładkami odpowiadającymi za różne funkcjonalności.

Komunikacja z serwerem Api została wydzielona do osobnej warstwy umieszczonej w katalogu *api*, który podzielono na podkatalogi odpowiedzialne za poszczególne grupy endpointów (np. uwierzytelnianie, zadania, wykłady, przypadki testowe). W każdym takim podkatalogu znalazł się plik (np. *lectureApi.ts*) odpowiadający za wykonywanie zapytań do serwera oraz drugi, *types.ts*, w którym znalazły się modele obiektów wysyłanych i odbieranych z serwera.

Warstwa logiki aplikacji umożliwia współdzielenie logiki między widokami i wyodrębnienie jej z kodu poszczególnych stron. W tym celu wykorzystano mechanizm *hooków*. Odpowiadają one za obsługę odświeżania tokenów uwierzytelniających, integrację z edytorem kodu oraz wizualizację grafów. Dodatkowo do zarządzania stanem aplikacji wykorzystano bibliotekę *Redux Toolkit*. Umożliwia ona przechowywanie informacji istotnych z punktu widzenia całej aplikacji. W tym przypadku została wykorzystana do przechowywania danych zalogowanego użytkownika w formie *userSlice*. Znajdują się w nim zarówno dane profilu użytkownika (e-mail, imię, nazwisko oraz token), jak i informacja o tym, czy jest zalogowany. Gdyby jednak używać jedynie tego mechanizmu, informacje zniknęłyby zawsze po odświeżeniu strony lub restarcie przeglądarki. Ze względu na to zapisywane są również do pamięci trwałej przeglądarki *localStorage*. Takie podejście sprawia, że platforma nie wymaga od użytkownika ponownego logowania przy każdym nowym jej otwarciu, ale pobiera jego poprzednie dane i, w razie potrzeby, jedynie odświeża token. Na listingu 4 znajduje się fragment kodu projektu, który odpowiada za zarządzanie stanem użytkownika oraz zapisywanie go w pamięci trwałej przeglądarki.

```

1  interface UserState {
2      user: User | null;
3      isAuthenticated: boolean;
4  }
5
6  const storedUser = localStorage.getItem('user');
7
8  const initialState: UserState = {
9      user: storedUser ? JSON.parse(storedUser) : null,
10     isAuthenticated: !!storedUser,
11 };
12
13 const userSlice = createSlice({
14     name: 'user',
15     initialState,
16     reducers: {
17         login(state, action: PayloadAction<User>) {
18             state.user = action.payload;
19             state.isAuthenticated = true;
20             localStorage.setItem('user', JSON.stringify(action.payload));
21             localStorage.setItem('token', action.payload.token || '');
22         },
23         logout(state) {
24             state.user = null;
25             state.isAuthenticated = false;
26             localStorage.removeItem('user');
27             localStorage.removeItem('token');
28         },
29     },
30 });

```

Listing 4: Fragment kodu *userSlice*, odpowiedzialnego za zarządzanie stanem użytkownika oraz przechowywanie informacji w pamięci trwałej przeglądarki (*localStorage*).

4.1. STRUKTURA KOMPONENTÓW

Aplikacja wykorzystuje mechanizm uwierzytelniania oparty na tokenach JWT. Jeśli użytkownik przejdzie przez proces logowania, otrzymuje on token dostępu, który wykorzystywany jest do autoryzacji kolejnych zapytań do serwera API. Jak już wspomniano, w przypadku, gdy ważność tokenu wygasa, aplikacja komunikuje się z serwerem, aby go odświeżyć. Proces ten odbywa się w tle, bez konieczności interakcji użytkownika, co poprawia komfort korzystania z platformy.

Zrealizowano również mechanizm ochrony dostępu do poszczególnych widoków aplikacji. W tym celu wykorzystano trasy chronione, które sprawiają, że niektóre ścieżki dostępne są wyłącznie dla zalogowanych użytkowników, a jeszcze inne – jedynie dla użytkowników z uprawnieniami administratora. W przypadku próby wejścia na niedozwolony widok, użytkownik zostaje automatycznie przekierowany na ekran logowania. Przykład takich komponentów widoczny jest na listingu 5. Taki mechanizm stanowi jedynie dodatkowe zabezpieczenie po stronie aplikacji klienckiej, bo ostateczna weryfikacja uprawnień użytkownika odbywa się po stronie serwera.

```
1 function UserRoute({ children }: { children: React.ReactNode }) {
2   const isAuthenticated = localStorage.getItem('isAuthenticated') === 'true';
3   const isAdmin = localStorage.getItem('isAdmin') === 'true';
4
5   return isAuthenticated && !isAdmin ? <>{children}</> : <Navigate to="/login" replace />;
6 }
7
8 function AdminRoute({ children }: { children: React.ReactNode }) {
9   const isAuthenticated = localStorage.getItem('isAuthenticated') === 'true';
10  const isAdmin = localStorage.getItem('isAdmin') === 'true';
11
12  return !isAuthenticated
13    ? <Navigate to="/login" replace />
14    : !isAdmin
15      ? <Navigate to="/" replace />
16      : <>{children}</>;
17 }
```

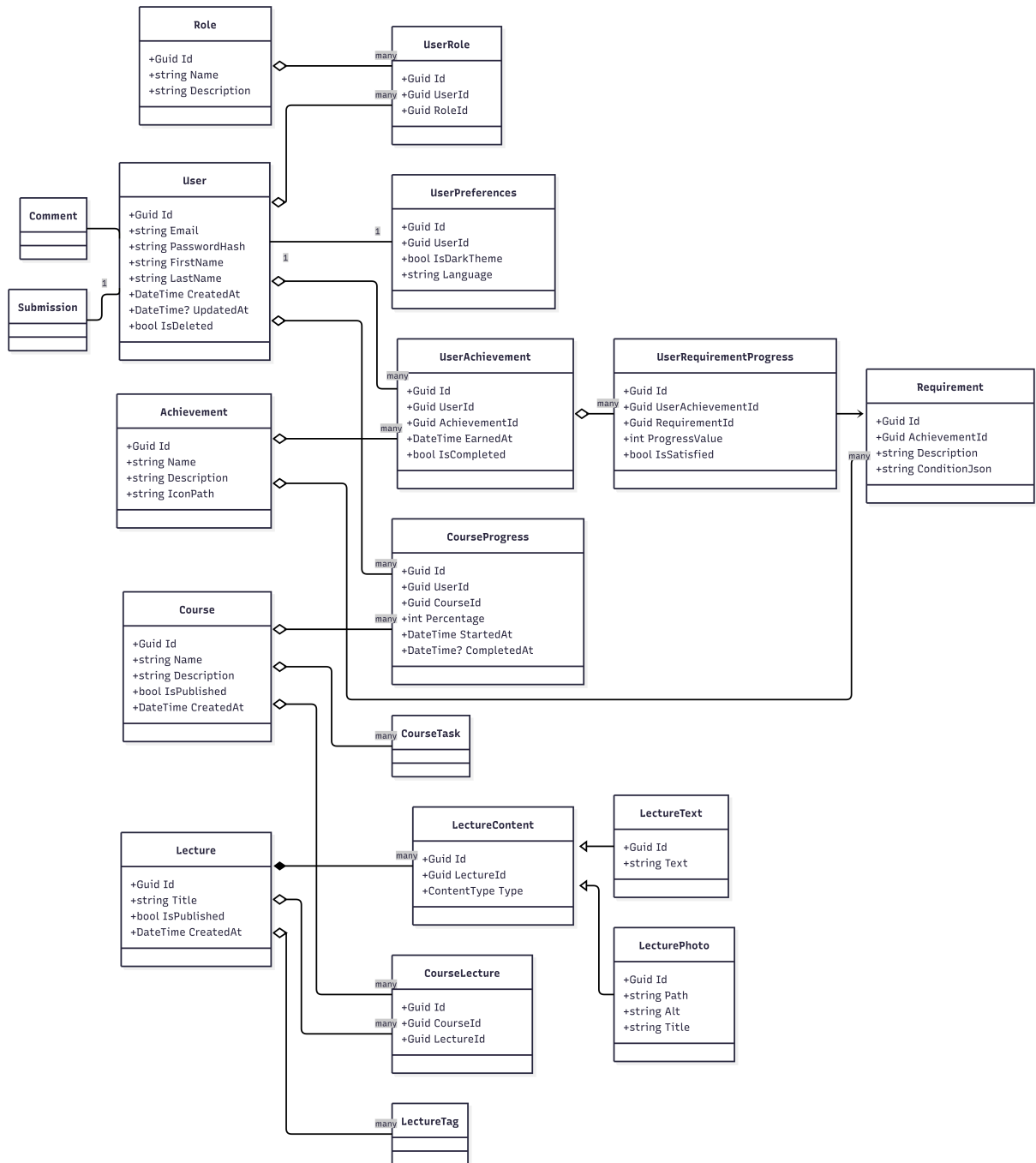
Listing 5: Komponenty *UserRoute* oraz *AdminRoute* odpowiedzialny za ograniczenie dostępu do widoków.

4.1.3. Baza danych

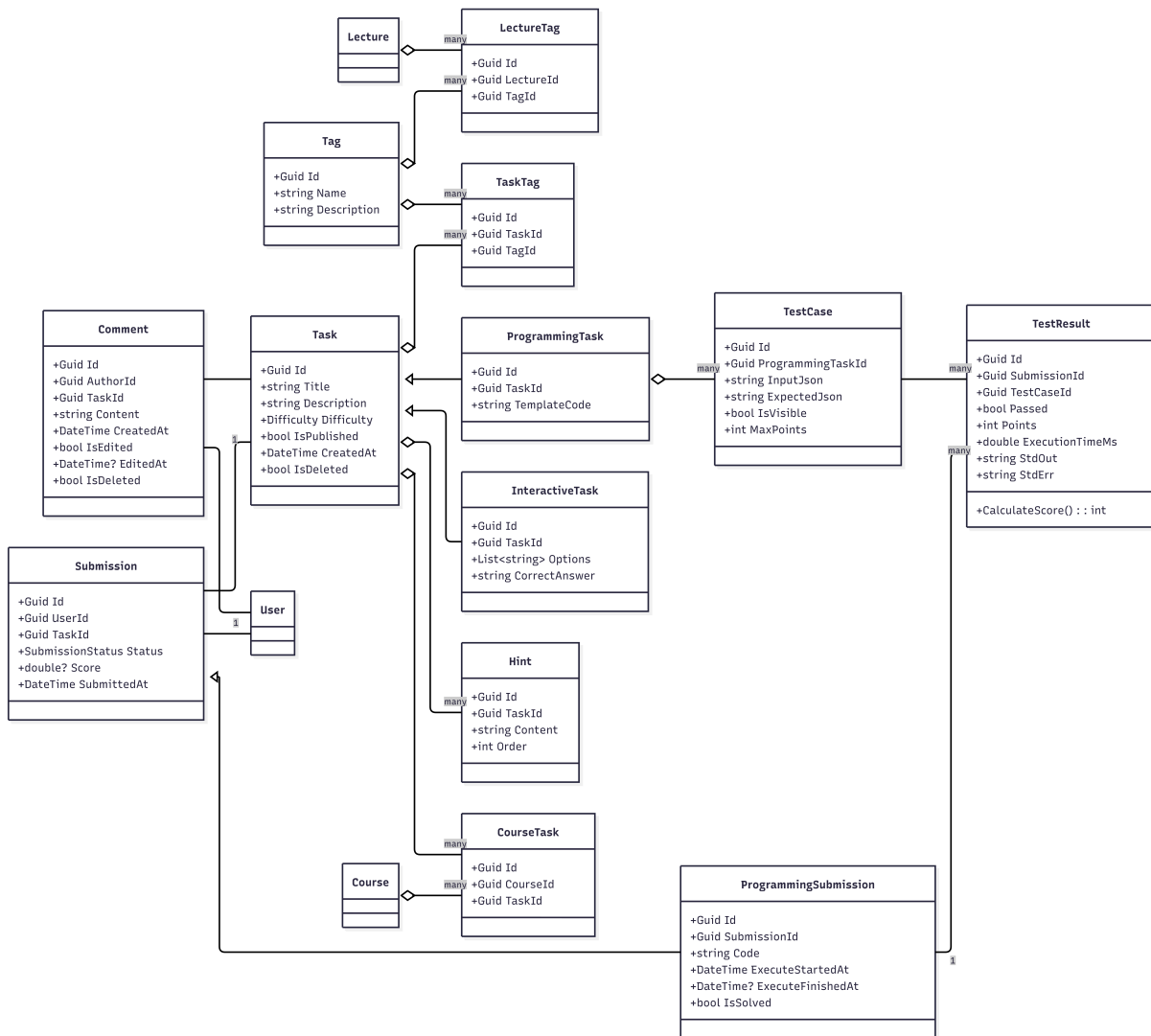
Warstwa przechowywania danych została zrealizowana przy użyciu Microsoft SQL Server 2022, czyli systemu zarządzania relacyjnymi bazami danych [6]. Główne klasy systemu obejmują użytkowników z przypisanymi rolami i przechowywanymi statystykami, kursy składające się z lektur i zadań oraz przesłane rozwiązania. Większość klas jest w relacji wiele do wielu, zdefiniowany model jest elastyczny. Na przykład kursy mogą mieć wiele lektur lub zadań, aby nie ograniczać materiałów do pojedynczego obiektu. Jest to przydatne w przypadkach, gdzie użytkownik chce zamieścić wykład z podstawami na początku kilku kursów. Kluczowym elementem zadań programistycznych są przypadki testowe zawierające początkowe i oczekiwane dane, aby następnie przydzielić odpowiedni procent z maksymalnej liczby punktów za zadanie. Zarazem zadania jak i lektury przechowują informacje, czy są ukończone lub czy są opublikowane, co poprawia jakość użytkowania zarazem administratorom, mogącym dopracować materiał przed opublikowaniem i użytkownikom mogącym na bieżąco przeglądać progres nauki. Model danych zarządzany jest poprzez mechanizm migracji pozwalający na kontrolowaną ewolucję struktury.

Przygotowano diagram klas przedstawiający modele w bazie danych. Został dla czytelności podzielony na dwie części – pierwszą, zawierającą klasy związane z użytkownikiem, kursami i lekturami (Rys. 4.2), oraz drugą zawierającą klasy związane z zadaniami (Rys. 4.3). Klasy bez żadnych wartości zostały dodane podwójnie, aby pokazać połączenia pomiędzy dwoma częściami diagramu.

4.1. STRUKTURA KOMPONENTÓW



Rysunek 4.2: Diagram klas – część 1



Rysunek 4.3: Diagram klas – część 2

4.1.4. Serwis wykonujący kod (Code executor)

Code Executor to mikroserwis odpowiedzialny za wykonanie kodu nadesłanego przez użytkownika. Analogicznie do serwera, został zaimplementowany w ASP.NET Core 9, w przeciwieństwie jednak do niego, obejmuje dużo bardziej ograniczony zakres funkcjonalności. Ze względu na to, że wykonywanie kodu jest funkcjonalnością zupełnie odrębną od pozostałych, postanowiono wydzielić ją z serwera API. Stanowiło to także dodatkową warstwę bezpieczeństwa. Code Executor funkcjonuje niezależnie od wszystkich pozostałych komponentów i nie komunikuje z nimi, nie ma też połączenia z bazą danych. Aby skorzystać z jego funkcjonalności, należy komunikować się z nim poprzez interfejs REST API.

W momencie wdrażania systemu tworzone jest kilka kopii tego serwisu, a wysyłane zapytania trafiają najpierw do *load-balancera*, który następnie przekierowuje je naprzemiennie pomiędzy różne kopie serwisu. Dokładniej architektura i mechanizm działania tego komponentu zostały opisane w rozdziale piątym o zdalnym wykonywaniu kodu.

4.1.5. Serwis analizujący kod (Code analyzer)

Code Analyzer to kolejny mikroserwis, który również ściśle powiązany jest z częścią aplikacji odpowiedzialną za rozwiązywanie zadań programistycznych. Jego główną rolą jest wsparcie edytora kodu, który wyświetlany jest użytkownikowi jako element strony. Na podstawie kodu wpisanego przez użytkownika, Code Analyzer dokonuje analizy poprawności, umożliwia podpowiadanie wpisywanych słów, a także wyświetlanie podstawowych informacji po najechaniu myszą. Również został napisany w ASP.NET Core 9.0.

Komunikacja z interfejsem użytkownika przebiega za pomocą otwartoźródłowej biblioteki SignalR, służącej do obsługi powiadomień pomiędzy serwerem (w tym przypadku Code Analyze-rem) a klientem (czyli przeglądarką z interfejsem). Do komunikacji wykorzystuje ona, o ile jest to możliwe, protokół WebSocket oparty o TCP [7]. Dzięki temu, mimo że użytkownik pisze kod w przeglądarce, dostaje on szybko informację zwrotną. Sam serwis nie wykonuje kodu, nie ma też połączenia z innymi komponentami poza interfejsem, ponieważ nie są mu zwyczajnie potrzebne. Do analizy kodu wykorzystany został kompilator C# (Roslyn), omówiony dokładniej w rozdziale piątym. Pomimo wydajności serwisu, zdecydowano się go oddzielić od reszty, ponieważ potencjalnie jest on narażony na największy ruch, a co za tym idzie, możliwość niespodziewanego zakończenia działania.

Sama w sobie struktura komponentu jest bardzo prosta, udostępnia on na zewnątrz trzy publiczne metody, co widać na listingu 6

```
1 public class RoslynHub : Hub
2 {
3     // Podkreślanie błędów
4     public async Task<DiagnosticDto[]> AnalyzeCode(string code)
5     {
6         return await _diagnosticService.AnalyzeAsync(...);
7     }
8
9     // Podpowiadanie kolejnych słów
10    public async Task<CompletionItemDto[]> GetCompletions(string code, int line, int column)
11    {
12        return await _completionService.GetCompletionsAsync(...);
13    }
14
15    // Dodatkowe informacje po najechnaniu mysza
16    public async Task<QuickInfoDto?> GetQuickInfo(string code, int line, int column)
17    {
18        return await _quickInfoService.GetQuickInfoAsync(...);
19    }
20 }
```

Listing 6: Kod serwisu analizującego kod pisany przez użytkownika

4.1.6. Serwis obsługujący wizualizacje (Visualizer Service)

Visualizer Service operuje na podobnych mechanizmach co opisane wcześniej komponenty Code Analyzer i Code Executor, pełni jednak odmienną funkcję. Został zaimplementowany w ASP.NET Core 9.0 i w analogiczny sposób komunikuje się z interfejsem użytkownika poprzez wykorzystanie biblioteki SignalR. Stanowi podstawę części aplikacji odpowiedzialnej za wizualizacje. W bezpieczny sposób uruchamia kod algorytmu, który użytkownik chce zwizualizować, a następnie wysyła powiadomienia, informujące interfejs o tym, jak ma przebiegać wizualizacja. W odpowiedzi na zdarzenia interfejs aktualizuje animacje. Dzięki wydajnej metodzie komunikacji, udało się uzyskać płynne rysowanie kolejnych zdarzeń. Wykonywanie kodu zostało w nim zaimplementowane w analogiczny sposób jak w serwisie Code Executor. Żeby zapobiec nadmiernemu wykorzystaniu zasobów, limit animacji został ustawiony domyślnie na 15 sekund. Struktura komponentu jest analogiczna do poprzedniego, udostępnia on metodę do rozpoczęcia wizualizacji, a interfejs przesyła mu odpowiednie dane, w tym kod użytkownika i strukturę grafu.

4.2. Infrastruktura i wdrożenie

4.2.1. Ogólna koncepcja wdrożenia

Aplikacja została zaprojektowana z myślą o łatwym wdrożeniu. Część funkcjonalności planowana była pod kątem środowiska Microsoft Azure zapewniającym skalowalność i wysoką dostępność, jednakże rozwiązanie nie jest od niego całkowicie uzależnione. W trakcie rozwoju aplikacji, stworzone zostało środowisko ciągłej integracji i dostarczania CI/CD, dzięki któremu kontenery powstałe na podstawie kodu były automatycznie publikowane w rejestrze kontenerów GitHub. Kontenery uruchamiane są przez wywołanie dedykowanych plików docker-compose. Uruchomienie jest przygotowane w dwóch środowiskach:

- Środowisko produkcyjne – pobiera informacje z rejestru kontenerów
- Środowisko deweloperskie – pobiera informacje z lokalnych wersji na komputerze.

4.2.2. Konteneryzacja

Do poprawnego działania aplikacji lokalnie uruchamiane są następujące kontenery:

- *database* – kontener z bazą danych Microsoft SQL Server 2022 przechowujący dane użytkowników, kursów, zadań i rozwiązań

- *backend* – kontener z serwerem API ASP.NET Core obsługującym logikę biznesową i komunikację z pozostałymi komponentami
- *frontend* – kontener z aplikacją React, udostępniający interfejs użytkownika
- *code-executor* – kontener w konfigurowalnej liczbie kopii z serwisem ASP.NET Core, odpowiedzialny za bezpieczne wykonywanie i testowanie kodu użytkowników w izolowanych środowiskach
- *azurite* – emulator Azure Blob Storage do lokalnego przechowywania plików multimedialnych w środowisku deweloperskim
- *code-analyzer* – kontener z serwisem ASP.NET Core analizujący poprawność kodu, zapewniający podpowiedzi i podkreślanie błędów w trakcie pisania przez użytkownika
- *visualizer* – kontener z serwisem ASP.NET wykonujący kod przesłany do wizualizacji i wysyłający na bieżąco powiadomienia do aplikacji internetowej
- *load balancer* – serwer NGINX zarządzający przekierowywaniem ruchu z serwera API do odpowiednich kontenerów code-executor

Plik docker compose narzuca kilka dodatkowych ograniczeń kontenerom. Do najważniejszych należą ograniczenia Code Executora, w tym ograniczenia sieci, liczby procesów i dostępu do pamięci, przede wszystkim ze względów bezpieczeństwa. Dzięki wykonywaniu kodu w oddzielnym kontenerze, który nie ma dostępu do bazy danych, zapewniona została dodatkowa warstwa bezpieczeństwa. Analiza kodu została wydzielona od Code Executora, pomimo tego, że część ich zadań może wydawać się zbliżona. Jest to jednak celowy zabieg. Dzięki temu oddzieleniu, obciążenie Code Executora się zmniejsza. Code Analyzer odbiera zapytania od interfejsu z każdym przyciskiem na klawiaturze podczas pisania kodu, więc szansa na jego awarię jest znacznie większa. Poza tym funkcjonalność ta ma mniejszy priorytet od samego wykonywania kodu i gdyby taka awaria nastąpiła, aplikacja może działać dalej aż do czasu naprawy. Visualizer – serwis odpowiedzialny za wizualizację – został rozdzielony od Code Executora z myślą o możliwości dalszego rozwoju tego systemu o kolejne animacje. W takim przypadku jego kod mógłby stać się bardziej wymagający w utrzymaniu, a wydajność – pogorszyć się. Priorytetem przy projektowaniu infrastruktury wdrożenia była jak największa odporność i wysoka dostępność systemu, co uzyskano przez rozdzielenie komponentów o największej skłonności do awarii od pozostałych, które są krytyczne dla funkcjonowania.

Jak już wspomniano wcześniej, rozwiązanie było projektowane przede wszystkim pod kątem środowiska Azure, jednakże istnieje możliwość stosunkowo łatwej zmiany konfiguracji aplikacji.

4.3. INSTRUKCJA WDROŻENIA

Narzucone zostało użycie Azure Blob Storage i częściowo Microsoft SQL Server. Istnieje możliwość zmiany tych ustawień, ale konieczne jest wtedy dostosowanie kodu oraz pliku docker compose. W takim przypadku należałoby odpowiednio dostosować kontener bazy danych, warstwę repozytoriów komunikujących się z bazą, a także serwis do obsługi plików multimedialnych.

Użycie *load-balancera* także jest łatwo konfigurowalne, jak i liczba kopii kontenera *code-executor*. W razie potrzeby możliwa jest również całkowita rezygnacja z serwera NGINX. Wymagałoby to zmian wyłącznie w plikach dockerowych i środowiskowych.

4.3. Instrukcja wdrożenia

Do wdrożenia obecnie istniejącego systemu bez żadnych zmian, potrzebne jest wykonanie kilku kroków. Aplikacja projektowana była pod kątem szybkiego i bezproblemowego uruchomienia.

4.3.1. Wymagania wstępne

1. Założono, że na komputerze zainstalowana jest najnowsza wersja **Dockera**. Przydatny jest także jego interfejs graficzny – Docker Desktop.
2. Do pobrania repozytorium potrzebna jest zainstalowana najnowsza wersja **gita**.
3. Do zarządzania bazą danych zalecane (ale niewymagane) jest pobranie najnowszej wersji **SQL Server Management Studio**
4. Przydatne, ale też niewymagane, jest pobranie najnowszej wersji **Visual Studio** w wersji 2026 lub 2022.
5. Do włączenia interfejsu graficznego zalecana jest najnowsza wersja **przeglądarki internetowej** np. Google Chrome. Aplikacja nie jest wspierana na urządzenia mobilne.

4.3.2. Środowisko deweloperskie

Środowisko deweloperskie zostało zoptymalizowane pod kątem rozwoju i testowania oprogramowania. Podczas jego uruchomienia, kontenery tworzą się na podstawie kodu użytkownika. Wykorzystane do ich stworzenia obrazy zajmują więcej pamięci, nie są przystosowane więc do wdrożenia, są jednak dobre do lokalnego uruchomienia i znalezienia potencjalnych błędów.

1. Należy stworzyć folder, np. o nazwie *AlgoRhythmProject*
2. Do niego należy pobrać repozytoria:


```
git clone https://github.com/AlgoRhythmProject/Backend.git oraz
git clone https://github.com/AlgoRhythmProject/Frontend.git
```
3. Należy utworzyć w folderze *AlgoRhythmProject/Backend/AlgoRhythm* pierwszy plik *.env* ze zmiennymi środowiskowymi.
4. W pliku tym należy ustalić wartość następujących zmiennych:
 - (a) SA_PASSWORD – hasło do bazy danych
 - (b) JWT_KEY – klucz potrzebny do generowania tokenu uwierzytelniającego JWT
 - (c) CONNECTION_STRING – wykorzystywany do połączenia z bazą danych przez serwer
 - (d) SENDGRID_API_KEY – klucz wykorzystywany przez serwis wysyłający maile
 - (e) FRONTEND_URL – adres, na którym włączana jest aplikacja webowa, `http://localhost:5173`
 - (f) CODE_EXECUTOR_URL – w przypadku korzystania z *load-balancera* – jego adres (`http://executor-lb:80`), w przeciwnym przypadku adres serwisu (`http://executor:8080`) *code-executor*
 - (g) GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET – klucze do logowania przez serwis Google
5. W folderze *AlgoRhythmProject/Frontend/AlgoRhythm* należy utworzyć drugi plik ze zmiennymi środowiskowymi *.env*
6. W pliku należy umieścić:
 - (a) VITE_API_BASE_URL – adres serwera, `http://localhost:7062/api`
 - (b) VITE_ANALYZER_URL – adres *code-analizera*, `http://localhost:5095/roslynhub`

4.3. INSTRUKCJA WDROŻENIA

(c) `VITE_VISUALIZER_URL` – adres serwisu do wizualizacji, `http://localhost:5148/visualizerhub`

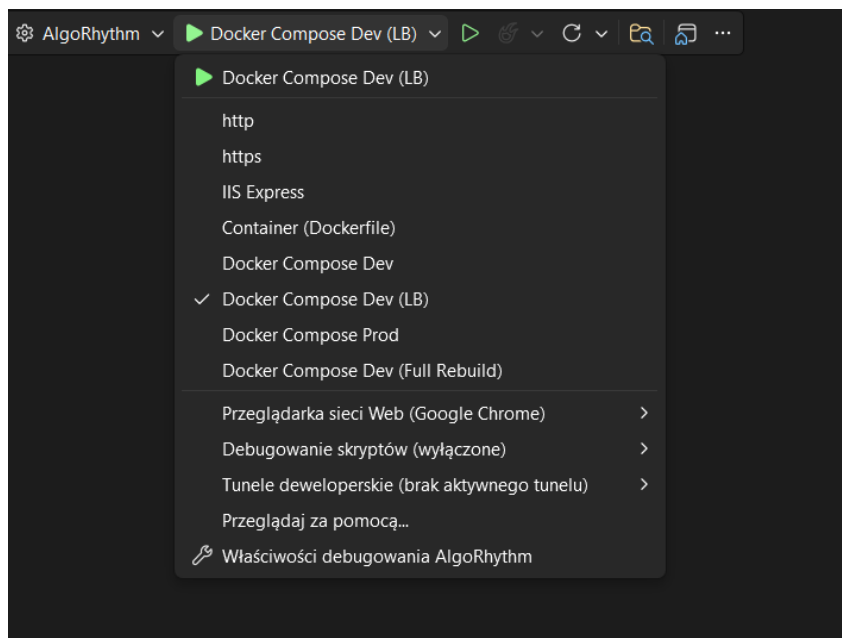
(d) `VITE_GOOGLE_CLIENT_ID` – potrzebny do logowania przez Google

7. Struktura katalogów została przedstawiona na Rys. 4.4.

8. Następnie należy wpisać w konsoli, w folderze *AlgoRhythmProject/Backend/AlgoRhythm* `docker-compose -f docker-compose.dev.yml up [-scale executor = liczba_kopii]`, gdzie *liczba_kopii* należy zastąpić oczekiwaną liczbą kopii *code-executora*, np. 2-3. W przypadku korzystania z Visual Studio i Docker Desktop, możliwe jest też wybranie konfiguracji *Docker Compose Dev (LB)* z menu na górze (Rys. 4.5).

```
AlgoRhythmProject/  
├── Frontend/  
│   └── AlgoRhythm/  
│       └── .env (zmienne środowiskowe z punktu 6)  
└── Backend/  
    ├── AlgoRhythm/  
    │   ├── .env (zmienne środowiskowe z punktu 4)  
    │   └── docker-compose.dev.yml
```

Rysunek 4.4: Struktura katalogów projektu



Rysunek 4.5: Wybór konfiguracji startowej w Visual Studio

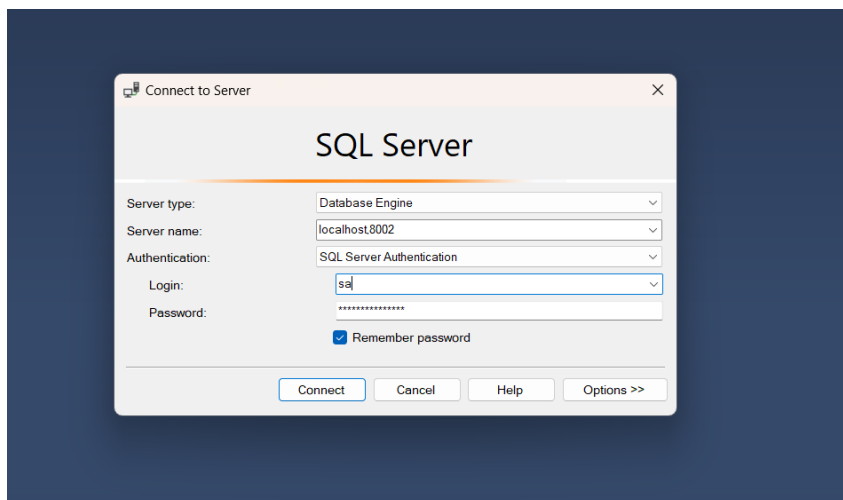
W konsoli powinna pojawić się informacja o utworzeniu kontenerów. Za pierwszym razem może zająć to kilka minut. Włączony serwer powinien sam odpowiednio zaktualizować bazę

i uzupełnić ją początkowymi danymi. Po poprawnym włączeniu w konsoli powinna pojawić się odpowiednia informacja, analogiczna do tej widocznej na Rys. 4.6. Aplikacja powinna być dostępna przez przeglądarkę pod adresem interfejsu użytkownika.

```
[+] Running 9/9
✓Container visualizer           Started           35.8s
✓Container code_analyzer        Started           35.8s
✓Container backend              Healthy          73.4s
✓Container executor_load_balancer Started           33.1s
✓Container algorhythm-executor-9 Removed          3.4s
✓Container algorhythm-executor-10 Started           32.6s
✓Container azurite              Started           30.4s
✓Container database             Healthy          43.9s
✓Container frontend             Started           58.0s
```

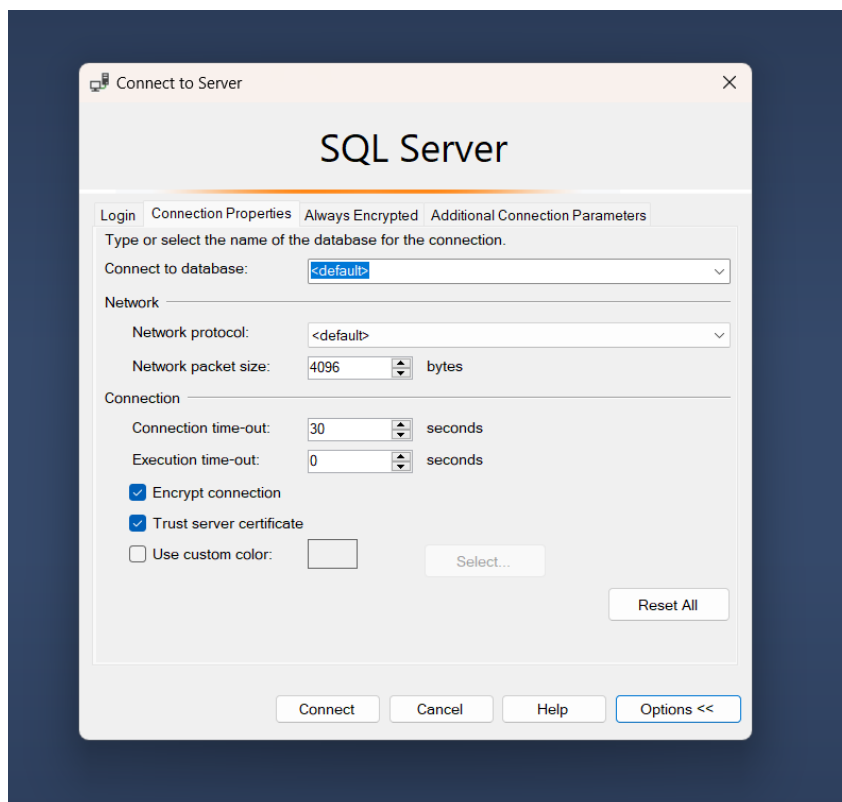
Rysunek 4.6: Przykładowa informacja z konsoli o poprawnym uruchomieniu kontenerów

Dokumentację API *Swagger* można zobaczyć pod adresem <http://localhost:7062/swagger/index.html> dla serwera, a dla serwisu wykonującego kod – <http://localhost:7070/swagger/index.html>. Możliwe jest też zalogowanie do bazy danych. W tym celu należy uruchomić Microsoft SQL Server Management Studio i wpisać dane logowania, co pokazano na Rys. 4.7 i 4.8.



Rysunek 4.7: Logowanie do serwera bazy danych – cz.1

4.3. INSTRUKCJA WDROŻENIA



Rysunek 4.8: Logowanie do serwera bazy danych – cz.2

4.3.3. Środowisko produkcyjne

Środowisko produkcyjne jest lepiej przystosowane do opublikowania. Jednym z powodów jest to, że obrazy użyte w trakcie jego uruchamiania są lżejsze. Nie wykorzystuje ono lokalnych plików, nie buduje kontenerów, tylko pobiera je z rejestru. Podczas aktualizacji kodu źródłowego na głównych gałęziach repozytoriów ich zawartość jest automatycznie dostosowywana.

Aby uruchomić aplikację w środowisku produkcyjnych, należy wykonać kolejne kroki:

1. Należy pobrać plik *docker-compose.prod.yml*
2. Należy utworzyć w tym samym folderze plik *.env* ze zmiennymi środowiskowymi
3. W pliku należy umieścić następujące zmienne:
 - (a) Wszystkie z poprzedniej sekcji poza zaczynającymi się od przedrostka VITE (SA_PASSWORD, JWT_KEY, CONNECTION_STRING, SENDGRID_API_KEY, FRONTEND_URL, CODE_EXECUTOR_URL, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET)
 - (b) AZURE_STORAGE_CONNECTION_STRING, wykorzystywany do połączenia z Azure Blob Storage. Domyślnie do przechowywania plików używany jest kontener z

azurite, służący jedynie do lokalnego testowania. W przypadku korzystania z niego, wartość można skopiować z *docker-compose.dev.yml*

(c) ANALYZER_URL – adres serwisu analizującego kod, lokalnie `http://localhost:5095/roslynhub`

(d) VISUALISER_URL – adres wizualizatora, lokalnie `http://localhost:5148/visualizerhub`

(e) BACKEND_URL – adres serwera, lokalnie `http://localhost:7062/api`

4. Następnie należy wpisać w konsoli

```
docker-compose -f docker-compose.prod.yml up [-scale executor = liczba_kopii],
```

gdzie *liczba_kopii* należy zastąpić oczekiwaną liczbą kopii *code-executora* (sugerowana liczba to 2-3). Analogicznie, w przypadku korzystania z Visual Studio i Docker Desktop możliwe jest wybranie konfiguracji *Docker Compose Prod*.

W konsoli powinna pojawić się informacja o pobraniu i utworzeniu kontenerów. Uruchomiony serwer powinien sam odpowiednio zaktualizować bazę. Jeśli proces wdrażania przebiegnie pomyślnie, w konsoli powinna pojawić się odpowiednia informacja, analogiczna do tej widocznej na Rys. 4.9.

Różnicą między środowiskiem produkcyjnym a deweloperskim jest to, że w środowisku produkcyjnym wszystkie zmienne środowiskowe powinny znaleźć się w jednym pliku. W przypadku pracy zarówno na środowisku produkcyjnym jak i deweloperskim i częstego przełączania się pomiędzy nimi, sugerowane jest, by stworzyć plik *AlgoRhythmProject/Backend/AlgoRhythm/.env* i umieścić w nim wszystkie zmienne konieczne dla środowiska produkcyjnego oraz plik *AlgoRhythmProject/Frontend/AlgoRhythm/.env* ze zmiennymi z sekcji o środowisku deweloperskim. Takie rozwiązanie obejmować będzie zmienne potrzebne dla obu przypadków.

```
[+] Running 8/8
✓Container visualizer      Created      0.0s
✓Container azurite         Created      0.0s
✓Container code_analyzer   Created      0.0s
✓Container database        Running      0.0s
✓Container algohythm-executor-9 Created      0.0s
✓Container backend         Running      0.0s
✓Container executor_load_balancer Created      0.0s
✓Container frontend        Created      0.0s
Attaching to executor-9, azurite, backend, code_analyzer, database, executor_load_balancer, frontend, visualizer
```

Rysunek 4.9: Przykładowa informacja z konsoli o poprawnym uruchomieniu kontenerów

4.3.4. Zewnętrzne zależności i utrzymanie

Aplikacja rozwijana była pod kątem możliwości stosunkowo łatwej konfiguracji zewnętrznych zależności przy minimalnych zmianach w kodzie i jak największej utrzymywalności. Obecnie platforma jest zależna od następujących serwisów:

1. Roslyn — kompilator języka C#, aktywnie rozwijany przez Microsoft, natywnie zintegrowany z Visual Studio, zapewniający wsteczną kompatybilność z wcześniejszymi wersjami języka.
2. Microsoft SQL Server – zdecydowano się na użycie go w związku z dobrą integracją z Entity Framework. Zgodnie z jego dokumentacją, do innych wspieranych przez to narzędzie baz danych należą m.in. [8]:
 - (a) MySQL
 - (b) SQLite
 - (c) PostgreSQL

Zatem zmiana w tym przypadku nie powinna być problematyczna.

3. SendGrid – serwis, który służy do wysyłania wiadomości e-mail. W przypadku chęci modyfikacji tej zależności, konieczne byłoby napisanie nowego serwisu implementującego interfejs *ISender* znajdujący się na listingu 7.
4. Azure Blob Storage – usługa umożliwiająca bezpieczne przechowywanie danych, w tym przypadku zdjęć oraz filmów. Podobnie jak w przypadku serwisu *SendGrid*, tutaj również możliwa jest zmiana za pomocą nowej implementacji interfejsu *IFileStorageService* pokazanego na listingu 8.

```
1 public interface ISender
2 {
3     Task SendEmailAsync(string toEmail, string subject,
4         string plainTextContent, string htmlContent);
5 }
```

Listing 7: Interfejs wykorzystywany do wysyłania maili podczas rejestracji

```

1  public interface IFileStorageService
2  {
3      Task<LectureVideo?> GetVideoInfoAsync(string identifier);
4      Task<string> UploadFileAsync(string identifier, Stream content, string contentType);
5      Task<(Stream stream, string contentType)> GetFileAsync(string identifier);
6      Task<bool> DeleteFileAsync(string identifier);
7      Task<(List<BlobFileInfo> files, string? continuationToken)> ListFilesAsync(
8          int pageSize = 50,
9          string? continuationToken = null
10     );
11 }

```

Listing 8: Interfejs wykorzystywany do obsługi plików

4.4. Komunikacja między komponentami

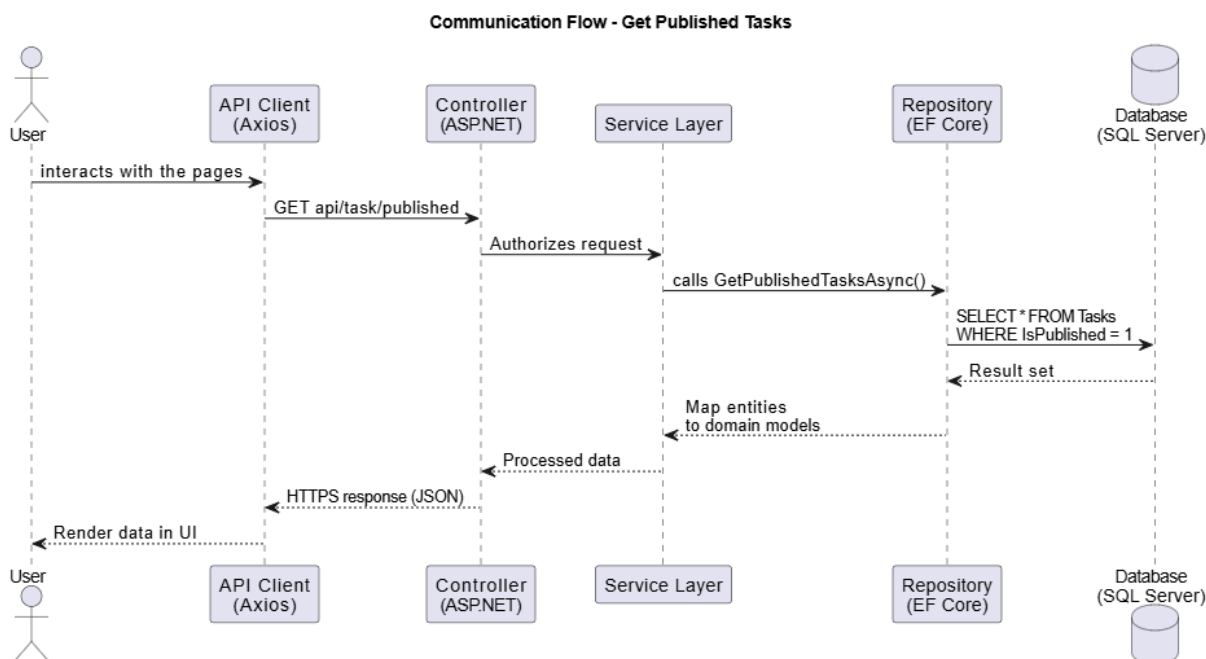
Wymiana informacji między poszczególnymi elementami systemu została zorganizowana w oparciu o standardowe protokoły i wzorce architektoniczne, zapewniając spójność, bezpieczeństwo i efektywność działania całej platformy. Komunikacja odbywa się głównie poprzez protokół HTTP/HTTPS z wykorzystaniem architektury REST API, natomiast dane przesyłane są w formacie JSON.

4.4.1. Proces komunikacji

Przykład procesu komunikacji został przedstawiony na rysunku 4.10. Dotyczy on specyficznie sytuacji, w której użytkownik wejdzie na ekran z listą opublikowanych zadań programistycznych i obrazuje to, co dzieje się aż do momentu ich wyświetlenia.

Kiedy użytkownik wejdzie na stronę, aplikacja webowa wysyła zapytanie do serwera API, by pobrać odpowiednie dane (w tym przypadku jest to zapytanie *GET api/task/published*). Robi to za pomocą biblioteki *Axios*, która umożliwia wysłanie zapytania HTTP i komunikację z API. Tak jak było wcześniej wspomniane, zapytanie po stronie serwera odebrane jest przez kontroler, a następnie przekazane do serwisu, który w przypadku bardziej złożonych zapytań, wykonuje dodatkową logikę. Następnie serwis wywołuje funkcję z repozytorium, które bezpośrednio komunikuje się z bazą danych. Odpowiedź ponownie przechodzi przez repozytorium, serwis i kontroler i zostaje przekazana użytkownikowi.

4.5. MODUŁY SYSTEMU



Rysunek 4.10: Komunikacja między komponentami przy pobieraniu opublikowanych zadań

4.4.2. Uwierzytelnianie i autoryzacja

System wykorzystuje mechanizm tokenów JWT do uwierzytelniania i autoryzacji żądań. Po pomyślnym zalogowaniu użytkownik otrzymuje token, który jest wymagany do korzystania z większości metod. Serwer API weryfikuje ważność tokenu oraz sprawdza uprawnienia użytkownika przed wykonaniem żądanej operacji. Tokeny posiadają określony czas ważności, po którym użytkownik musi ponownie się uwierzytelnić lub przedłużyć token. Obsługa błędów została zaprojektowana zgodnie ze standardami HTTP, gdzie każda odpowiedź zawiera odpowiedni kod statusu oraz szczegółowe komunikaty opisujące przyczynę ewentualnego niepowodzenia operacji. Serwisy *code-analyzer* i *visualizer* komunikują się z interfejsem użytkownika przez SignalR, czyli otwartoźródłową bibliotekę stworzoną dla ASP.Net Core do wysyłania powiadomień do klienckich sieci Web po stronie klienta.

4.5. Moduły systemu

System został podzielony na logiczne moduły zgodnie z zasadą pojedynczej odpowiedzialności. Moduły są wydzielone niezależnie od ich fizycznego rozmieszczenia w architekturze i większość z nich obejmuje elementy po stronie aplikacji webowej, serwera API i bazy danych. Podział na moduły był szczególnie przydatny na etapie analizy i projektowania zaproponowanego rozwią-

zania. Poza tym obecnie zwiększa czytelność i poprawia organizację kodu, ułatwia rozbudowę oraz umożliwia niezależny rozwój poszczególnych funkcjonalności. Każdy moduł obejmuje ściśle określony zakres funkcjonalności.

4.5.1. Moduł zarządzania użytkownikami

Moduł zarządzania użytkownikami stanowi fundament aplikacji. Odpowiada on za rejestrację, logowanie, uwierzytelnianie, zarządzanie profilami użytkowników, rolami i uprawnieniami oraz preferencjami. Proces rejestracji przebiega następująco: użytkownik najpierw podaje wymagane dane, a potem następuje weryfikacja adresu e-mail przy użyciu usługi *SendGrid*. Podczas logowania i rejestracji użytkownik ma dostęp do funkcji pomocniczych, jak ponowne wysłanie kodu weryfikacyjnego lub resetowanie hasła w przypadku zapomnienia. System ról umożliwia rozróżnienie między zwykłymi użytkownikami a administratorami, którzy posiadają rozszerzone uprawnienia do zarządzania platformą. Każdy nowo zalogowany użytkownik otrzymuje rolę *user* i może ona zostać zmieniona na rolę *admin* w panelu administratora. W panelu administratora dostępna jest również opcja odebrania uprawnień użytkownikom platformy.

4.5.2. Moduł kursów i materiałów edukacyjnych

Moduł kursów obsługuje logikę kursów oraz wykładów – materiałów dydaktycznych. Kursy mogą zawierać wiele lektur, a lektury mogą należeć do wielu kursów. Tak samo wygląda relacja pomiędzy lekturami, a komponentami lektur takimi jak teksty, zdjęcia czy filmy. Umożliwia to ponowne używanie przygotowanego materiału i nie zmusza użytkownika do wykonywania tej samej pracy kilka razy. Moduł umożliwia również przypisanie wykładom tagów, które wskazują na to, w jakim zakresie tematycznym leżą oraz umożliwiają filtrowanie treści. W bazie danych przechowywany jest również indywidualny postęp każdego użytkownika dla każdego z opublikowanych kursów. Jest on automatycznie wyliczany na bieżąco na podstawie procentu ukończonych lektur i zadań dzielonych na sumę wszystkich materiałów w danym kursie.

4.5.3. Moduł zadań programistycznych i interaktywnych

Moduł zadań stanowi centrum nauki praktycznej. Obsługuje on zadania, testy, tagi, podpowiedzi i inne elementy wspierające rozwiązywanie problemów programistycznych. Każde zadanie ma przypisany poziom trudności, opis problemu, ograniczenie czasowe i pamięciowe oraz zestaw przypadków testowych. Przypadki testowe zawierają dane wejściowe, oczekiwane wyniki i maksymalną liczbę punktów, którą otrzymuje użytkownik po rozwiązaniu problemu w określonym czasie. Moduł umożliwia również dodawanie i usuwanie tagów oraz podpowiedzi dla poszczegól-

nych zadań.

4.5.4. Moduł uruchamiania i analizy kodu

Moduł uruchamiania i oceniania kodu jest ściśle powiązany z komponentem *Code Executor*. Zapewnia bezpieczne środowisko do uruchamiania kodu użytkownika w kontenerach Docker oraz analizę kodu w czasie rzeczywistym. Dzięki temu aplikacja jest chroniona przed szkodliwymi kodami takimi jak nieskończona pętla, czy wstrzykiwanie złośliwego kodu. Moduł ten łączy się bezpośrednio z modułem zadań programistycznych. Kiedy użytkownik napisze rozwiązanie problemu i uruchomi swój program, napisany przez niego kod jest odpowiednio formatowany i wystosowywane jest zapytanie do osobnego komponentu odpowiadającego za wykonywanie kodu. Tam program jest uruchamiany, a wyniki porównywane z przypisanymi do danego zadania przypadkami testowymi. Ważną częścią tego procesu jest jasne informowanie użytkownika o błędach. Proces wykonywania kodu dokładniej opisany został w rozdziale 5.

Dodatkowo moduł umożliwia analizę kodu na bieżąco podczas pisania. Dzięki temu użytkownik od razu widzi wszystkie typy błędów występujące w kodzie i może je łatwo zlokalizować dzięki podkreślanii błędnego fragmentu na czerwono. Zaimplementowana została również mechanika podpowiadania i dokończania słów za użytkownika. Podczas pisania moduł sugeruje jak możemy dokończyć dany wyraz dopasowując istniejące metody, funkcje lub wcześniej użyte zmienne. Nie proponuje ona jednak bardziej złożonych rozwiązań (na przykład pełnych funkcji), jak często wygląda to w innych rozwiniętych środowiskach do pisania. Pomijając aspekty logistyczne wprowadzenia takich możliwości, zbyt duże sugestie ze strony edytora kodu mogłyby osłabić wartość edukacyjną platformy.

4.5.5. Moduł wizualizacji

Moduł ten odpowiada za zakładkę wizualizacji, a dokładniej za obsługiwanie, kompilację i prezentację algorytmu napisanego przez użytkownika. Strona została zaprojektowana pod algorytmy operujące na grafach i to im odpowiada widoczna wizualizacja. Domyślny kod, który wyświetla się użytkownikowi po przełączeniu się na zakładkę, jest implementacją algorytmu przeszukiwania grafu. Stanowi on sugestię tego, w jaki sposób powinien wyglądać kod wprowadzany przez użytkownika. Użytkownik ma dużą dowolność wprowadzania własnego kodu, ale obecnie jedynym obsługiwanym językiem – jak w przypadku modułu zadań programistycznych – jest *C#*.

Poza edytorem kodu dostępna również jest tablica do wyświetlania grafów, którą użytkownik może modyfikować przy pomocy dedykowanego panelu. Zawiera on opcje dodawania i usuwa-

nia wierzchołków oraz krawędzi, nadawania wag krawędziom, a także oznaczania wierzchołków startowych i końcowych. Następnie uruchamiany jest napisany przez użytkownika kod i na jego podstawie, na stworzonym grafie przeprowadzona zostaje wizualizacja. Ważnym elementem tego moduły jest również krótka instrukcja obsługi, która krok po kroku przeprowadza użytkownika przez proces korzystania ze strony wizualizacji.

5. Zdalne wykonywanie kodu

Jedną z najważniejszych funkcjonalności stworzonej platformy jest zdalne wykonywanie kodu. Nie jest to zadanie trywialne, nie ma też do tego jednego dobrego podejścia. Do najbardziej znaczących wyzwań należą, w wielu przypadkach wzajemnie się wykluczające, wydajność i bezpieczeństwo. Użytkownik może, niekoniecznie świadomie, napisać kod, którego wykonanie byłoby nierozsądne, chociażby nieskończoną rekursję. W takim przypadku wywołanie programu mogłoby doprowadzić do awarii systemu. Zatem, projektując moduł zdalnego wykonywania kodu, zwany dalej w pracy także *sandboxem* (z ang. piaskownica) albo *code executorem* (z ang. serwis wykonujący kod), zwracano szczególną uwagę na te dwie wspomniane kwestie. W swoich założeniach sandbox powinien być w stanie obsłużyć co najmniej kilkunastu użytkowników w tym samym czasie, testy wydajności modułu zostały opisane w następnym rozdziale.

5.1. Możliwe sposoby implementacji

Zdalne wykonywanie kodu można wykonać na kilka sposobów. Każdy z nich ma swoje wady i zalety. Dobrą praktyką jest rozdzielenie serwisu wykonującego od głównego serwera, jak na przykład w pracy Haohui Liang et al [2]. Duża część z istniejących serwisów do zapewnienia bezpieczeństwa korzysta z narzędzi konteneryzacji takich jak *Docker*. Dzięki API, które on udostępnia, możliwe jest ograniczenie zasobów systemowych, do których użytkownik ma dostęp, w tym ilości pamięci, sieci, liczby procesów, a także plików. Ponadto samo stworzenie kontenera jest stosunkowo wydajne i bezpieczne (Scheepers, Mathijs Jeroen [9]). Bezpieczne wykonywanie kodu jest też możliwe do zaimplementowania na inne sposoby, w szczególności poprzez pełnoprawne maszyny wirtualne, jednakże nie będą one dokładniej omawiane w tej pracy.

5.1.1. Konteneryzacja w Dockerze

Na początku warto wyjaśnić skrótowo kilka podstawowych pojęć związanych z konteneryzacją. To pojęcie jest ściśle powiązane z wirtualizacją, czyli abstrakcją i izolacją środowiska wykonawczego. Kontenery natomiast, w odróżnieniu od maszyn wirtualnych, dzielą jądro (kernel) systemu

operacyjnego maszyny, na którym są tworzone. Tworzone są na podstawie obrazu zdefiniowanego przez użytkownika. Docker udostępnia interfejs umożliwiający zwiększenie bezpieczeństwa wykonywanego kodu, w szczególności przez ograniczenie liczby procesów, ilości pamięci, uprawnień użytkownika, a także dostępu do plików.

5.1.2. Gotowy obraz kontenera

Jedno z możliwych podejść polega na skorzystaniu z odpowiedniego obrazu, by uruchomić program bezpośrednio w kontenerze. Na podstawie przesłanego rozwiązania tworzony jest odpowiedni plik lub całe rozwiązanie. Następnie należy skorzystać z istniejących kompilatorów lub interpreterów, np. w przypadku C++ można by to zrobić przy pomocy GCC.

To rozwiązanie może być jednak mało wydajne. Dużo zależy od tego, czy kontener byłby włączany z poziomu serwera, czy może byłby przygotowany wcześniej. Kolejną wadą takiego podejścia jest także ograniczona możliwość diagnostyki – należałoby wtedy ręcznie parsować standardowe wyjście i standardowe wyjście błędów.

5.1.3. Pula gotowych kontenerów

Możliwe jest także wcześniejsze utworzenie i trzymanie dostępnej puli kontenerów, a także odpowiednie przekierowywanie ruchu pomiędzy nimi (tzw. *load balancing*). W najprostszym przypadku żądania można by wysyłać naprzemiennie. W bardziej skomplikowanym kontenery byłyby dynamicznie tworzone na podstawie zapytań serwera. To rozwiązanie jest stosunkowo trudne w implementacji i utrzymaniu bez korzystania z narzędzi do orkiestracji kontenerów, na przykład takich jak *kubernetes*. Narzędzie to ułatwia zarządzanie wieloma kontenerami, automatyzując i skalując oparte na nich rozwiązania. W tej pracy jednak nie był uwzględniany, przede wszystkim ze względu na jego większą złożoność. Ponadto, by jego użycie było uzasadnione, przewidywany ruch w aplikacji musiałby być znacząco większy. Warto jednak wspomnieć, że rozwiązanie oparte wyłącznie na puli kontenerów z gotowym kompilatorem wiąże się z podobnymi problemami co wcześniej, czyli ograniczoną możliwością diagnostyki błędów.

5.1.4. Kontenery z serwisem wykonującym

Rozwiązanie zaproponowane w ramach tej pracy łączy najważniejsze cechy poprzednio wspomnianych. Do wykonania kodu wykorzystany jest oddzielny kontener, w którym znajduje się program sprawujący kontrolę nad procesem kompilacji kodu. Tworzona jest statyczna liczba takich kontenerów, możliwa do ustawienia przez API Dockera. Zapytania serwera wysyłane są do nich naprzemiennie, a ruchem sieciowym zarządza *load balancer* zaimplementowany w serwe-

rze NGINX, który odpowiada za równomierne rozdzielanie obciążenia pomiędzy poszczególne instancje. Przy braku odpowiedzi od kontenera podejmowane są dalsze próby.

W każdym kontenerze znajduje się serwis wykorzystujący *Roslyn*, czyli zbiór otwartoźródłowych kompilatorów udostępnionych przez firmę Microsoft dla języków programowania C# i Visual Basic. Określany jest on czasem platformą *Compiler-as-a-Service* (z ang. kompilator jako serwis). Dzięki niemu możliwa jest znacząca kontrola całego przebiegu kompilacji kodu, w tym parsowania, analizy semantycznej, a także generowania kodu pośredniego. Ze względu na wydajność, w serwisie nie zdecydowano się na rozdzielenie procesu, na którym wykonywany jest kod użytkownika. Dzięki temu wykonanie kodu w sandboxie zajmuje porównywalną ilość czasu do wykonania na lokalnym komputerze, co ma kluczowe znaczenie, jeśli chodzi o zapewnienie poprawnego funkcjonowania platformy i jej atrakcyjności. Istnieje przez to możliwość wyłączenia kontenera przez wykonywany kod, dzięki Dockerowi możliwe jest jednak szybkie przywrócenie jego sprawności. W tym czasie zapytania są odbierane przez pozostałe repliki.

Wadą podejścia opartego na *Roslyn*ie jest mniejsza możliwość rozszerzalności serwisu. W przypadku, gdyby wymagane było wsparcie innych języków, program musiałby być napisany od nowa. Jednakże samo rozwiązanie cechuje się dobrą wydajnością, a także możliwością łatwego skalowania. W przypadku spodziewanej większej liczby użytkowników aplikacji, możliwa jest łatwa zmiana konfiguracji i stworzenie większej liczby kontenerów. Trochę więcej pracy wymagałaby zmiana polegająca na dynamicznym uruchamianiu kontenerów. Mając na uwadze wszystko powyższe, to podejście zostało uznane za najlepiej spełniające wymagania postawione platformie.

5.1.5. Roslyn – kompilator jako usługa

W tradycyjnych kompilatorach wykonywany proces działa na zasadzie *czarnej skrzynki*, ponieważ nie ma możliwości, by w niego ingerować. W wielu przypadkach nie jest to zwyczajnie potrzebne. Roslyn jednak udostępnia interfejs programistyczny aplikacji (ang. *application programming interface*, **API**) dla następujących etapów (na podstawie: V. Aho, R. Sethi, J.D. Ullman [1])

1. **Analiza składniowa** (parsing). Następuje w drugiej fazie kompilacji, po analizie leksykalnej. Tworzone jest drzewo składniowe, odwzorowujące gramatyczną strukturę kodu.
2. **Analiza semantyczna**. W tym etapie następuje sprawdzenie poprawności semantycznej kodu źródłowego z zasadami języka, na przykład pod kątem spójności typów.
3. **Generowanie kodu pośredniego**. Następnie tworzony jest kod pośredni. Zgodnie z założeniami, powinien być on łatwo tłumaczony na kod maszyny docelowej i szybki do wyge-

nerowania. Roslyn umożliwia wygenerowanie kodu CIL (*Common Intermediate Language*).

W tych etapach możliwe jest dalsze zabezpieczenie kodu, chociażby poprzez wykrywanie wyrażeń niedozwolonych przez platformę. W ramach pracy nie korzystano jednak z dodatkowych mechanizmów, ponieważ zgodnie z postawionymi wcześniej wymaganiami zabezpieczenia narzucone przez konteneryzację powinny być już wystarczające. Dodatkowe mechanizmy wyraźnie zmniejszałyby wydajność serwisu.

5.2. Mechanizm działania

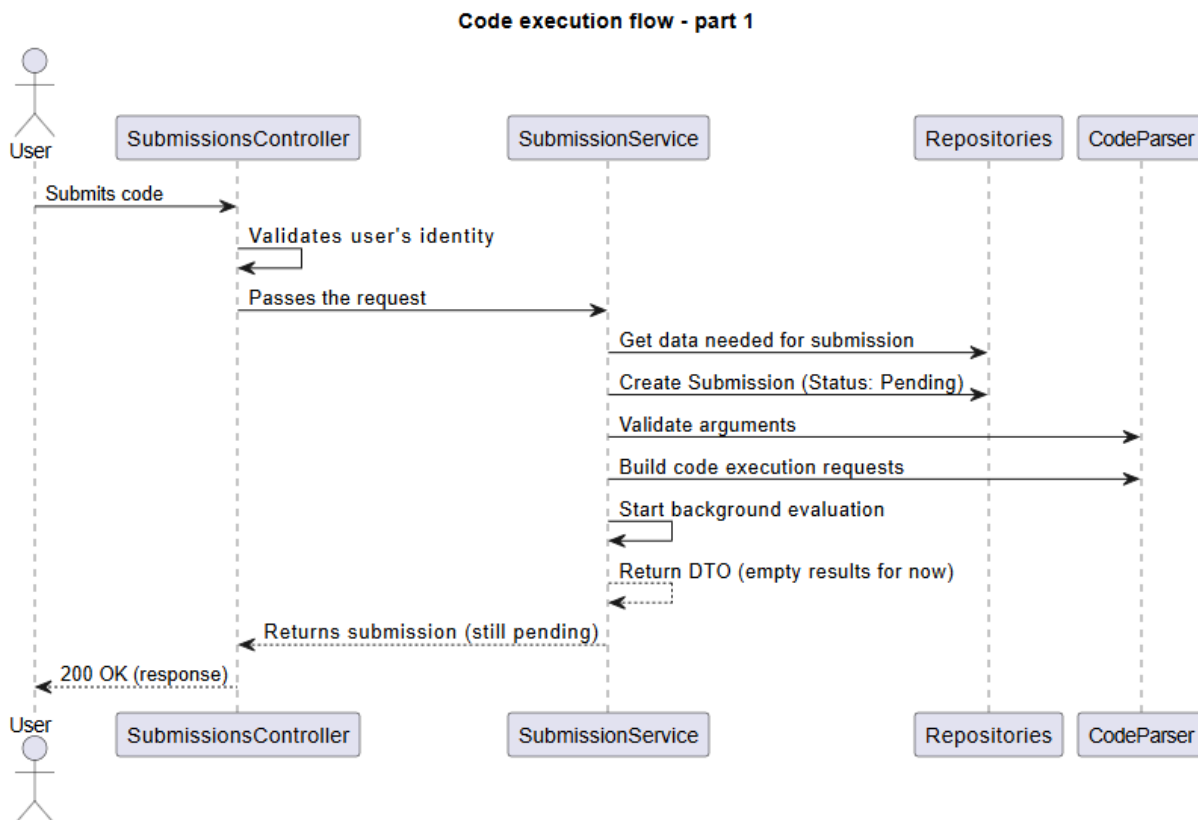
Wykonywanie kodu nadesłanego przez użytkownika jest złożonym procesem, w który zaangażowana jest znaczna część komponentów aplikacji. Na rysunkach 5.1 i 5.2 umieszczono uproszczone schematy, ułatwiające lepsze zrozumienie działania tej funkcjonalności aplikacji. Dla większej czytelności pominięto na nich mniej istotne części, w tym część ścieżek z błędami.

W momencie gdy użytkownik napisze kod i naciśnie przycisk, który go uruchamia, tworzony jest w bazie nowy obiekt typu *Submission*, czyli kolejne rozwiązanie danego zadania. Przypisywany jest mu status *pending*, oznaczający, że rozwiązanie oczekuje na sprawdzenie. Serwer wysyła zapytanie do serwisu odpowiedzialnego za wykonywanie kodu, jednak nie czeka na odpowiedź, by nie blokować interfejsu użytkownika. W ramach odpowiedzi na wysłanie rozwiązania, informuje jedynie warstwę kliencką o tym, czy zostało ono poprawnie dodane do bazy.

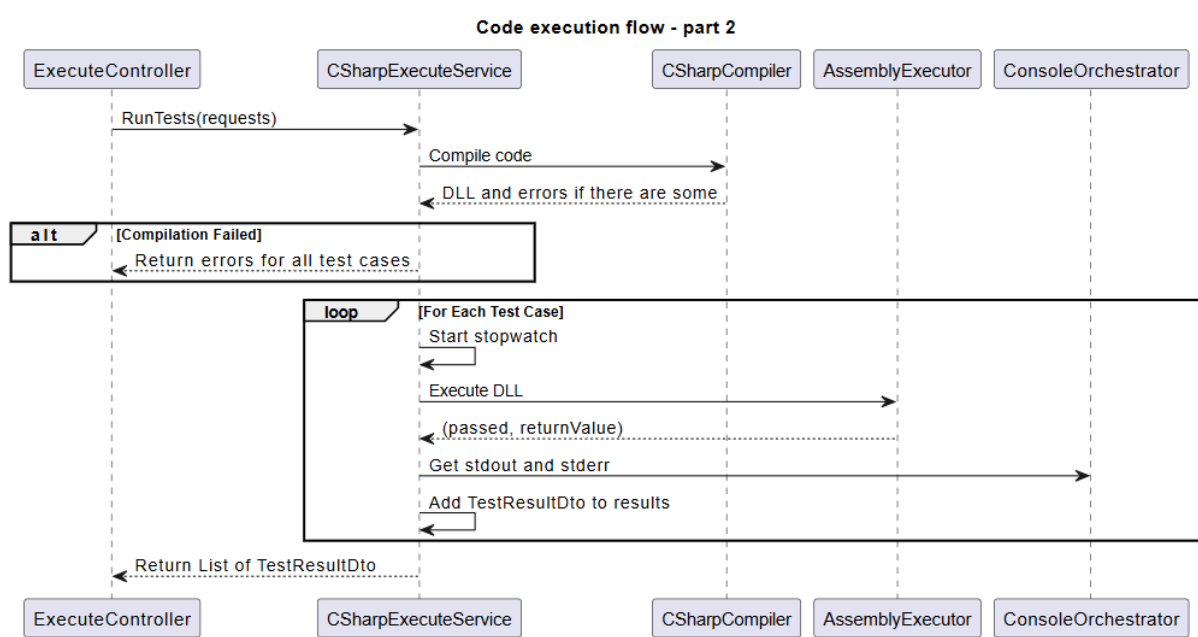
Po otrzymaniu odpowiedzi o poprawnym zapisie aplikacja internetowa cyklicznie odpytuje serwer, czy ewaluacja się już skończyła. Jeśli otrzyma pozytywną odpowiedź, wyświetla odpowiednie wyniki. W tym samym czasie serwer zajmuje się dalszym przygotowaniem kodu do jego uruchomienia. Dokonywane jest odpowiednie sparsowanie kodu i przypadków testowych, by przygotować je w formacie, który oczekiwany jest przez osobny komponent – *code executor*.

Code executor to znajdujący się w oddzielnym kontenerze serwis API, który otrzymuje odpowiednio przygotowane przez backend zapytanie. Na podstawie kodu użytkownika tworzony jest kod pośredni. Po jego wygenerowaniu, program użytkownika jest przechowywany w pamięci w postaci *MemoryStream*. Program generujący kod pośredni został przedstawiony na listingach 9 i 10.

5.2. MECHANIZM DZIAŁANIA



Rysunek 5.1: Uproszczony schemat sekwencji wykonywania kodu – cz.1 (interfejs i backend)



Rysunek 5.2: Uproszczony schemat sekwencji wykonywania kodu – cz.2 (sandbox)

```

1  public CSharpCompilationResult Compile(string code)
2  {
3      // Tworzenie drzewa składniowego
4      SyntaxTree tree = CSharpSyntaxTree.ParseText(code);
5
6      // Opcje kompilacji
7      CSharpCompilationOptions compilationOptions = new(
8          // kompilacja do biblioteki DLL
9          outputKind: OutputKind.DynamicallyLinkedLibrary,
10         // większe optymalizacje, usuwanie martwego kodu
11         optimizationLevel: OptimizationLevel.Release,
12         allowUnsafe: false, // zakaz blokow unsafe
13         checkOverflow: true, // sprawdzanie przepelnien
14         platform: Platform.X64, // architektura procesora
15         concurrentBuild: true, // uzycie kompilacji rownoleglej
16         deterministic: true, // deterministyczna kompilacja
17         // obsługa typow nullable
18         nullableContextOptions: NullableContextOptions.Enable,
19         // ingoruj diagnostyki oznaczone jako wyciszone
20         reportSuppressedDiagnostics: false,
21         warningLevel: 4, // maksymalna liczba ostrzezen
22         // domyslne raportowanie diagnostyki
23         generalDiagnosticOption: ReportDiagnostic.Default
24     );
25
26     // Obiekt kompilacji - pojedyncze wywołanie kompilatora
27     CSharpCompilation compilation = CSharpCompilation.Create(
28         AssemblyName,
29         [tree],
30         GetMetadataReferences(),
31         compilationOptions
32     );
33
34
35
36     ...

```

Listing 9: Fragment kodu odpowiedzialny za generowanie kodu pośredniego – cz.1

5.2. MECHANIZM DZIAŁANIA

```
1      // Opcje emisji kodu pośredniego
2      EmitOptions emitOptions = new(
3          // format do debugowania (bez znaczenia)
4          debugInformationFormat: DebugInformationFormat.Embedded,
5          // klasyczne adresowanie
6          highEntropyVirtualAddressSpace: false);
7
8      // Emisja kodu pośredniego do pamięci
9      using MemoryStream ms = new();
10     EmitResult result = compilation.Emit(ms, options: emitOptions);
11
12     // Zbieranie błędów
13     if (!result.Success)
14     {
15         List<ExecutionErrorDto> errors = [... result.Diagnostics
16             .Where(d => d.Severity == DiagnosticSeverity.Error)
17             .Select(d => new ExecutionErrorDto(d)) ];
18
19         return new(false, null, errors);
20     }
21
22     ms.Seek(0, SeekOrigin.Begin);
23
24     return new(true, ms, []);
25 }
```

Listing 10: Fragment kodu odpowiedzialny za generowanie kodu pośredniego – cz.2

Proces wykonywania kodu zarządzany jest przez *CSharpExecuteService* (Rys. 5.2). Przyjmuje on jako argument obiekt *ExecuteCodeRequest* (Rys. 11), zawierający informacje m.in. o metodzie do wywołania, klasie w której się ona znajduje, czy limicie czasu. Każdy taki obiekt odpowiada jednemu przypadkowi testowemu (*TestCase*).

```

1 public class ExecuteCodeRequest
2 {
3     public Guid TestCaseId { get; set; }
4     public string Code { get; set; } = string.Empty;
5     public List<FunctionParameter> Args { get; set; } = [];
6     public TimeSpan Timeout { get; set; } = TimeSpan.FromSeconds(5);
7     public string ExecutionClass { get; set; } = "Solution";
8     public string ExecutionMethod { get; set; } = "Solve";
9     public string ExpectedValue { get; set; } = string.Empty;
10    public int MaxPoints { get; set; } = 10;
11 }

```

Listing 11: Dane przesyłane serwisowi wykonującemu kod

```

1 public class FunctionParameter
2 {
3     public string Name { get; set; } = string.Empty;
4     public string Value { get; set; } = string.Empty;
5 }

```

Listing 12: Sposób przekazania parametrów serwisowi wykonującemu kod

CSharpExecuteService po otrzymaniu kodu pośredniego, przekazuje go dalej do serwisu odpowiedzialnego za uruchomienie programu i sprawdzenie wyniku (*AssemblyExecutor*), przekazując mu pozostałe potrzebne dane.

Następnie, korzystając z mechanizmu refleksji, wywoływana jest przekazana metoda *ExecutionMethod*. Jeśli zwróci oczekiwany wynik (*ExpectedValue*) w ramach limitu czasu (*Timeout*), test zostaje zaliczony pozytywnie. Uproszczony kod klasy *AssemblyExecutor*, odpowiedzialny za wykonanie funkcji, został przedstawiony na listingu 13

5.2. MECHANIZM DZIAŁANIA

```
1 public (bool? passed, object? returnedValue) Execute(  
2     MemoryStream assemblyStream,    // Wygenerowany kod  
3     string className,                // Klasa metody  
4     string methodName,               // Metoda do wywołania  
5     List<FunctionParameter>? args,   // Lista parametrow  
6     string expectedValue             // Oczekiwany wynik  
7 )  
8 {  
9     Assembly assembly = Assembly.Load(assemblyStream.ToArray());  
10    Type? type = assembly.GetType(className);  
11    if (type == null) throw new Exception(...);  
12  
13    object? instance = isStatic ? null :  
14        Activator.CreateInstance(type);  
15  
16    MethodInfo? method = type.GetMethod(methodName);  
17  
18    if (method == null) throw new Exception(...);  
19  
20    ParameterInfo[] parametersInfo = method.GetParameters();  
21    object?[] parameters = args.ConvertArgs(parametersInfo);  
22  
23    object? returnedValue = method.Invoke(instance, parameters);  
24  
25    var expectedValueConverted =  
26        JsonConvert.DeserializeObject(expectedValue, method.ReturnType);  
27  
28    return (  
29        Equals(expectedValueConverted, returnedValue),  
30        returnedValue  
31    );  
32 }
```

Listing 13: Uproszczony fragment kodu odpowiedzialny za wywołanie funkcji

Konkretne przypadki, na których testowana jest funkcja, przekazywane są w postaci parametrów, zawartych w obiekcie *FunctionParameter* (Rys. 12). Taki obiekt zawiera nazwę parametru oraz wartość, z którą wywołana zostaje funkcja. Argumenty przekazywane są w formie tekstowej (*string*). Może to się na początku wydawać ograniczeniem, ale dzięki temu praktycznie każdy typ

może być porównywany, o ile ma zaimplementowany serializator i metodę *Equals*. Parametry są deserializowane dynamicznie, a ich typ jest uzyskiwany również przez refleksję.

Na przykład w celu uruchomienia funkcji zaprezentowanej na listingu 14, należy podać wartości dwóch argumentów – *a* oraz *b*. Jeśli te wartości są równe odpowiednio 3 oraz 2, dwa obiekty *FunctionParameter* przyjmą formę: (Name = "a", Value = "3") i (Name = "b", Value = "2"). Liczba takich obiektów zawsze odpowiada liczbie parametrów wejściowych funkcji.

```
1 public class Solution
2 {
3     public int AddTwoNumbers(int a, int b)
4     {
5         return a + b;
6     }
7 }
```

Listing 14: Przykładowy fragment kodu z funkcją dodającą do siebie dwie liczby

6. Raport z testów aplikacji

W celu zapewnienia wysokiej jakości oraz niezawodności platformy AlgoRhythm przeprowadzono kompleksowy proces testowania na różnych poziomach architektury systemu. Podczas pracy, wraz ze wzrostem rozmiarów projektu, testy były niezbędne do ciągłego sprawdzania czy nanoszone zmiany nie wpływają negatywnie na już wprowadzone funkcjonalności. Mechanizmy testowania zostały zintegrowane z platformą Github, dzięki czemu przy każdym nadesłaniu zmian w kodzie do repozytorium wyniki testów były automatycznie weryfikowane. Pozwoliło to na wykrywanie błędów na wczesnym etapie oraz zapobiegało scalaniu kodu, który tych testów nie przechodził. Poza tym pomocne okazały się narzędzia do generowania podsumowań i raportów z testów, które umożliwiły sprawdzenie, jaka część kodu jest pokryta testami. Wspierały także dalsze plany rozwoju i pracę – pomagały zaplanować kolejne przypadki testowe i ścieżki, które pomogą zwiększyć pokrycie.

6.1. Testy jednostkowe

Pierwszym rodzajem testów, które powstały w ramach pracy, były testy jednostkowe. Przede wszystkim skupiono się na pisaniu ich w ramach modułu wykonywania kodu użytkowników, a także przy funkcjonalnościach, które umożliwiły powstanie panelu administratora. Jako że duża część modułów aplikacji nie mogłaby istnieć w izolacji od pozostałych, to inne rodzaje testów okazały się bardziej przydatne.

Powstałe testy jednostkowe weryfikują różnorodne scenariusze wykonywania kodu. Obejmują zarówno te przypadki, gdy użytkownik nadesłał poprawne rozwiązanie, jak i te, w których kod zawierał błędy składniowe, semantyczne oraz logiczne. Do pisania testów jednostkowych wykorzystano narzędzia takie jak xUnit (po stronie serwera) i vitest (po stronie interfejsu użytkownika). Przykład testu jednostkowego został przedstawiony na listingu 15. Test ten sprawdza funkcję odpowiadającą za zmianę formy nadesłanego kodu na *ExecutreRequest*, czyli za wyodrębnienie z niej nazwy klasy, funkcji oraz argumentów.

```

1  [Fact]
2  public void ParseToExecuteRequest_ComplexReturnType_ParsesCorrectly()
3  {
4      var code = @"
5          public class Solution
6          {
7              public List<int> GetNumbers(int count)
8              {
9                  return new List<int>();
10             }
11         };
12
13     var result = _parser.ParseToExecuteRequest(code);
14
15     Assert.NotNull(result);
16     Assert.Equal("Solution", result.ExecutionClass);
17     Assert.Equal("GetNumbers", result.ExecutionMethod);
18     Assert.Single(result.Args);
19     Assert.Equal("count", result.Args[0].Name);
20 }

```

Listing 15: Test jednostkowy parsowania kodu

6.2. Testy integracyjne

W ramach tworzenia testów powstały również testy integracyjne, które pozwoliły na testowanie zależności między poszczególnymi komponentami systemu oraz integracji z bazą danych i zewnętrznymi usługami. Aby trzymać się dobrych praktyk programistycznych, testy tworzone od razu przy wprowadzeniu nowej funkcjonalności. Ze względu na wielowarstwową architekturę aplikacji ten rodzaj testów był najczęściej wykorzystywany. Jako dodatkową zaletę tych testów można wymienić fakt, że wiele testów tego typu miało powtarzalną strukturę i aby napisać nowe, można było skorzystać z tych już istniejących.

Testy integracyjne na początku tworzą instancje serwera API oraz serwisu wykonujące kod (o ile było to konieczne), a także korzystają z bazy danych zapisanej w pamięci. W ramach testu wykonywane jest zapytanie do kontrolera, które następnie – poprzez serwis oraz repozytorium – trafia do bazy danych. Przykładowe testy integracyjny są widoczne na listingach 16 i 17.

6.2. TESTY INTEGRACYJNE

```
1 [Fact]
2 public async Task POST_ForgotPassword_UnverifiedEmail_Returns400()
3 {
4     string email = $"unverified-{Guid.NewGuid()}@test.com";
5     await AddUserToDb(email, "User", TestConstants.TestUserPassword, emailConfirmed: false);
6
7     var request = new ResetPasswordRequestDto { Email = email };
8     var content = new StringContent(JsonConvert.SerializeObject(request));
9
10    var response = await _httpClient.PostAsync("/api/authentication/forgot-password", content);
11
12    Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
13 }
```

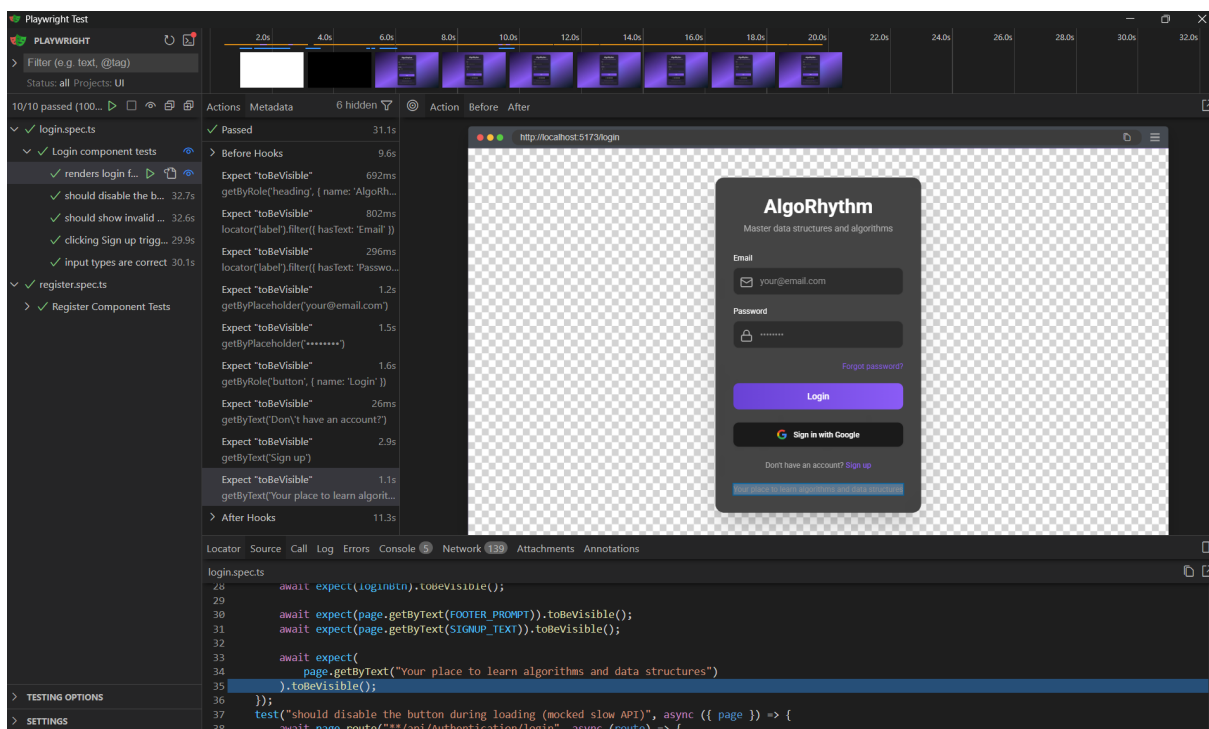
Listing 16: Test integracyjny – próba skorzystania z możliwości przypomnienia hasła przez użytkownika, który nie został zweryfikowany, powinna zakończyć się błędem

```
1 [Fact]
2 public async Task POST_ForgotPassword_ValidEmail_Returns200()
3 {
4     string email = $"forgot-{Guid.NewGuid()}@test.com";
5     await AddUserToDb(email, "User", TestConstants.TestUserPassword, emailConfirmed: true);
6
7     var request = new ResetPasswordRequestDto { Email = email };
8     var content = new StringContent(JsonConvert.SerializeObject(request),
9         Encoding.UTF8, "application/json");
10
11    var response = await _httpClient.PostAsync(
12        "/api/authentication/forgot-password", content);
13
14    response.EnsureSuccessStatusCode();
15    var responseBody = await response.Content.ReadAsStringAsync();
16    Assert.Contains("password reset code has been sent", responseBody);
17 }
```

Listing 17: Test integracyjny – próba skorzystania z możliwości przypomnienia hasła przez użytkownika, która powinna zakończyć się powodzeniem

6.3. Testy interfejsu użytkownika (UI)

W ramach tworzenia aplikacji webowej postanowiono również przetestować powstały interfejs użytkownika. W tym celu stworzono testy UI, których główną rolą była weryfikacja poprawności działania interfejsu, a także zapewnienie intuicyjności oraz responsywności aplikacji webowej w różnych przeglądarkach. Do ich napisania wykorzystano narzędzie Playwright. Ze względu na ich dłuższy czas pisania i wykonywania, testowano jednak wyłącznie kluczowe ścieżki użytkownika. Na ten moment powstałe testy obejmują przede wszystkim procesy logowania oraz rejestracji. Przykładowy test został zaprezentowany na Rys. 6.1.



Rysunek 6.1: Testy UI logowania

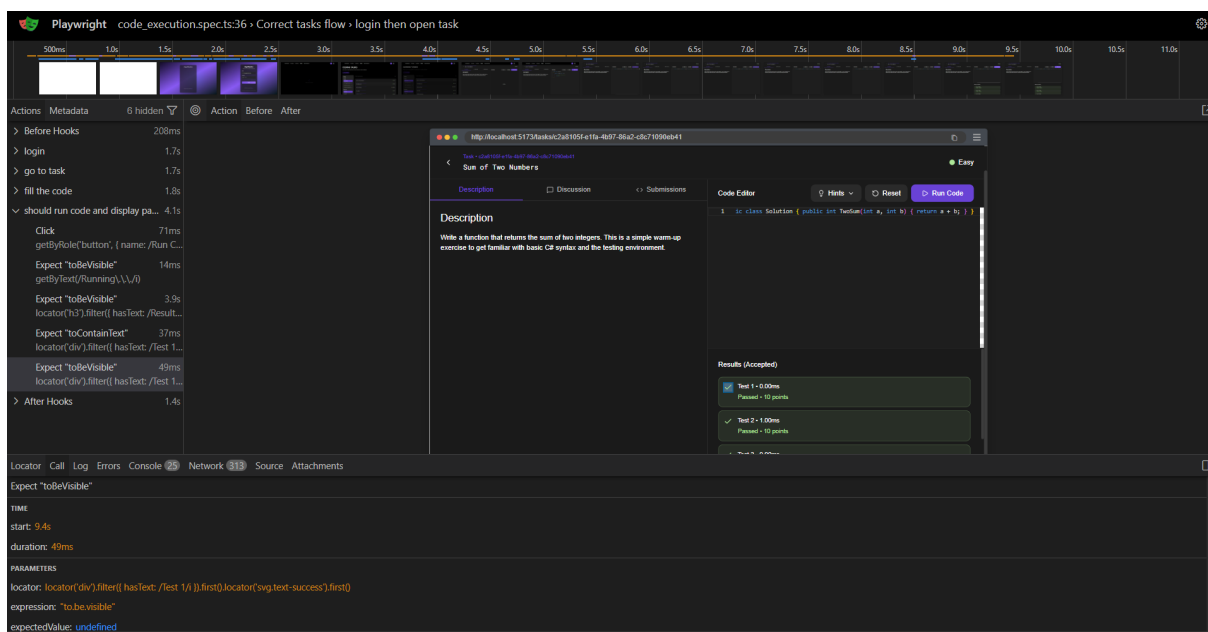
6.4. Testy end-to-end (E2E)

W procesie testowania oprogramowania stworzono też testy end-to-end (z ang. *od początku do końca*), również napisane z wykorzystaniem frameworka Playwright. Skupiały się one na kompleksowej weryfikacji funkcjonalności na prawdziwej aplikacji. W czasie ich trwania tworzone było pełnoprawne rozwiązanie kontenerowe. Z tego powodu były jednak zdecydowanie bardziej czasochłonne, zarówno w uruchomieniu, jak i napisaniu. Ponadto cechują się wrażliwością komputera, na którym są wykonywane.

6.5. TESTY WYDAJNOŚCIOWE

Same testy E2E i UI są też, ze względu na swoją specyfikę, trudne w utrzymaniu. W trakcie wykonywania testu framework potrzebuje specyficznych selektorów, by wiedzieć na jaki element nacisnąć, które w trakcie rozwoju aplikacji często mogą się zmieniać. Z tego względu zdecydowano się przetestować w ten sposób wyłącznie krytyczne ścieżki, czyli proces logowania i przesłania kodu do oceny.

Przykładowy test typu end-to-end przedstawiono na Rys. 6.2.



Rysunek 6.2: Testy end-to-end wykonywania kodu

6.5. Testy wydajnościowe

W celu oceny wydajności systemu oraz jego zdolności do obsługi wielu jednoczesnych żądań przeprowadzono testy wydajnościowe z wykorzystaniem narzędzia Postman. Testy skupiały się na sprawdzaniu kodu, ponieważ to przy nim występują największe możliwości problemów z wydajnością.

Sprawdzono wydajność wykonania kodu obliczającego czterdziesty piąty wyraz ciągu Fibonacciego, przedstawionego na listingu 18. Przeprowadzony test trwał jedną minutę i był wykonywany na pojedynczym kontenerze. Podczas jego trwania liczba aktywnych użytkowników wahała się od 5 do 20, a liczba wysyłanych żądań wynosiła od 1 do 7 na sekundę. Średni czas wykonywania kodu wyniósł 8814 ms.

Dla porównania ten sam kod został wykonany dziesięciokrotnie na lokalnej maszynie w trybie debugowania. W tym przypadku uśredniony czas wykonywania wyniósł około 9500 ms. Należy

zauważyć, że podczas lokalnego uruchamiania kodu użytkownik nie będzie zazwyczaj kompilować aplikacji w trybie *release*, co ogranicza możliwość zastosowania znacznych optymalizacji przez kompilator.

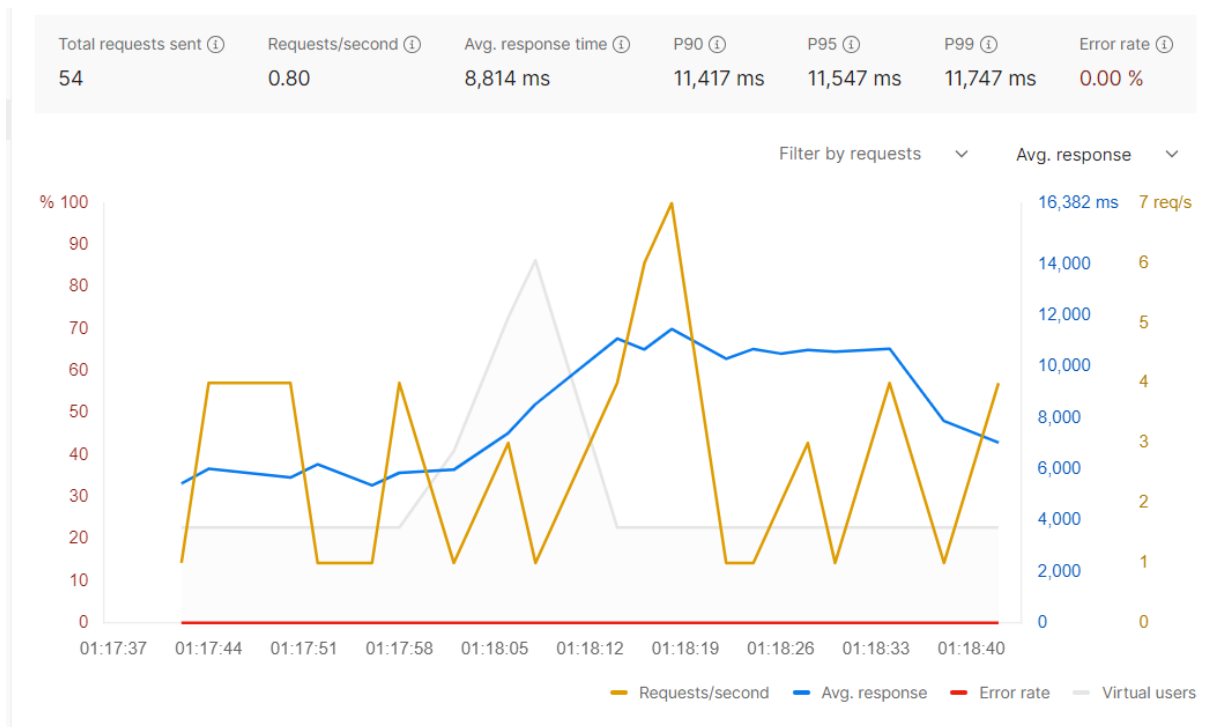
W przypadku serwisu wykonującego kod proces ten przebiega w trybie zoptymalizowanym, przeznaczonym do publikacji. Dzięki temu możliwe jest skrócenie czasu wykonywania kodu i szczególnie przy większym ruchu takie rozwiązanie pozwala zamaskować dodatkowy nakład czasowy związany z obsługą pozostałych zapytań.

```

1  int Fibonacci(int n)
2  {
3      if (n <= 1) return n;
4      return Fibonacci(n - 1) + Fibonacci(n - 2);
5  }

```

Listing 18: Kod użyty do testów wydajnościowych



Rysunek 6.3: Wyniki testów wydajnościowych

6.6. PODSUMOWANIE WYNIKÓW TESTÓW

6.6. Podsumowanie wyników testów

W ramach pracy skorzystano z narzędzi do automatycznego generowania raportów z testów, zintegrowanych z pozostałymi wspomnianymi wcześniej narzędziami. Dzięki temu potwierdzić można było dobre przetestowanie kluczowych funkcjonalności, co zostało przedstawione na Rys. 6.4, 6.5 i 6.6. Przeprowadzone testy na wszystkich poziomach architektury systemu potwierdziły, że platforma AlgoRhythm spełnia założone wymagania funkcjonalne i нефункционалне, a także wspomogły proces rozwoju oprogramowania. Niestety, dogłębne sprawdzenie wszystkich możliwych interakcji na tak rozbudowanej platformie nie jest wykonalne w tak małym zespole. Aby finalnie przetestować całe narzędzie, konieczne byłoby również skorzystanie z pomocy dużej grupy aktywnych użytkowników, którzy mogliby na bieżąco zgłaszać zauważone problemy. Mimo to testy w aplikacji okazały się bardzo pomocne. Przyspieszały postępy w pracy i pomagały unikać błędów, a jeśli takowe wystąpiły, to wspierały szybką naprawę.

File	Statements	Branches	Functions	Lines
CourseFormModal.tsx	82.7% 110/133	83.33% 55/66	91.89% 34/37	84.48% 98/116
LectureContentModal.tsx	75.58% 65/86	76.66% 69/90	79.16% 19/24	76.54% 62/81
LectureFormModal.tsx	97.33% 73/75	92.85% 26/28	100% 19/19	97.01% 65/67
LecturePreviewModal.tsx	92% 23/25	96.66% 29/30	100% 5/5	95.45% 21/22
TagManager.tsx	79.48% 31/39	70.37% 19/27	100% 10/10	81.57% 31/38
TaskFormModal.tsx	82.63% 138/167	66% 66/100	92.3% 36/39	83.66% 128/153
TestCaseManager.tsx	76.19% 32/42	72.09% 31/43	63.63% 14/22	80% 32/40

Rysunek 6.4: Raport z pokrycia kodu testami panelu administratora

CodeExecutor	140	53	193	409	72.5%
CodeExecutor.Controllers.ExecuteController	6	0	6	28	100%
CodeExecutor.Helpers.ArgumentConverter	5	0	5	27	100%
CodeExecutor.Helpers.ConsoleOrchestrator	15	0	15	36	100%
CodeExecutor.Program	18	1	19	43	94.7%
CodeExecutor.Helpers.AssemblyExecutor	14	1	15	56	93.3%
CodeExecutor.Services.CSharpExecuteService	82	51	133	219	61.6%

Rysunek 6.5: Raport z pokrycia kodu testami wykonywania kodu

AlgoRhythm.Services.Tasks.HintService	34	0	34	69	100%	4	4	100%
AlgoRhythm.Services.CodeExecutor.CSharpCodeParser	89	1	90	163	98.8%	43	52	82.6%
AlgoRhythm.Services.Common.TagService	40	1	41	83	97.5%	9	12	75%
AlgoRhythm.Services.Common.CommentService	45	2	47	87	95.7%	10	14	71.4%
AlgoRhythm.Services.CodeExecutor.CodeExecutorService	10	1	11	38	90.9%	2	4	50%
AlgoRhythm.Services.Users.UserStreakService	40	5	45	92	88.8%	21	22	95.4%
AlgoRhythm.Services.Users.AuthService	504	84	588	1018	85.7%	124	174	71.2%
AlgoRhythm.Services.Tasks.TaskService	99	21	120	222	82.5%	44	68	64.7%
AlgoRhythm.Services.Courses.CourseService	76	26	102	167	74.5%	19	48	39.5%
AlgoRhythm.Services.Submissions.SubmissionService	182	110	292	421	62.3%	20	40	50%

Rysunek 6.6: Raport z pokrycia kodu testami kluczowych serwisów

7. Instrukcja obsługi platformy

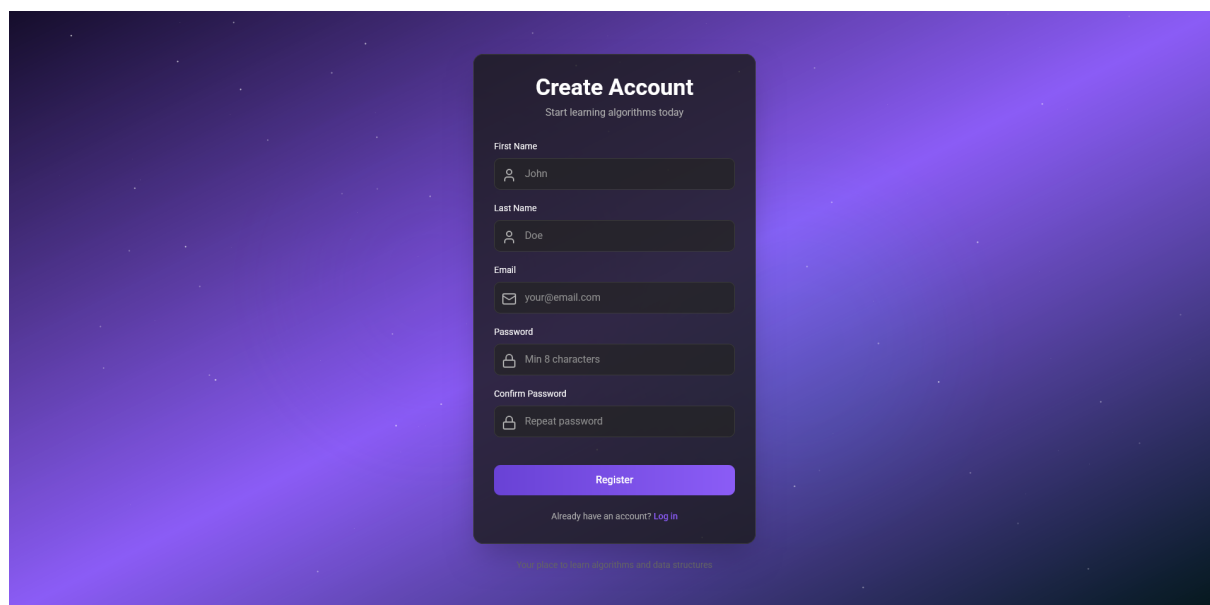
Niniejszy rozdział przedstawia sposób działania stworzonej platformy internetowej.

7.1. Proces rejestracji i logowania

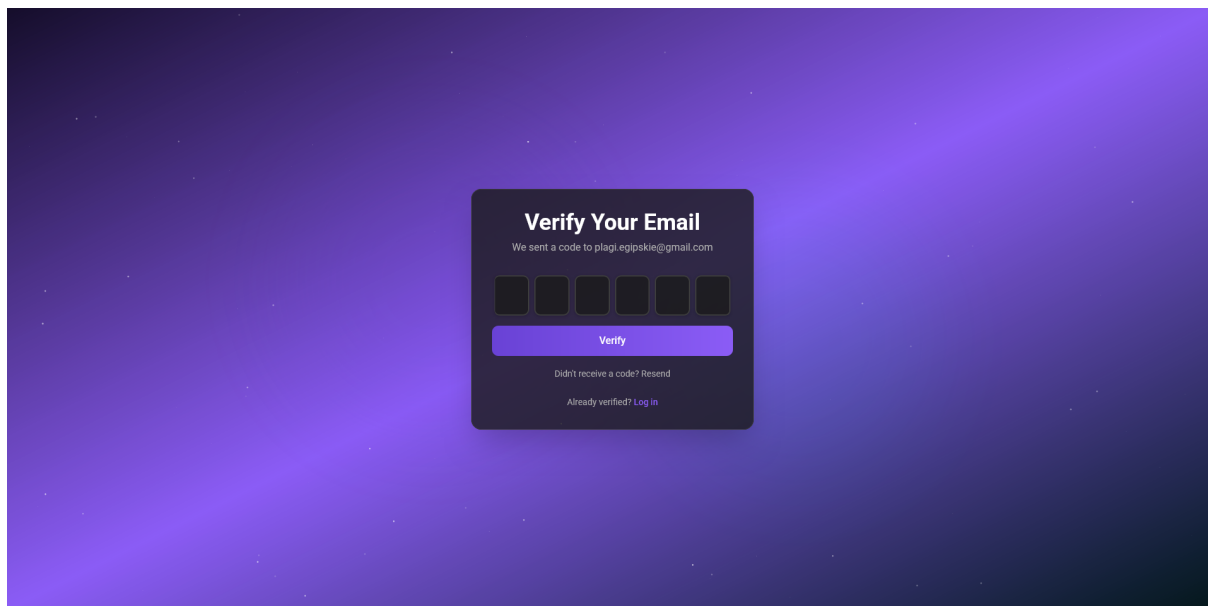
Ekranem, który wyświetla się użytkownikowi po włączeniu platformy, jest ekran logowania. W celu dalszego uzyskania dostępu do funkcjonalności systemu konieczne jest przejście przez proces autoryzacji.

7.1.1. Rejestracja

Jeśli użytkownik nie posiada jeszcze konta, ma możliwość stworzenia go. W tym celu konieczne już uzupełnienie formularza danymi (Rys. 7.1). Po kliknięciu przycisku, na podany adres e-mail zostaje wysłany kod weryfikacyjny, który następnie trzeba podać w aplikacji 7.2. Następnie użytkownik zostanie przekierowany do części aplikacji dostępnej dla zalogowanych użytkowników.

The image shows a mobile application interface for creating a new account. The background is a dark blue gradient. In the center is a white rounded rectangle containing the registration form. At the top of the form, it says 'Create Account' in bold, followed by the subtitle 'Start learning algorithms today'. The form has five input fields: 'First Name' with the value 'John', 'Last Name' with the value 'Doe', 'Email' with the value 'your@email.com', 'Password' with a hint 'Min 8 characters', and 'Confirm Password' with a hint 'Repeat password'. Each field has a small icon to its left (person for names, envelope for email, and key for passwords). Below the fields is a large blue button labeled 'Register'. At the bottom of the form, there is a link that says 'Already have an account? Log in'. At the very bottom of the white rectangle, in small text, it says 'You agree to our algorithms and data practices'.

Rysunek 7.1: Widok rejestracji



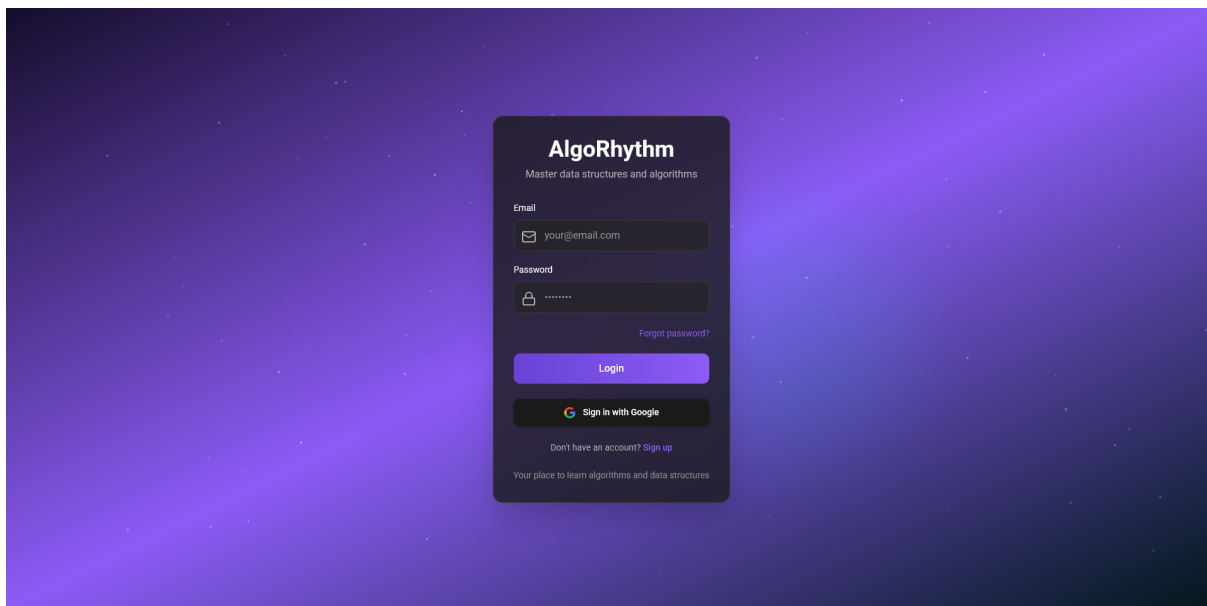
Rysunek 7.2: Widok weryfikacji e-mail

7.1.2. Logowanie

Aby zalogować się do aplikacji, użytkownik musi uzupełnić formularza poprzez podanie adresu e-mail, na który wcześniej zostało stworzone konto oraz odpowiadającego mu hasła (Rys. 7.3). Po wpisaniu danych należy wcisnąć przycisk *Login*. Jeśli wpisane dane są poprawne i powiązane z istniejącym kontem, użytkownik zostanie automatycznie przekierowany na główną stronę aplikacji. Jeśli nie, wyświetlona mu zostanie informacja o błędzie.

Alternatywną metodą uwierzytelniania jest logowanie z wykorzystaniem konta Google. W tym celu użytkownik wybiera przycisk *Sign in with Google* widoczny pod formularzem logowania. Proces ten nie wymaga wcześniejszej rejestracji w systemie, konieczne jest jedynie posiadanie aktywnego konta Google. Po przejściu przez proces uwierzytelniania użytkownik otrzymuje taki sam dostęp do platformy jak w przypadku standardowego logowania.

Użytkownik ma również przejścia przez proces resetu hasła. W tym celu musi wcisnąć przycisk *Forgot Password?*. Jeśli użytkownik nie posiada konta, powinien nacisnąć przycisk *Sign in*. Zostanie przekierowany na ekran rejestracji.



Rysunek 7.3: Widok logowania

7.2. Nawigacja

Po poprawnym zalogowaniu użytkownik ma dostęp do wszystkich funkcji aplikacji poza panelem administratora. Aby poruszać się między różnymi sekcjami, powinien korzystać z umieszczonego na górze paska z zakładkami. Użytkownik może wybierać z poniższych zakładek:

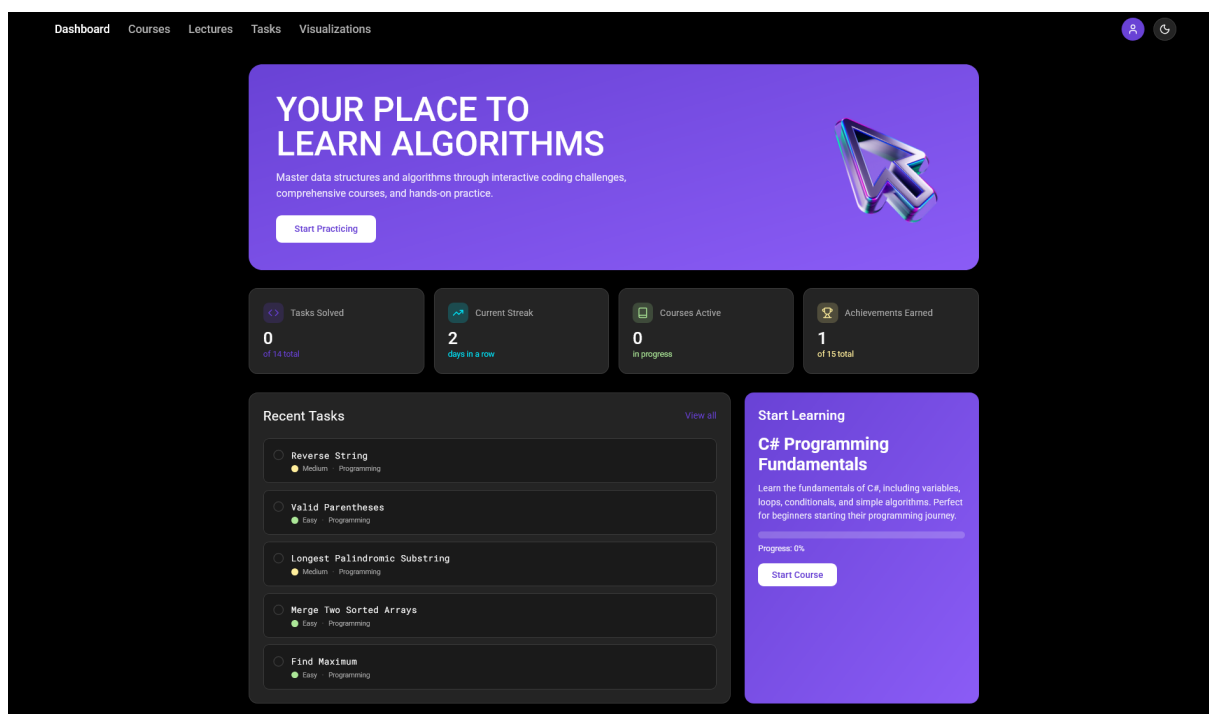
1. **Dashboard** – główny ekran aplikacji, który prezentuje podstawowe statystyki użytkownika i dalsze dostępne kroki.
2. **Lectures** – materiały teoretyczne, wykładowe.
3. **Tasks** – materiały praktyczne, czyli zadania programistyczne.
4. **Visualisations** – interaktywne wizualizacje algorytmów.
5. **Courses** – kursy, łączące materiały teoretyczne i zadania programistyczne.

Poza nimi na górnym pasku nawigacji znajduje się również ikonka, która przekierowuje użytkownika na stronę jego profilu oraz przycisk, dzięki któremu można przełączać się między trybem jasnym a ciemnym.

7.2. NAWIGACJA

7.2.1. Dashboard

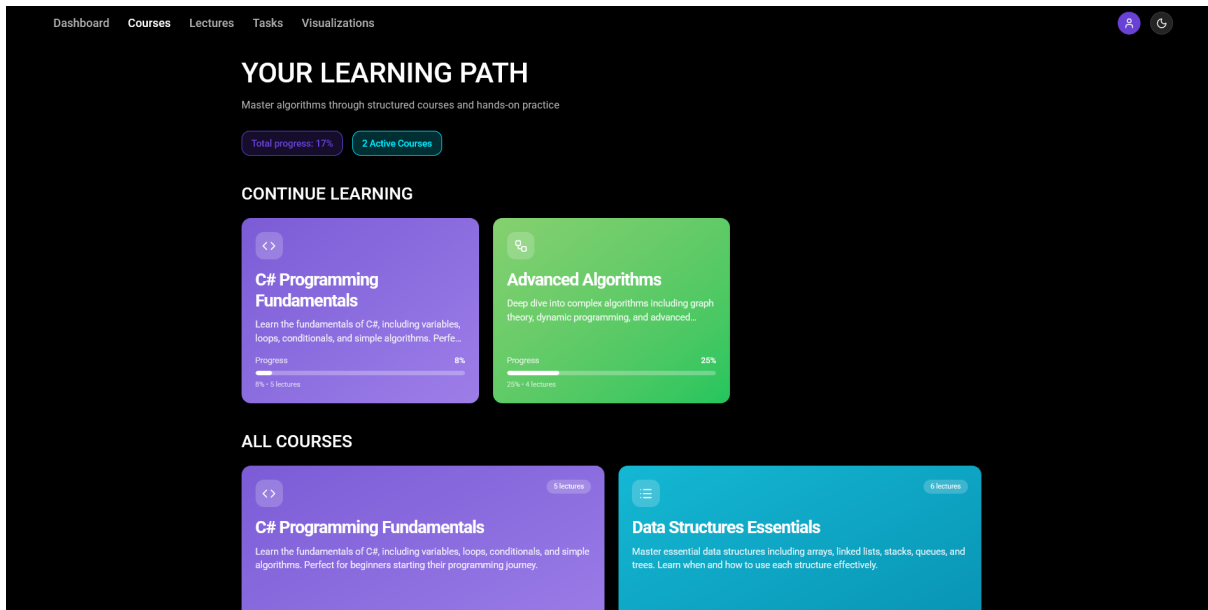
Na głównym dashboardzie aplikacji (Rys. 7.4) użytkownik ma możliwość podejrzenia swoich podstawowych statystyk oraz dalszego nawigowania do innych podstron platformy. Zadania na liście *Recent Tasks* kierują do widoków konkretnych zadań. Po prawej stronie mamy podgląd jednego z dostępnych kursów, a jeśli użytkownik rozpoczął już któryś z nich, wyświetli się on w tym miejscu z możliwością kontynuacji.



Rysunek 7.4: Dashboard

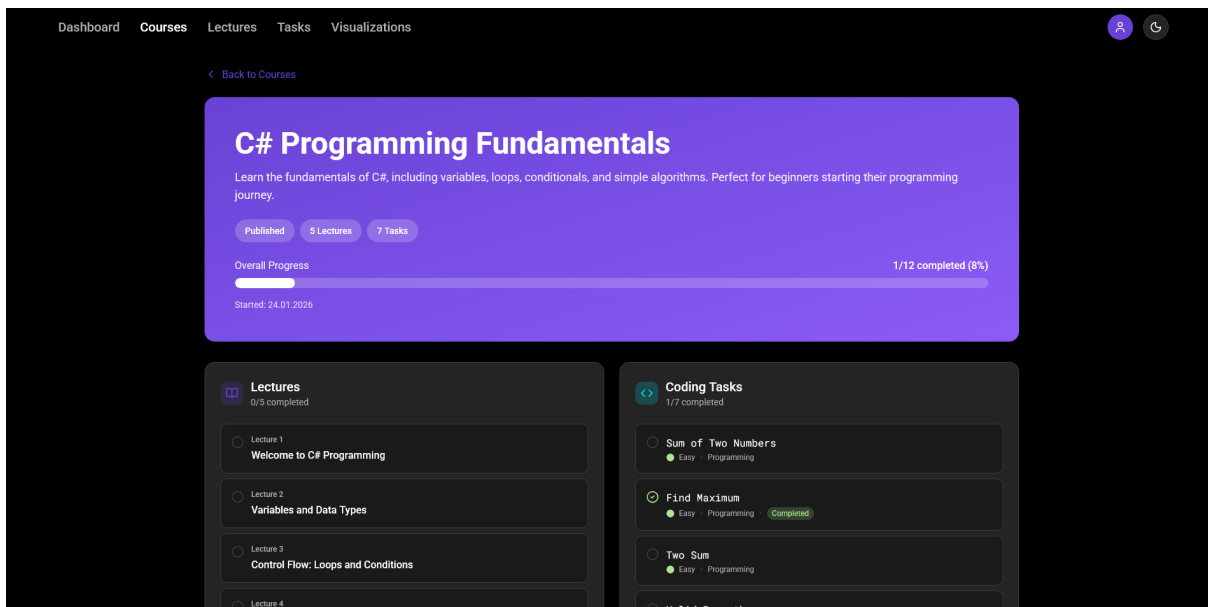
7.2.2. Lista kursów

Po kliknięciu **Courses** użytkownikowi zostaje zaprezentowany podgląd dostępnych i aktywnych już kursów (Rys. 7.5), które tematycznie łączą ze sobą treści teoretyczne i praktyczne. Po kliknięciu wybranego, zostanie przekierowany na jego widok.

Rysunek 7.5: *Courses* – widok listy kursów

7.2.3. Widok kursu

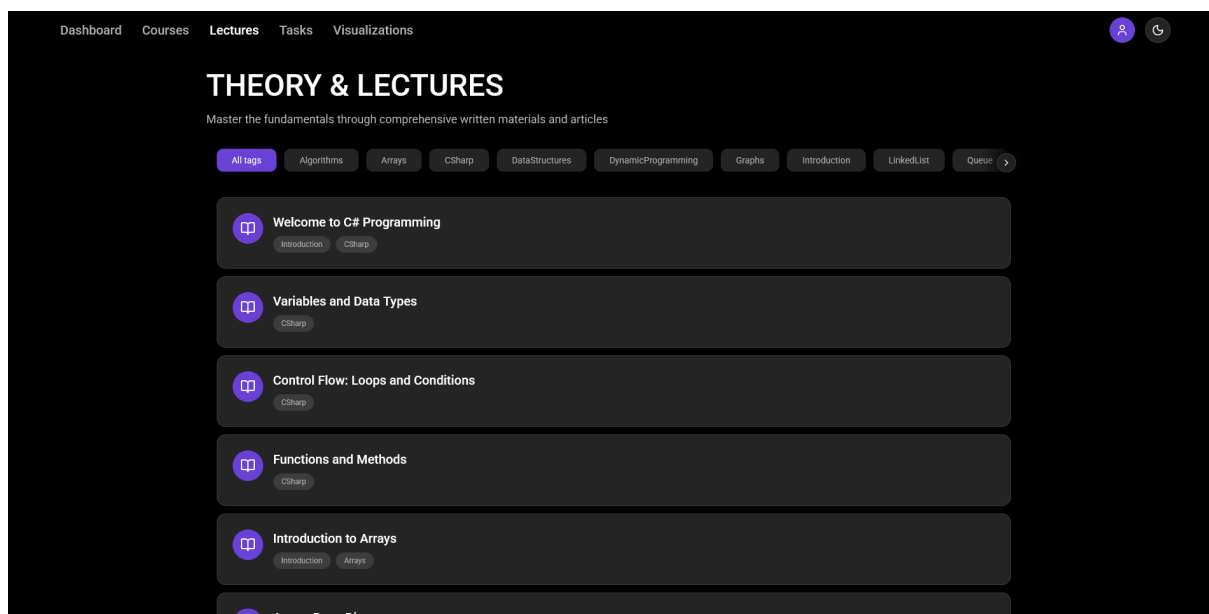
Po przejściu do widoku konkretnego kursu (Rys.7.6) użytkownik ma podgląd zadań i wykładów, które wchodzą w jego skład, a także wskaźnika z procentem rozwiązanych i przeczytanych już treści. Dzięki tej stronie może w łatwy sposób monitorować swoje postępy. Postępy te naliczają się automatycznie.



Rysunek 7.6: Widok kursu

7.2.4. Lista wykładów

Po przejściu do strony **Lectures** (Rys. 7.7), użytkownik uzyskuje dostęp do listy opublikowanych treści wykładowych. Użytkownik może zapoznawać się z nimi w dowolnej kolejności. Ma również możliwość skorzystania z systemu tagów. Dzięki nim może filtrować wykłady, dzięki temu zawężając wyświetlaną listę do zagadnień, które akurat go interesują.



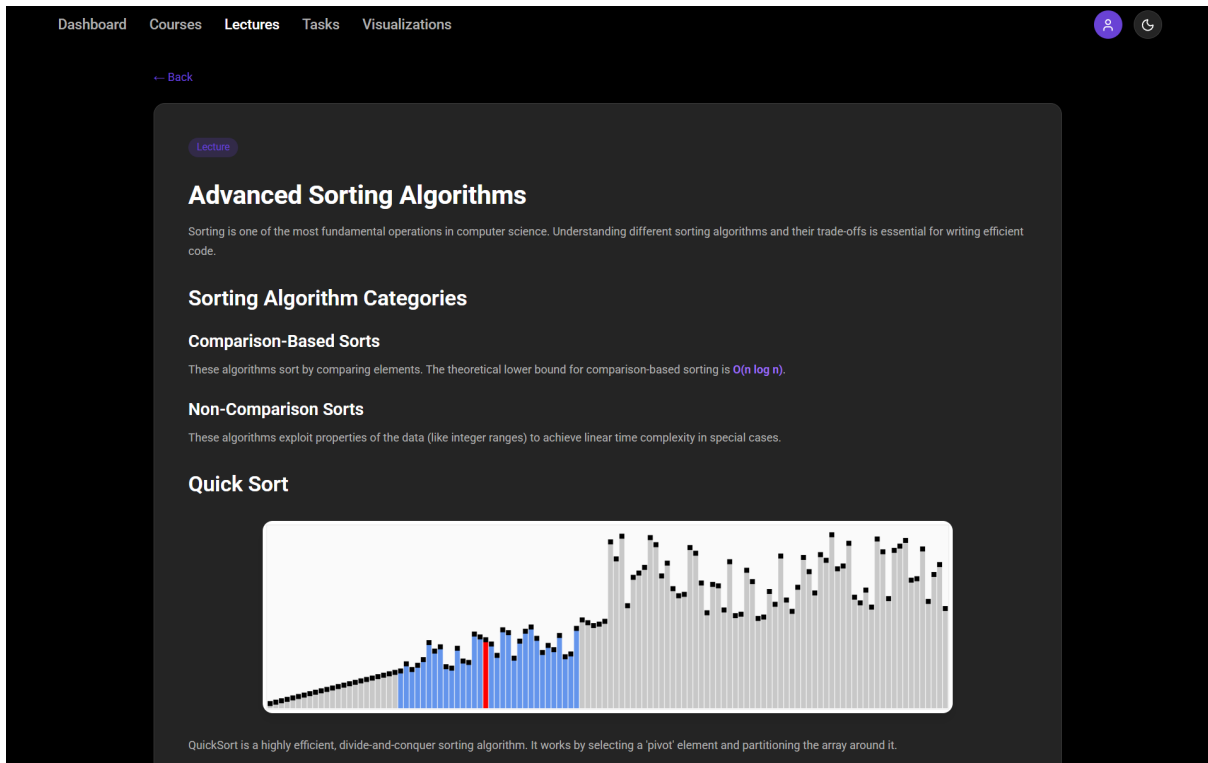
Rysunek 7.7: Widok listy wykładów

7.2.5. Widok wykładu

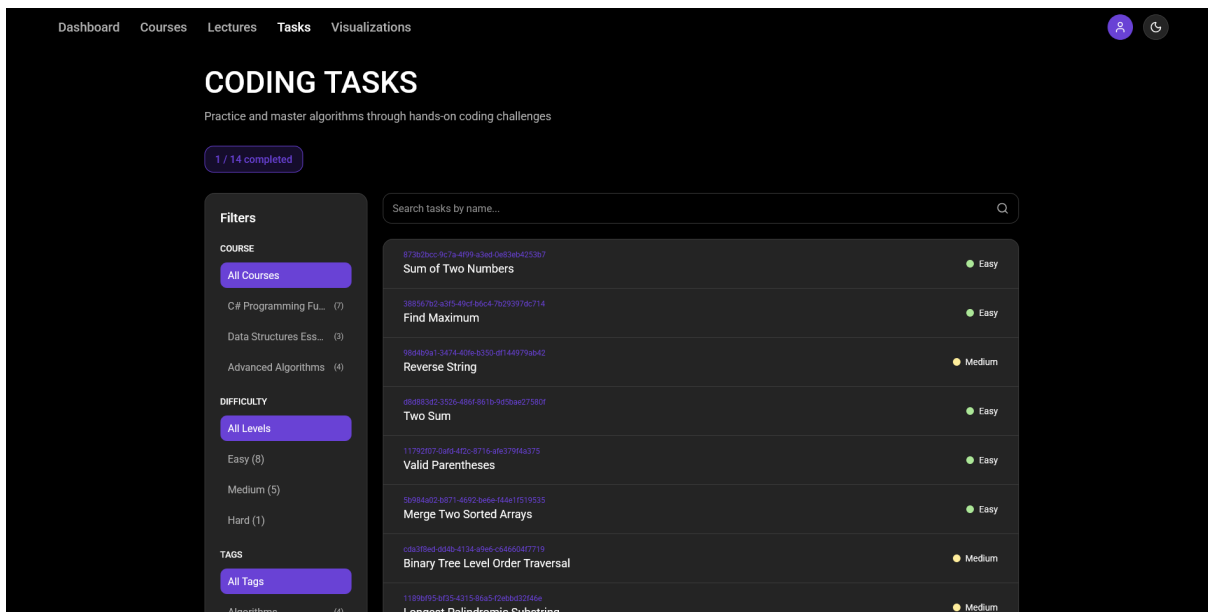
Na widoku konkretnego wykładu (Rys. 7.8) użytkownik może zapoznać się z jego pełną treścią. Po przeczytaniu całości powinien nacisnąć przycisk *Complete*, który znajduje się na dole ekranu. Jedynie wtedy wykład zostanie oznaczony jako przeczytany i będzie później wliczał się do zaliczonych wykładów w ramach kursów, do których należy.

7.2.6. Lista zadań

Zakładka **Tasks** (Rys. 7.9) przedstawia listę opublikowanych zadań programistycznych. Znajdująca się po lewej stronie sekcja umożliwia filtrowanie ich po kursach, do których należą, tagach oraz poziomach trudności. Filtrowanie następuje automatycznie po wybraniu jednej z opcji. Aby przejść do konkretnego zadania, należy wybrać je z listy.



Rysunek 7.8: Widok wykładu

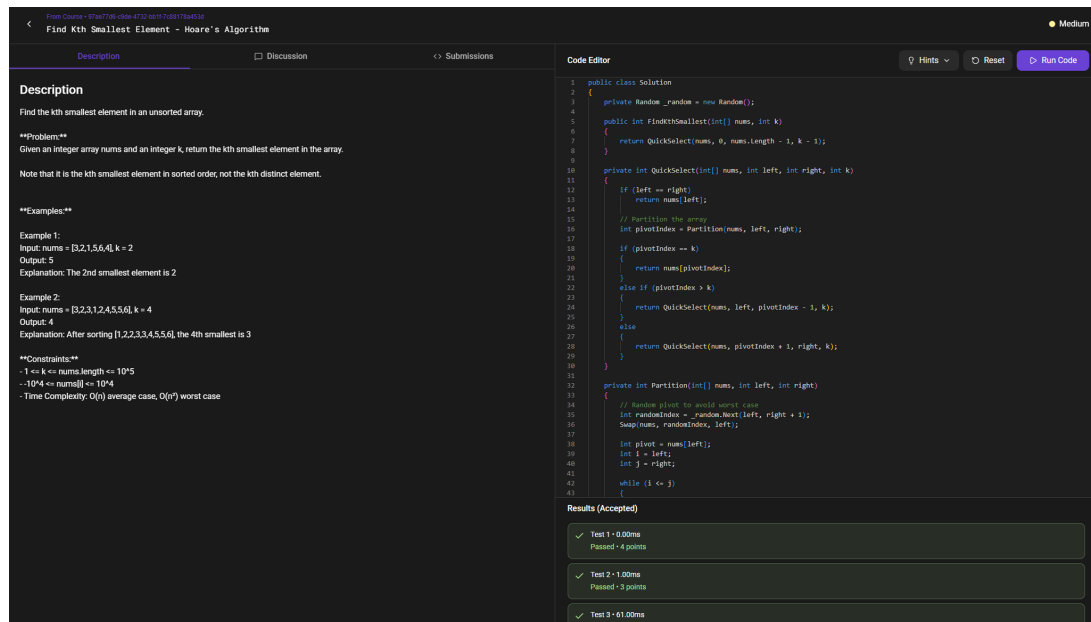


Rysunek 7.9: Widok listy zadań

7.2. NAWIGACJA

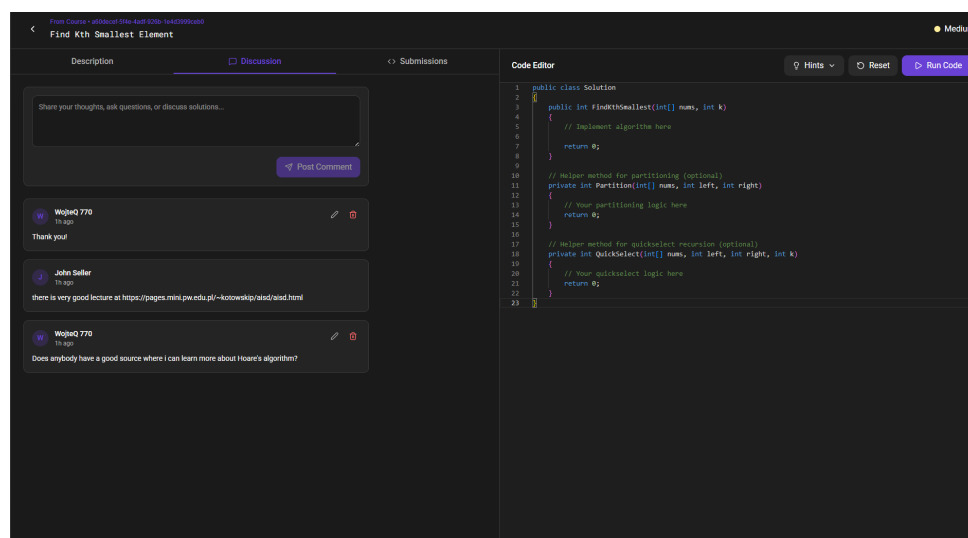
7.2.7. Widok zadania

Strona zadania (Rys. 7.10) składa się z dwóch głównych części: lewej, informacyjnej, i prawej, którą tworzy przede wszystkim edytor kodu.



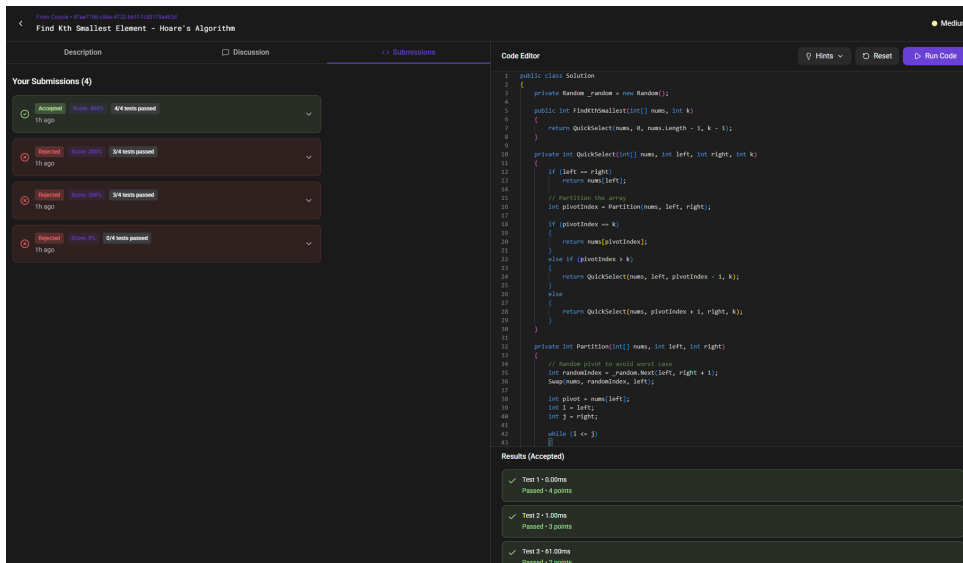
Rysunek 7.10: Widok zadania

Po lewej stronie użytkownik może przełączać się pomiędzy trzema zakładkami. Pierwsza z nich prezentuje opis zadania, wystarczający, by móc poprawnie je rozwiązać. Druga zapewnia dostęp do komentarzy, które powiązane są z poszczególnymi zadaniami. Zamieszczony komentarz można następnie edytować oraz usunąć.



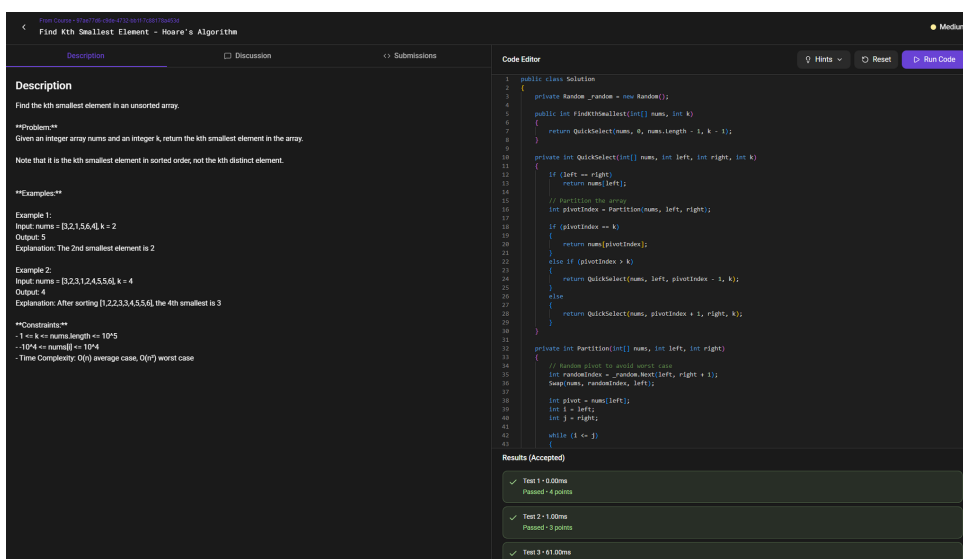
Rysunek 7.11: Widok zadania – zakładka z dyskusją

Ostatnia zakładka przedstawia poprzednie rozwiązania danego zadania zgłoszone przez użytkownika. Widoczna jest na rysunku 7.12.



Rysunek 7.12: Widok zadania – zakładka ze zgłoszeniami rozwiązań

Po prawej stronie znajduje się edytor kodu ze wstępnym kodem, na którym można bazować swoje rozwiązanie zadania. Podczas pisania, wyświetlają się w nim podpowiedzi dotyczące kodu, a błędy podkreślone są na czerwono. Użytkownik ma możliwość uruchomienia kodu za pomocą przycisku *Run Code* oraz podejrzenia wyników w dedykowanej sekcji poniżej (Rys. 7.13). Przycisk *Hints* umożliwia wyświetlenie listy podpowiedzi do zadań, które można odsłonić kliknięciem na nie.



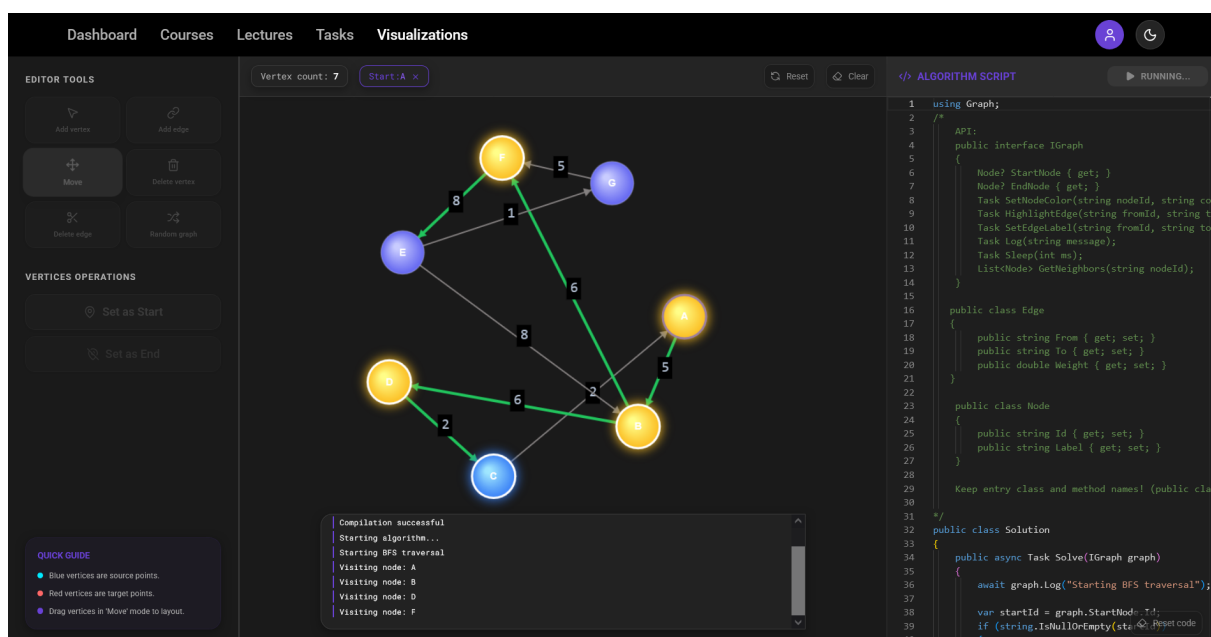
Rysunek 7.13: Widok zadania po uruchomieniu kodu

7.2.8. Widok wizualizacji

Po przejściu do zakładki *Visualizations*, użytkownikowi wyświetla się widok tworzenia, edytowania i uruchamiania wizualizacji (Rys. 7.14). Przy pierwszym przejściu na tę stronę powinien wyświetlić się szczegółowy tutorial jej użytkowania.

Strona składa się z trzech części:

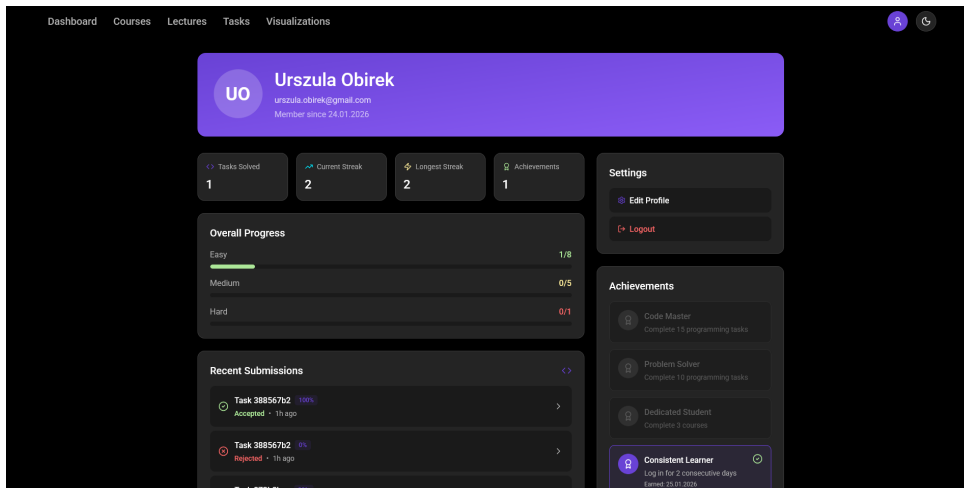
1. Panel po lewej umożliwia edytowanie grafu wyświetlonego na środku. Użytkownik może dodawać i usuwać wierzchołki oraz krawędzie, zmieniać wagi krawędzi oraz ustawić wierzchołek początkowy i końcowy algorytmu.
2. Środkowa część widoku przedstawia stworzony przez użytkownika graf. W trakcie działania programu wierzchołki mogą zmieniać kolory, aby wizualnie pokazać działanie algorytmu. Na samym dole widoczne są logi.
3. Ostatnia, znajdująca się po prawej część strony to edytor kodu z możliwością jego uruchomienia. Kod, który pojawia się na początku, może stanowić dla użytkownika wskazówkę tego, w jaki sposób powinno wyglądać jego rozwiązanie, by kod został uruchomiony poprawnie.



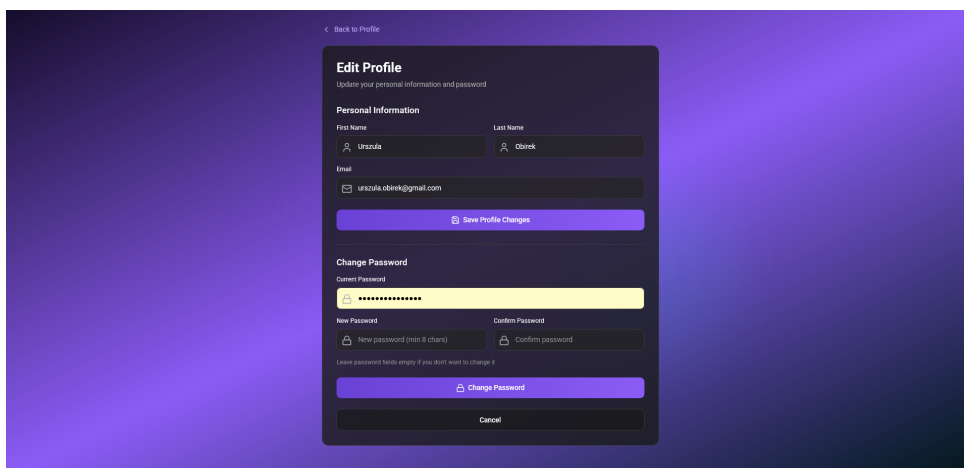
Rysunek 7.14: Widok wizualizacji po uruchomieniu algorytmu

7.2.9. Profil użytkownika

Na stronie swojego profilu (Rys. 7.15) użytkownik może podejrzeć więcej swoich statystyk, w tym również osiągnięcia (*Achievements*), które jak dotąd zdobył podczas aktywności na platformie, a także te, które jeszcze może zdobyć. Może również sprawdzić swój obecny *streak*, czyli liczbę dni, przez które codziennie logował się do aplikacji aż po aktualny dzień, oraz najdłuższy *streak* jak dotąd. Poza tym ma również możliwość przejścia do strony edycji danych (Rys. 7.16), która umożliwia zmianę imienia, nazwiska, maila oraz hasła.



Rysunek 7.15: Widok profilu użytkownika



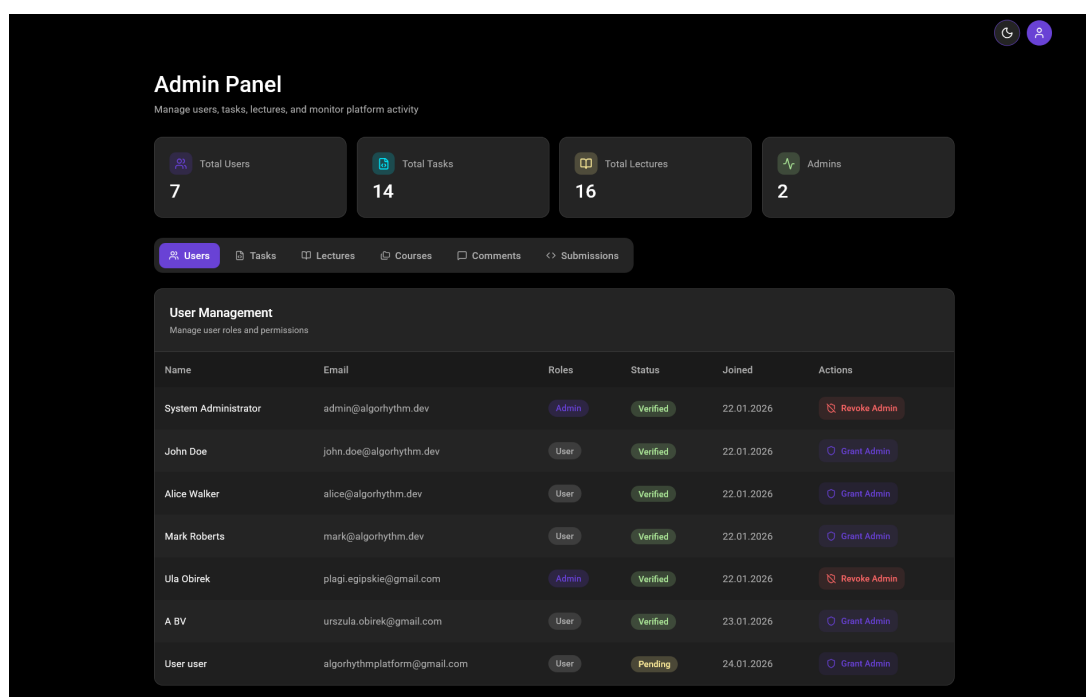
Rysunek 7.16: Widok edycji profilu

7.3. Panel administratora

Panel administratora (Rys. 7.17) jest stroną dostępną jedynie dla osób, których konta posiadają uprawnienia administratora. Po poprawnym procesie logowania na takie konto, użytkownik zostaje przekierowany bezpośrednio na specjalny panel. Nie ma możliwości utworzenia konta administratora – aby uzyskać jego uprawnienia, należy utworzyć zwykłe konto i skontaktować się z administratorem, który nada konieczne uprawnienia.

Administrator może korzystać z następujących zakładek:

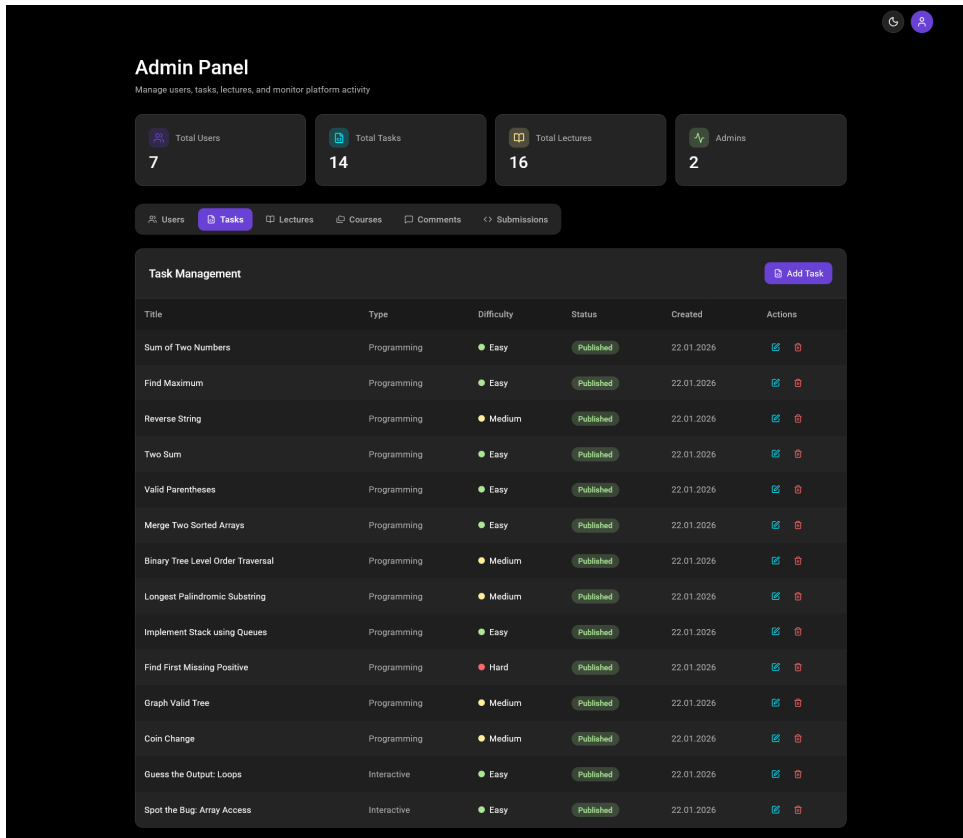
1. **Users** – zakładka, w której wyświetla się lista użytkowników. Administrator może odbierać i nadawać im uprawnienia administratora
2. **Tasks** – zakładka do zarządzania zadaniami programistycznymi.
3. **Lectures** – zakładka do zarządzania materiałami dydaktycznymi.
4. **Courses** – zakładka do zarządzania kursami.
5. **Comments** – zakładka do monitorowania komentarzy.
6. **Submissions** – zakładka do monitorowania zgłoszeń rozwiązań zadań.



Rysunek 7.17: Widok panelu administratora

7.3.1. Dodanie nowego zadania i edytowanie istniejącego

W celu dodania nowego zadania programistycznego, użytkownik musi przejść do zakładki *Tasks* (Rys. 7.18).



Rysunek 7.18: Widok zakładki z zadaniami w panelu administratora

Następnie należy nacisnąć przycisk *Add Task*. Otworzy się formularz. W celu poprawnego dodania zadania, konieczne jest jego uzupełnienie. Szczególną uwagę należy zwrócić na stworzenie test casów – ważny jest ich format, w którym najpierw, w polu *Input* konieczne jest podanie parametrów i ich wartości, a w polu *Expected* wartości, którą zwróci funkcja. Poza tym kluczowe jest stworzenie poprawnego, przykładowego kodu, który otworzy się użytkownikowi po przejściu na ekran zadania. Każdy taki kod, by później mógł być poprawnie wykonany, powinien składać się z publicznej klasy (domyślnie *Solution*) oraz dowolnej liczby funkcji, z których pierwsza będzie tą, której wynik będzie oceniany. Poprawnie uzupełniony formularz widoczny jest na rysunku 7.19.

7.3. PANEL ADMINISTRATORA

Edit Task

Title *

Sum of Two Numbers

Description

Write a function that returns the sum of two integers. This is a simple warm-up exercise to get familiar with basic C# syntax and the testing environment.

Task Type *

Programming

Difficulty *

Easy

Tags

Introduction

CSharp

+ Add Tag

Hints

+ Add Hint

Hint 1

Use the + operator (shift and =) to add two numbers together and return the result.

Template Code

```
public class Solution
{
    public int Sum(int a, int b)
    {
        // Write your solution here
        return 0;
    }
}
```

Test Cases

+ Add Test Case

Define input and expected output for automated testing

Test Case #1

Hidden

10 pts

Input:

{ "a": 0, "b": 0 }

Expected:

0

Test Case #2

Visible

10 pts

Input:

{ "a": 1, "b": 2 }

Expected:

3

Test Case #3

Visible

10 pts

Input:

{ "a": -5, "b": 10 }

Expected:

5

☒ Publish task immediately

Cancel

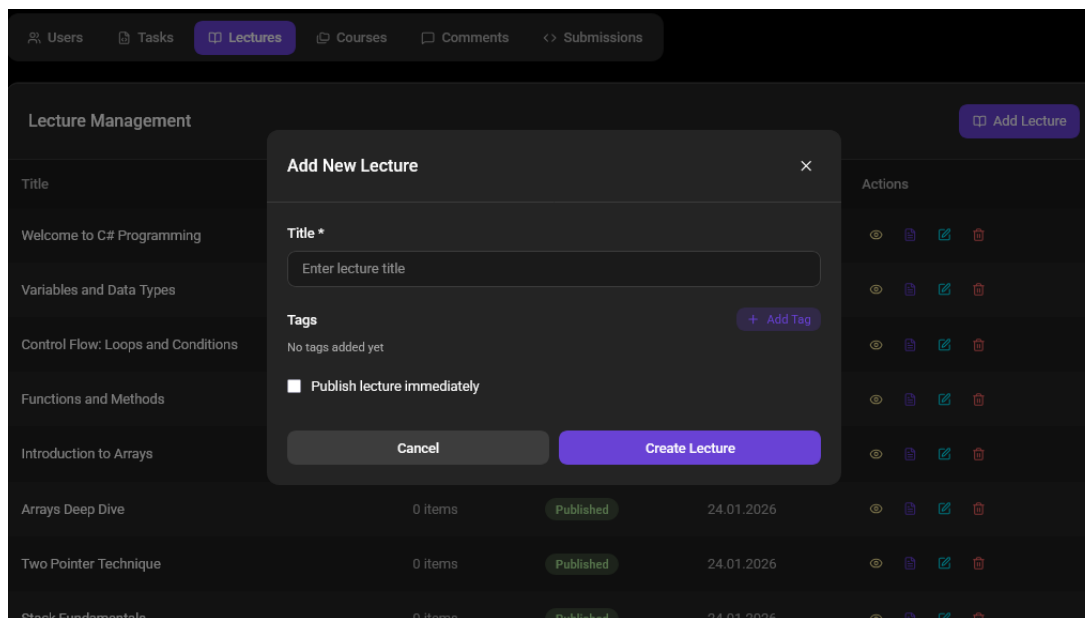
Update Task

Rysunek 7.19: Widok edytowania zadania – uzupełniony formularz

83

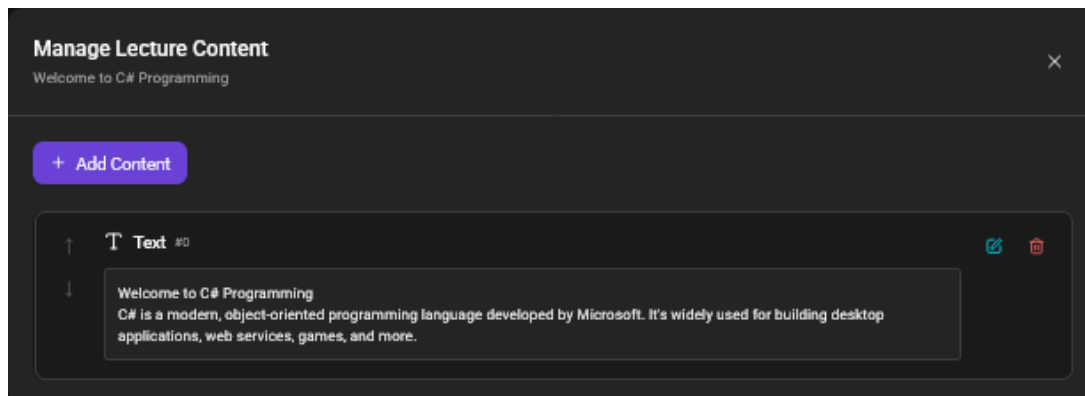
7.3.2. Dodanie nowego wykładu i edytowanie istniejącego

W celu dodania nowego materiału dydaktycznego, użytkownik musi przejść do zakładki wykładów oraz nacisnąć przycisk *Add lecture*. Otworzy się formularz (Rys. 7.20), w którym należy nadać wykładowi tytuł oraz przypisać do niego tagi.



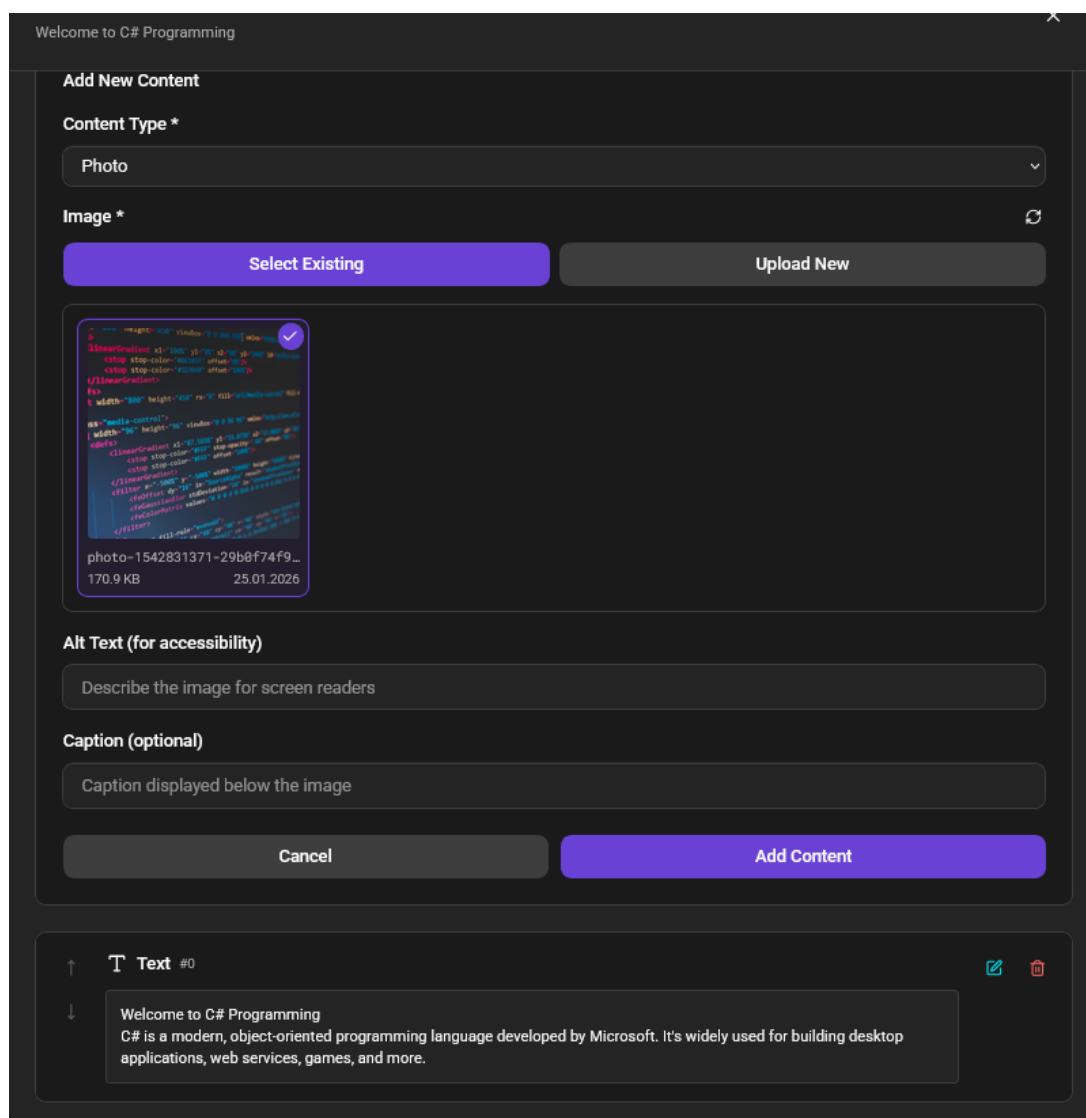
Rysunek 7.20: Widok dodania nowego wykładu

Następnie, aby uzupełnić treść wykładu, konieczne jest znalezienie go na liście i wybranie niebieskiej ikonki edytowania treści. Wyświetli się formularz, przedstawiony na Rys. 7.21. Po kliknięciu przycisku *Add content* mamy możliwość dodania do niego treści – tekstu, w formacie HTML, zdjęć lub filmów. Aby dodać zdjęcia oraz filmy, można wybrać je z tych już dodanych do bazy zdjęć, albo dodać do niej nowe (Rys. 7.22).



Rysunek 7.21: Widok edytowania zawartości wykładu

7.3. PANEL ADMINISTRATORA



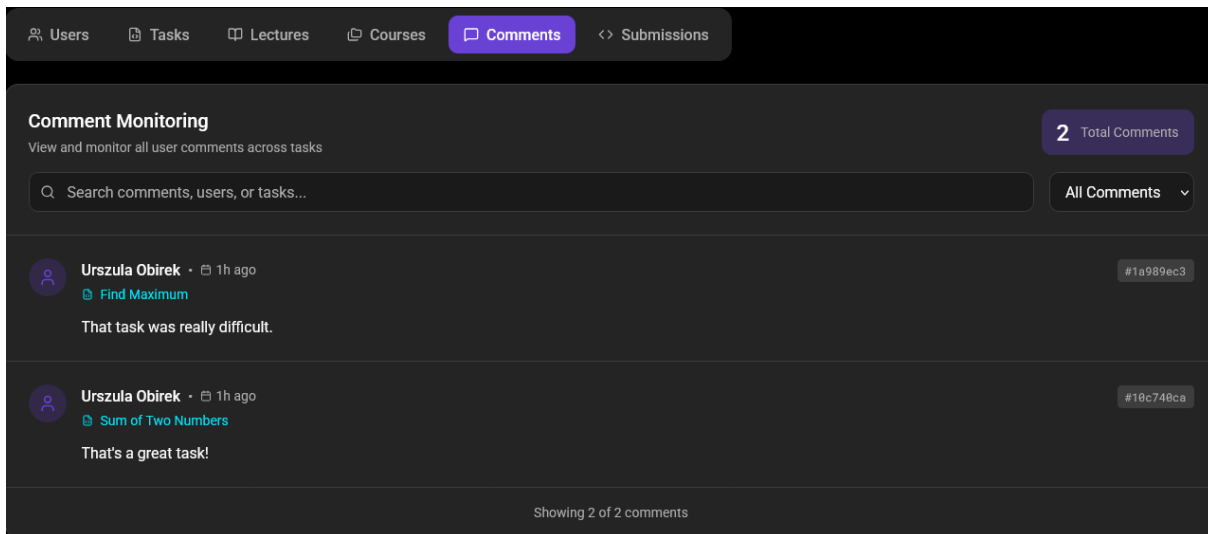
Rysunek 7.22: Widok edytowania zawartości wykładu – dodanie nowego zdjęcia

7.3.3. Monitorowanie komentarzy

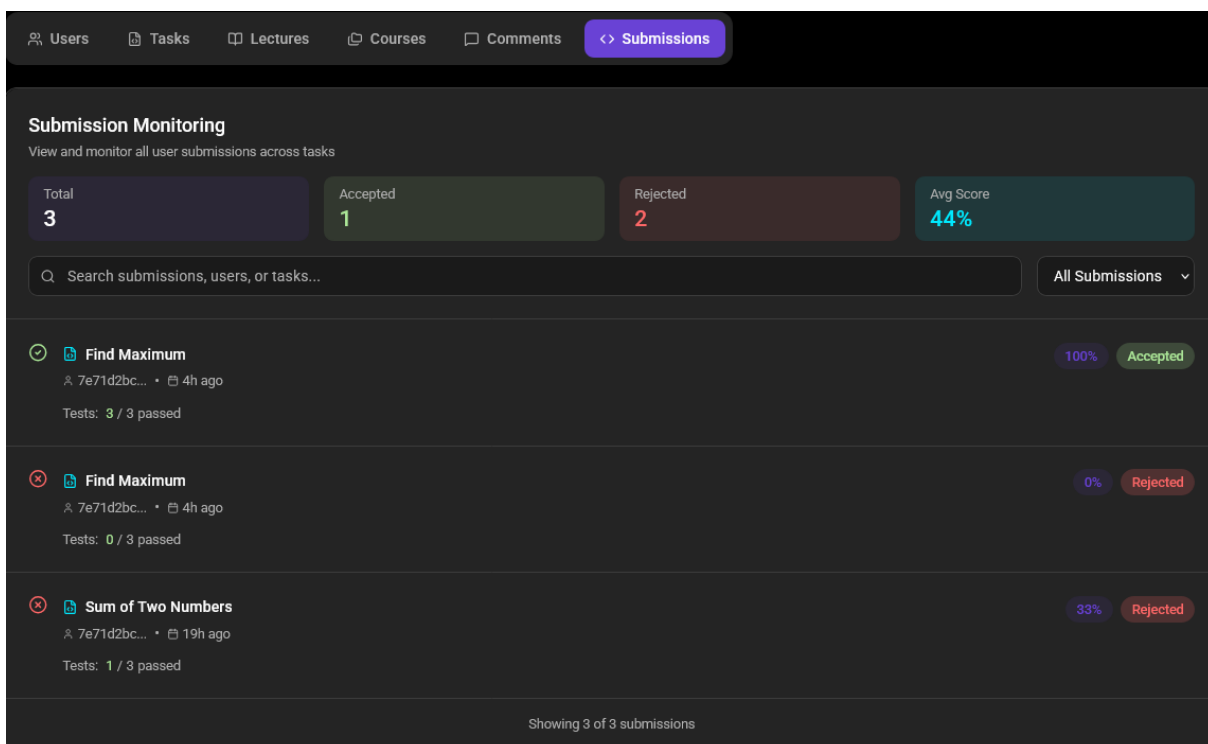
Administrator ma możliwość monitorowania umieszczonych na platformie komentarzy. Może je filtrować, korzystając z paska nad listą komentarzy.

7.3.4. Monitorowanie zgłoszeń rozwiązań

Administrator może również, korzystając z zakładki *Submissions*, przeglądać dotychczasowe zgłoszenia rozwiązań zadań i wyświetlić statystyki na ich temat.



Rysunek 7.23: Widok edytowania zawartości wykładu – dodanie nowego zdjęcia



Rysunek 7.24: Widok monitorowania zgłoszonych rozwiązań zadań

8. Podsumowanie

W ramach niniejszej pracy zaprojektowano i zaimplementowano aplikację AlgoRhythm – interaktywną platformę e-learningową dedykowaną nauce algorytmów i struktur danych.

Przed rozpoczęciem prac projektowych przeanalizowano istniejące na rynku rozwiązania, skupiając się na określeniu ich mocnych stron oraz obszarów wymagających udoskonalenia. Wiele z analizowanych platform specjalizuje się wyłącznie w jednym aspekcie. Przykładowo, oferowały rozbudowany system rozwiązywania zadań programistycznych, ale brakowało w nich możliwości zapoznania się z wiedzą teoretyczną. Przez to użytkownik, który chciałby uczyć się algorytmów i struktur danych, mógłby czuć potrzebę ciągłego przełączania się między różnymi narzędziami i wyszukiwania nowych. Ze względu na to jako cel projektu obrano stworzenie kompletnego narzędzia, pozwalającego na efektywną i komfortową naukę w jednym miejscu. Dzięki ograniczeniu tematyki do algorytmów i struktur danych udało się zaprojektować kompletną platformę. Powstałe rozwiązanie łączy w sobie materiały teoretyczne w formie wykładów, praktyczne zadania programistyczne z możliwością bezpiecznego i szybkiego sprawdzania i analizy pisanego kodu w czasie rzeczywistym oraz interaktywne i przejrzyste wizualizacje algorytmów, tworząc spójne środowisko edukacyjne wspierające kompleksowy proces nauki.

System został zbudowany w oparciu o nowoczesne technologie webowe: ASP.NET Core 9.0 po stronie serwera API, React 19.1.1 z TypeScript po stronie platformy internetowej oraz Microsoft SQL Server 2022 jako bazę danych. Architektura oparta na mikroservisach z wykorzystaniem konteneryzacji Docker zapewnia modularność, skalowalność oraz łatwość wdrożenia w środowisku chmurowym.

8.1. Ewolucja koncepcji systemu

Przed rozpoczęciem pisania kodu przygotowano dokładne specyfikacje funkcjonalności. Podzielono je pod względem priorytetów i zgodnie z tym rozkładem wdrażano do aplikacji. Stworzono także szczegółowy diagram klas, który ułatwił dalszą ich implementację oraz stworzenie relacji pomiędzy nimi w bazie danych.

Dzięki dokładnemu zaplanowaniu architektury bardzo rzadko pojawiała się potrzeba przebudowy założeń aplikacji. Jedyne większe zmiany jakie zachodziły były spowodowane chęcią ułatwienia korzystania z aplikacji. Przykładem może być zmiana koncepcji wykładu, aby mógł istnieć samodzielnie poza kursem i nie musiał być przypisany wyłącznie do jednego na raz.

Kolejnym systemem, który został uproszczony była hierarchia pozwoleń. Początkowo planowane było rozbieżne uprawnienia, aby różne akcje, które nie powinny być dostępne dla zwykłego użytkownika, miały swoje własne zezwolenia. W takiej formie różni użytkownicy mogliby mieć dostęp, np. do modyfikacji materiałów naukowych, a inni tylko do zarządzania innymi użytkownikami. Po dłuższych przemyśleniach zdecydowano się zastąpić to systemem ról, ponieważ przy takiej ilości funkcjonalności byłoby to niepotrzebnie skomplikowane do nadzorowania. Ponadto, wiele uprawnień naturalnie można byłoby udzielać na raz i bardzo rzadko byłaby potrzeba, by na przykład nadać komuś prawa tylko do nadawania innym uprawnień, a nie do modyfikacji zadań.

8.2. Osiągnięcia projektu

Kluczowym osiągnięciem projektu jest implementacja systemu zdalnego wykonywania kodu użytkowników. Wykorzystanie kompilatora Roslyn oraz izolacji na poziomie kontenerów Docker pozwoliło na stworzenie wydajnego i bezpiecznego środowiska, w którym użytkownicy mogą testować swoje rozwiązania bez ryzyka dla infrastruktury systemu i prywatności danych. Zaprojektowany mechanizm load balancingu umożliwia obsługę wielu użytkowników jednocześnie, a możliwość łatwego skalowania liczby kontenerów wykonawczych pozwala na dostosowanie wydajności do aktualnych potrzeb.

Istotnym elementem jest również zaawansowany edytor kodu, w którym udało się wprowadzić analizę błędów w czasie rzeczywistym. Oprócz znajomego z innych popularnych edytorów wyglądu, posiada on również opcję podpowiedzi składniowych. Ułatwia ona wyszukiwanie nazw funkcji, typów, jak również zmiennych wykorzystanych już w kodzie.

Jednym z nowszych dodatków, wyróżniających się w aplikacji, jest system wizualizacji algorytmów. Początkowo rolę tę miały wykonywać pliki .GIF i filmy będące częścią wykładów, jednak ostatecznie wprowadzono zupełnie nową zakładkę z dedykowanym interfejsem. Pomimo względnej prostoty obsługi, przygotowano krótki samouczek wprowadzający, który po kolei tłumaczy działanie poszczególnych elementów. Po utworzeniu grafu i wybraniu wierzchołków aplikacja pokazuje krok po kroku działanie zaimplementowanego algorytmu.

Interfejs użytkownika wyróżnia się nowoczesnym designem i dobrze przemyślanym zagospodarowaniem przestrzeni. Dzięki spójnemu systemowi kolorystycznemu przechodzenie między sek-

cjami aplikacji jest bardzo płynne i naturalne. Zaimplementowane żywsze kolory sprawiają, że platforma dystansuje się od tradycyjnego, mało ciekawego wyglądu większości stron edukacyjnych, nie będąc przy tym zbyt przytłaczająca.

8.3. Kierunki dalszego rozwoju

Stworzone rozwiązanie stanowi solidną podstawę do dalszego rozwoju. Modułarna architektura oraz czytelna struktura kodu ułatwiają implementację nowych funkcjonalności, takich jak dodatkowe języki programowania, nowe role czy większa paleta osiągnięć. Platforma jest gotowa do wdrożenia i może stanowić wartościowe narzędzie wspierające proces nauczania algorytmów i struktur danych na poziomie akademickim.

Na moment obecny dostępne są dwie role: administrator i użytkownik. Administrator ma dostęp do specjalnego panelu z możliwością zarządzania innymi użytkownikami, materiałami oraz z podglądem wszystkich komentarzy i przesłanych rozwiązań. Jako dalszą możliwość rozwoju mogłaby powstać nowa rola – na przykład nazwana edytor – która umożliwiałaby dodawanie, usuwanie i modyfikację materiałów dydaktycznych, ale pozwalałaby zarządzać pozostałymi użytkownikami. Gdyby taka funkcjonalność została dodana, a aplikację zaczęto by wykorzystywać do prowadzenia zajęć na uczelni, miałaby ona wiele zastosowań. W takim przypadku prowadzący pełniłby rolę administratora i mógłby chętnym studentom powierzyć rolę edytora. Mogliby oni aktualizować oraz zarządzać materiałami udostępnionymi na platformie bez obaw o nadużycia.

Kolejnym obszarem, w którym możliwy byłby rozwój aplikacji, są zadania interaktywne. Obecnie przygotowane są już fundamenty potencjalnej implementacji, ułatwiające przyszłe wdrożenie. Docelowo byłyby to dodatkowe zadania w kursach, sprawdzające wiedzę, bez pisania kodu. Przykładami mogłyby być zadania wielokrotnego wyboru, czy z bardziej zaawansowanych, umożliwiające wstawianie i usuwanie elementów do drzewa BST. Aktualna implementacja wizualizacji może zostać w prosty sposób rozbudowana, aby obsługiwać łatwe tworzenie drzewa i następne sprawdzanie poprawności rozwiązania.

Sama sekcja wizualizacji ma potencjał przy relatywnie niewielkim nakładzie pracy zostać poszerzona o kolejne struktury danych i operacje na nich. W celu zwiększenia atrakcyjności platformy możliwe jest też dodanie do niej kolejnych funkcjonalności, takich jak zapisywanie i udostępnianie innym użytkownikom swoich animacji. Możliwe byłoby załączanie ich do materiałów teoretycznych przez administratora. Stworzyłoby to dodatkową przestrzeń do dyskusji nad zadaniami i ich rozwiązaniami.

8.4. Wnioski

W ramach pracy nad projektem udało się stworzyć kompletną platformę do nauki, zawierającą zarówno materiały teoretyczne, zadania praktyczne oraz wizualizacje algorytmów. Dokładne zaplanowanie architektury i rozsądne podzielenie priorytetów pozwoliły na efektywne wykorzystanie czasu i stabilne rozbudowanie platformy.

Projektowanie funkcjonalności zdalnego wykonywania kodu okazało się szczególnie wartościowym wyzwaniem. Połączenie wydajności oraz bezpieczeństwa wymagało dokładnego zgłębienia tematu nie tylko mechanizmów wykonywania kodu, ale również konteneryzacji.

Platforma AlgoRhythm stanowi solidny fundament do dalszego rozwoju i może realnie wspierać proces nauczania algorytmów i struktur danych na poziomie akademickim. Modułarna architektura oraz przemyślana struktura kodu umożliwiają łatwe rozszerzanie funkcjonalności i dalszą rozbudowę aplikacji.

Bibliografia

- [1] A.V. Aho, R. Sethi, J.D. Ullman, *Kompilatory: Reguły, metody, narzędzia*, Wydanie II, Warszawa 2019
- [2] Haohui Liang et al, *Design and implementation of online automatic judging system*, online: <https://iopscience.iop.org/article/10.1088/1755-1315/69/1/012091/pdf>, IOP Conf. Ser.: Earth Environ. Sci 2017
- [3] Microsoft, *ASP.NET Core 9.0 Documentation*, Microsoft Learn, online: <https://learn.microsoft.com/aspnet/core/?view=aspnetcore-9.0>, dostęp: 12.01.2026.
- [5] Microsoft, *Azure Blob Storage Documentation*, Microsoft Learn, online: <https://learn.microsoft.com/azure/storage/blobs/>, dostęp: 13.01.2026.
- [8] Microsoft, *Entity Framework Documentation*, Microsoft Learn, online: <https://learn.microsoft.com/en-us/ef/>, dostęp: 30.01.2026
- [7] Microsoft, *SignalR Documentation*, Microsoft Learn, online: <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-10.0>, dostęp: 30.01.2026
- [6] Microsoft, *SQL Server 2022 Documentation*, Microsoft Learn, online: <https://learn.microsoft.com/sql/sql-server/?view=sql-server-ver16>, dostęp: 26.01.2026.
- [4] React Documentation, *React 19.1.1 Documentation*, online: <https://reactjs.org/docs/getting-started.html>, dostęp: 13.01.2026.
- [9] Scheepers, Mathijs Jeroen, *Virtualization and Containerization of Application Infrastructure: A Comparison*, online: <https://web.archive.org/web/20240928043944/https://thijs.ai/papers/scheepers-virtualization-containerization.pdf>, University of Twente Enschede, The Netherlands 2014
- [10] Yutaka Watanobe, Md. Mostafizer Rahman, Taku Matsumoto, Uday Kiran Rage, and Penugonda Ravikumar, *Online Judge System: Requirements, Architecture, and Experiences*,

International Journal of Software Engineering and Knowledge Engineering, online: <https://www.worldscientific.com/doi/epdf/10.1142/S0218194022500346>, 9.07.2022

- [11] G. Zacharis, K. Nikolopoulou, *Factors predicting University students' behavioral intention to use eLearning platforms in the post-pandemic normal: an UTAUT2 approach*, Education and Information Technologies, 2022.

Wykaz symboli i skrótów

API	Application Programming Interface
ASP.NET	Active Server Pages Network Enabled Technologies
CI/CD	Continuous Integration/Continuous Deployment
CIL	Common Intermediate Language
DTO	Data Transfer Object
GCC	GNU Compiler Collection
GNU	GNU's not UNIX
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
NGINX	engine-x
PHP	Hypertext Preprocessor
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

Spis rysunków

3.1	Widok głównej strony LeetCode	20
3.2	Widok zadania programistycznego strony LeetCode	20
3.3	Widok wizualizacji na stronie LeetCode	21
3.4	Widok zadania programistycznego na stronie HackerRank	22
3.5	Widok części z animacji dostępnych na stronie VisuAlgo	22
3.6	Widok animacji na stronie VisuAlgo	23
3.7	Widok animacji na stronie Uniwersytetu w San Francisco	23
4.1	Schemat architektury	25
4.2	Diagram klas – część 1	35
4.3	Diagram klas – część 2	36
4.4	Struktura katalogów projektu	43
4.5	Wybór konfiguracji startowej w Visual Studio	43
4.6	Przykładowa informacja z konsoli o poprawnym uruchomieniu kontenerów	44
4.7	Logowanie do serwera bazy danych – cz.1	44
4.8	Logowanie do serwera bazy danych – cz.2	45
4.9	Przykładowa informacja z konsoli o poprawnym uruchomieniu kontenerów	46
4.10	Komunikacja między komponentami przy pobieraniu opublikowanych zadań . . .	49
5.1	Uproszczony schemat sekwencji wykonywania kodu – cz.1 (interfejs i backend) . .	57
5.2	Uproszczony schemat sekwencji wykonywania kodu – cz.2 (sandbox)	57
6.1	Testy UI logowania	66
6.2	Testy end-to-end wykonywania kodu	67
6.3	Wyniki testów wydajnościowych	68
6.4	Raport z pokrycia kodu testami panelu administratora	69
6.5	Raport z pokrycia kodu testami wykonywania kodu	69
6.6	Raport z pokrycia kodu testami kluczowych serwisów	69

7.1	Widok rejestracji	70
7.2	Widok weryfikacji e-mail	71
7.3	Widok logowania	72
7.4	Dashboard	73
7.5	<i>Courses</i> – widok listy kursów	74
7.6	Widok kursu	74
7.7	Widok listy wykładów	75
7.8	Widok wykładu	76
7.9	Widok listy zadań	76
7.10	Widok zadania	77
7.11	Widok zadania – zakładka z dyskusją	77
7.12	Widok zadania – zakładka ze zgłoszeniami rozwiązań	78
7.13	Widok zadania po uruchomieniu kodu	78
7.14	Widok wizualizacji po uruchomieniu algorytmu	79
7.15	Widok profilu użytkownika	80
7.16	Widok edycji profilu	80
7.17	Widok panelu administratora	81
7.18	Widok zakładki z zadaniami w panelu administratora	82
7.19	Widok edytowania zadania – uzupełniony formularz	83
7.20	Widok dodania nowego wykładu	84
7.21	Widok edytowania zawartości wykładu	84
7.22	Widok edytowania zawartości wykładu – dodanie nowego zdjęcia	85
7.23	Widok edytowania zawartości wykładu – dodanie nowego zdjęcia	86
7.24	Widok monitorowania zgłoszonych rozwiązań zadań	86

Spis tabel

2.1	Przypadki użycia – aktor: administrator	16
2.2	Przypadki użycia – aktor: użytkownik, autoryzacja	16
2.3	Przypadki użycia – aktor: użytkownik, treści	17
3.1	Porównanie rozwiązań	24